

Monster **SCALE** Summit

What Flow Metrics Teaches Us About Designing Resilient Systems



Mourjo Sen
Senior Software Engineer

Booking.com

About Me



Booking.com

- **Senior Software Engineer** at Booking.com
- Building backend systems for **10 years**
- Writing about system resilience since 2020

Load Shedding in Web Services

10 min read · Sep 4, 2020 · [View story](#)

12.7K

Views

5.4K

Reads

-

Earnings

Killing me softly: Graceful shutdowns in Clojure

14 min read · Jan 27, 2020 · [View story](#)

8.1K

Views

2.3K

Reads

-

Earnings

mourjo.medium.com

Everything Fails. All the time.
– Werner Vogels

Metrics Should Help Solve Problems



PagerDuty APP 12:07 PM

Incident #1800: User Service API: Auth Response Time is High for prod - (95th percentile > 100 ms on average during the last 10m)

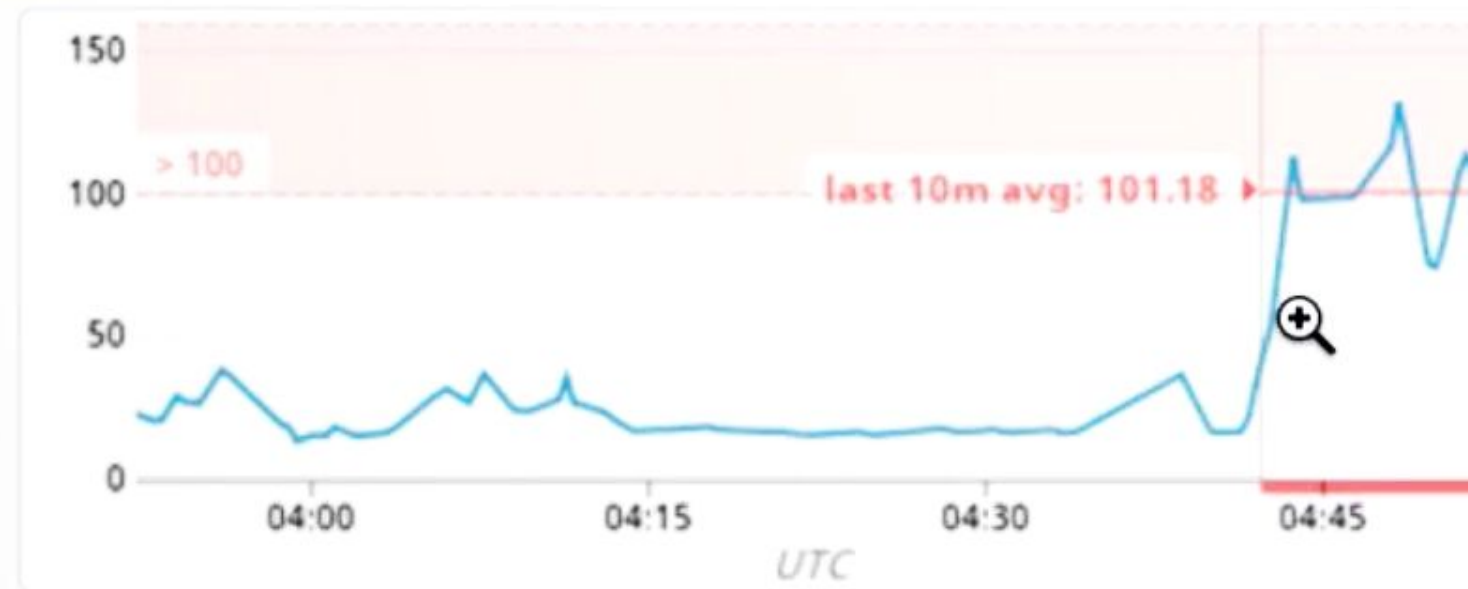
Assigned: Alexis Walsh

Priority: P1

Service: User Service API

Incident Description: User Service API: Auth Response Time is High for prod - (95th percentile > 100 ms on average during the last 10m)

Acknowledged by Alexis Walsh | Today at 12:07 PM (13 kB) ▾



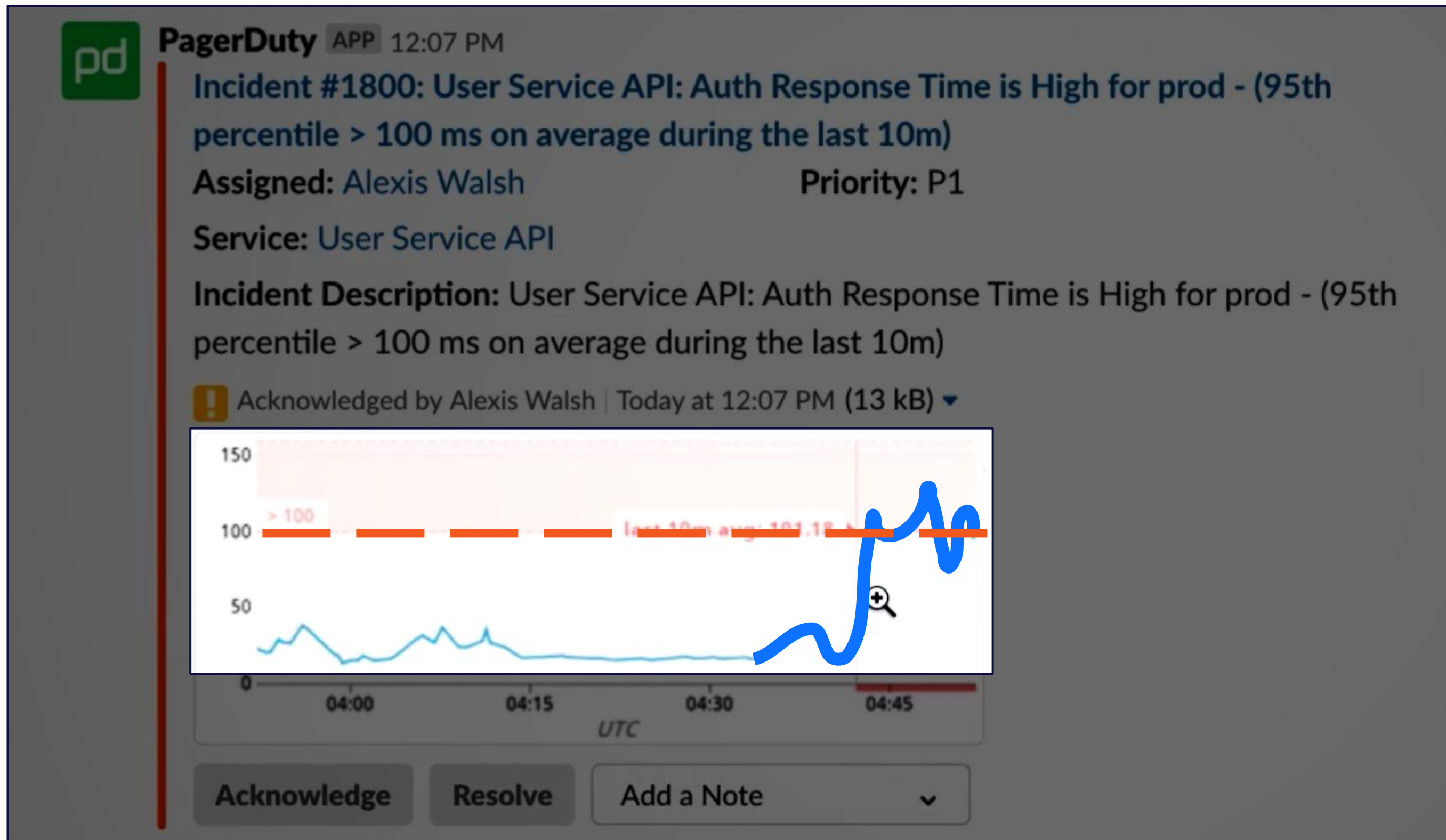
Acknowledge

Resolve

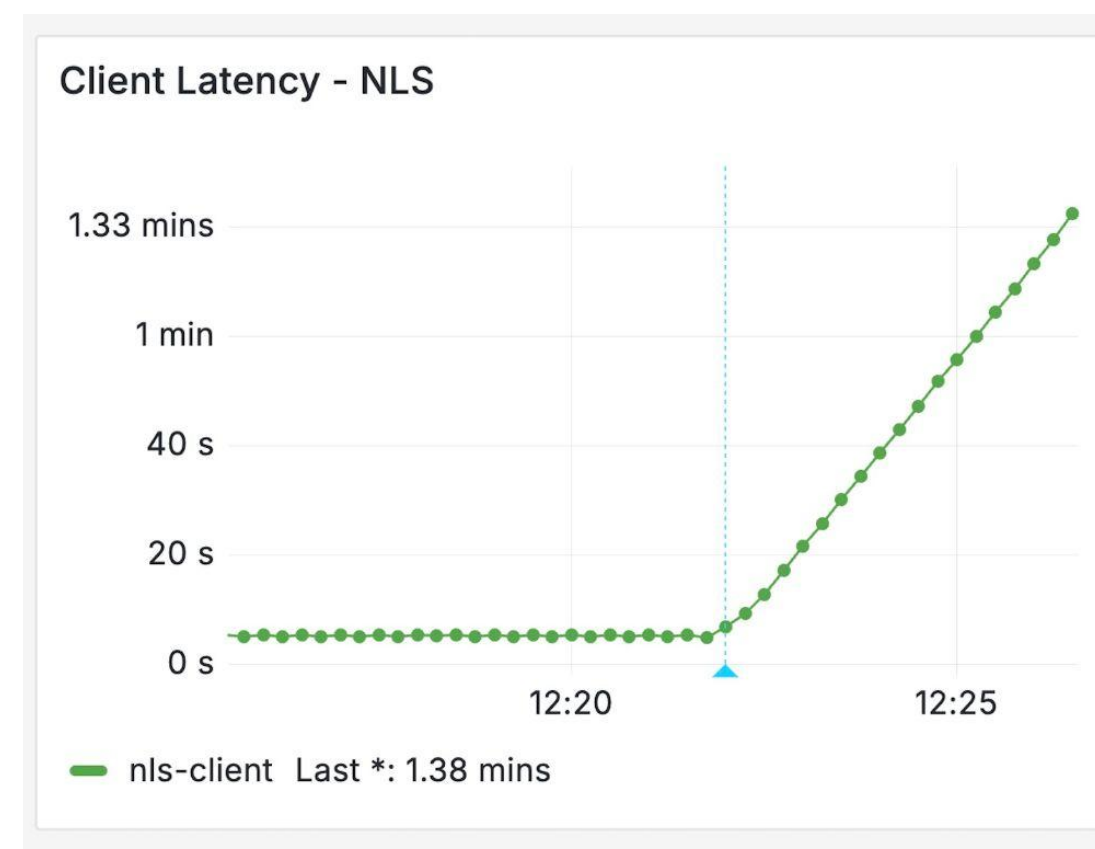
Add a Note



Metrics Should Help Solve Problems



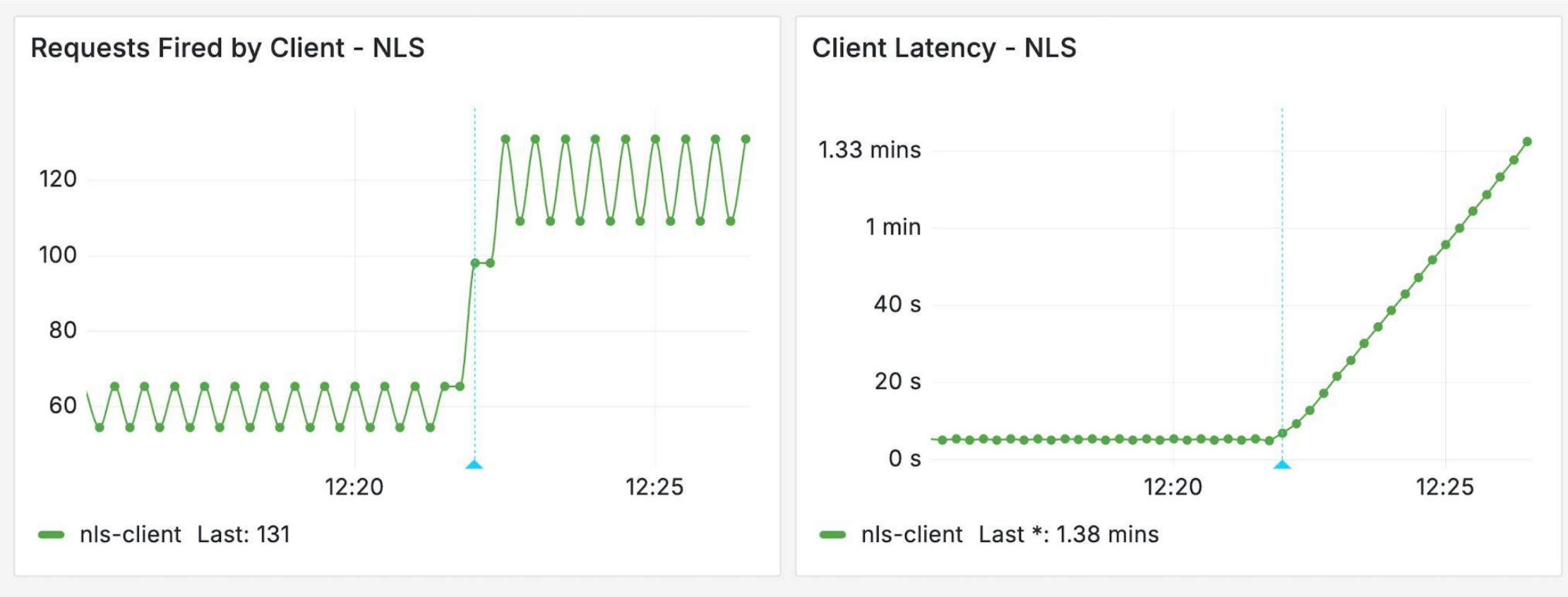
Metrics Should Help Solve Problems



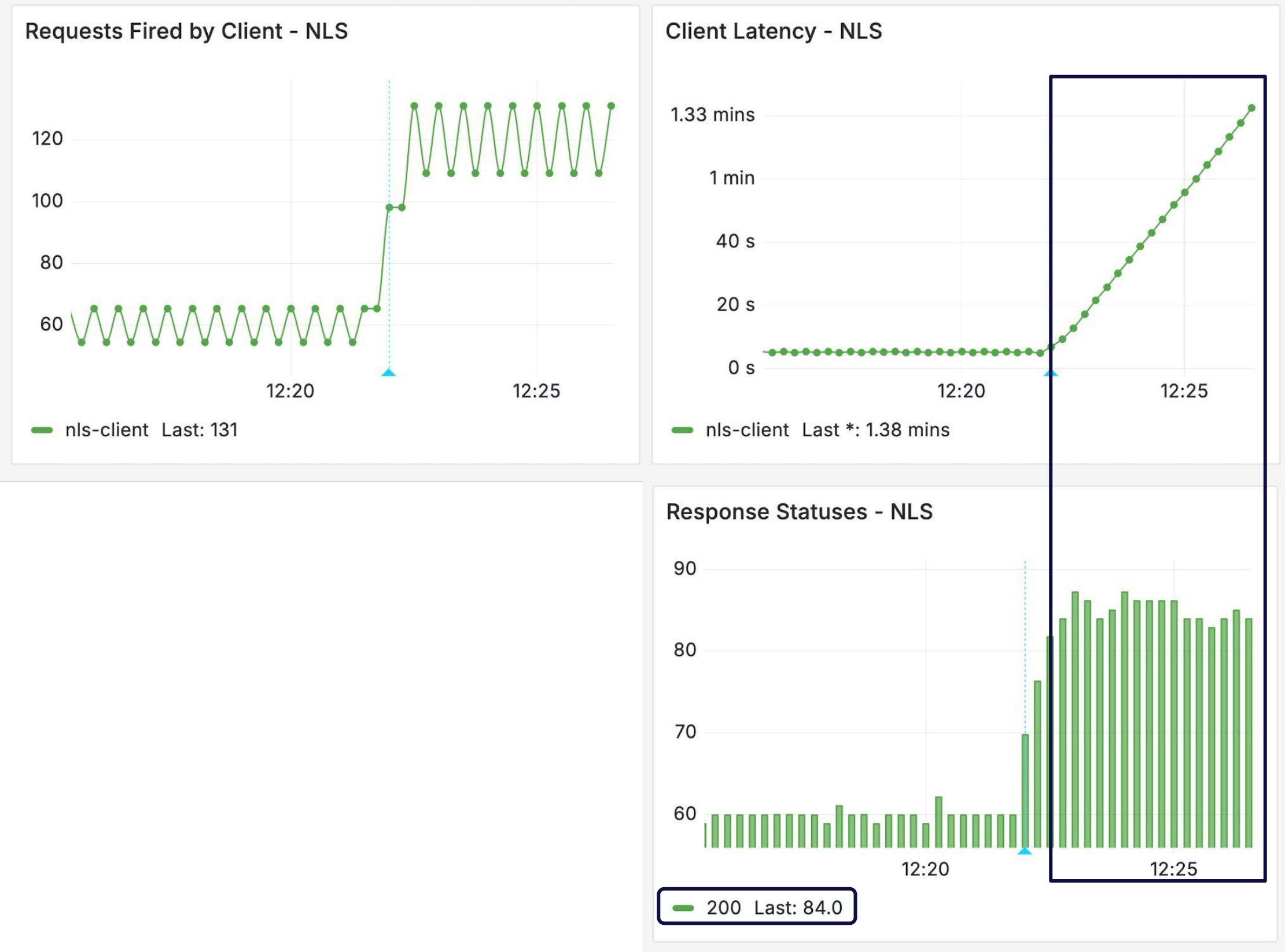
**Latency
increase
(impact)**

Metrics Should Help Solve Problems

Increase in
the **number**
of requests



Metrics Should Help Solve Problems



Everything is
okay
(200 status
codes)

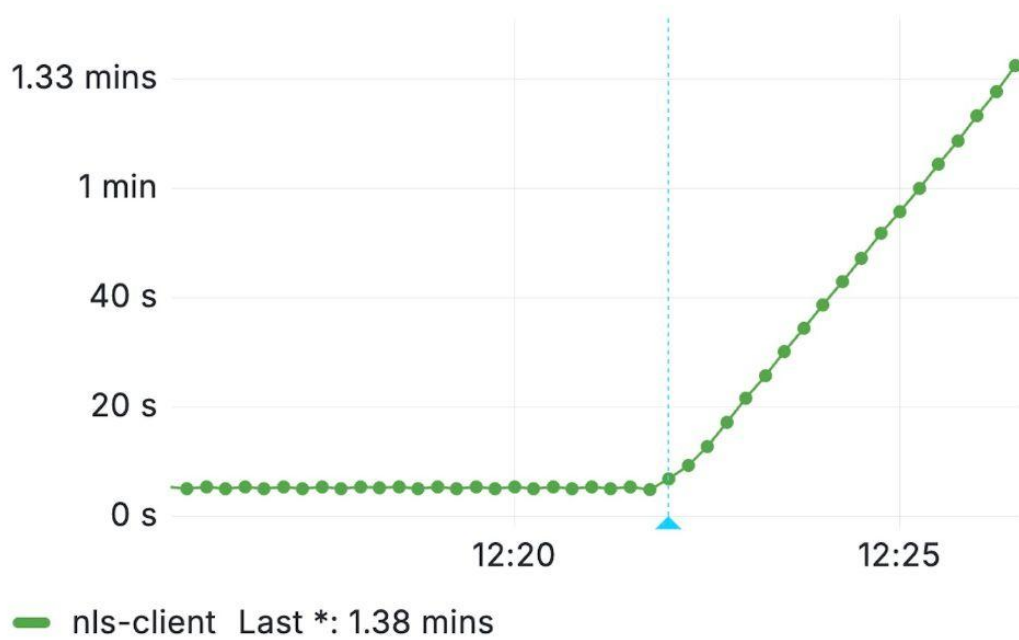
Metrics Should Help Solve Problems

No impact on
processing
time

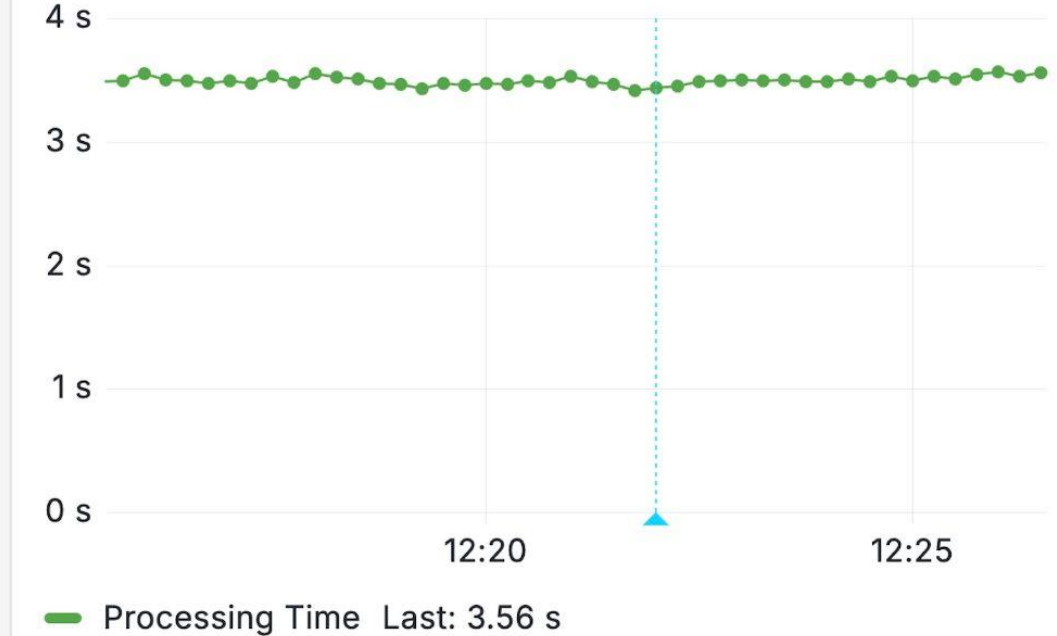
Requests Fired by Client - NLS



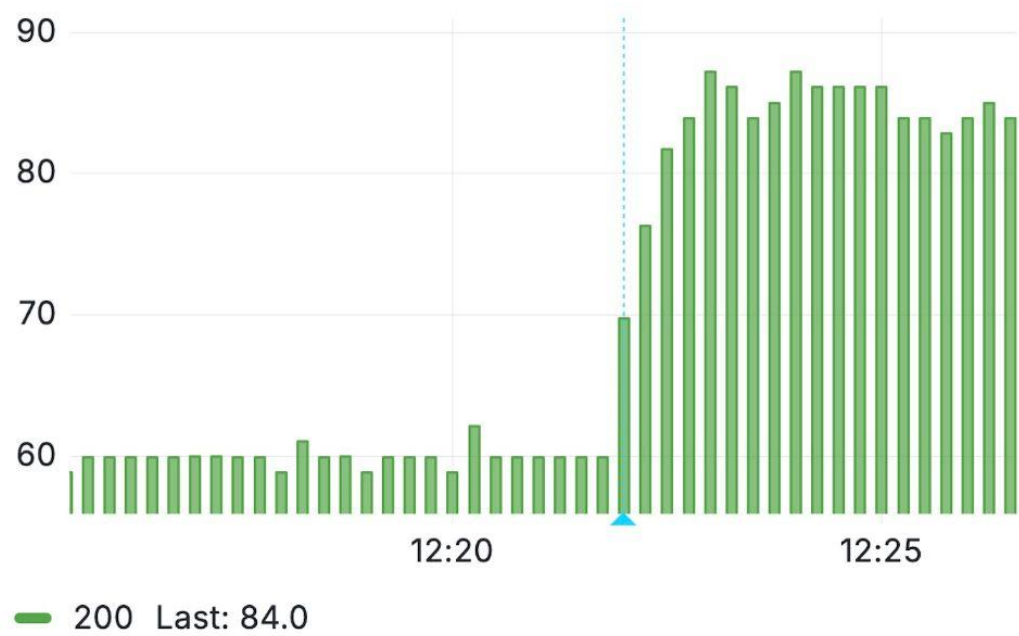
Client Latency - NLS



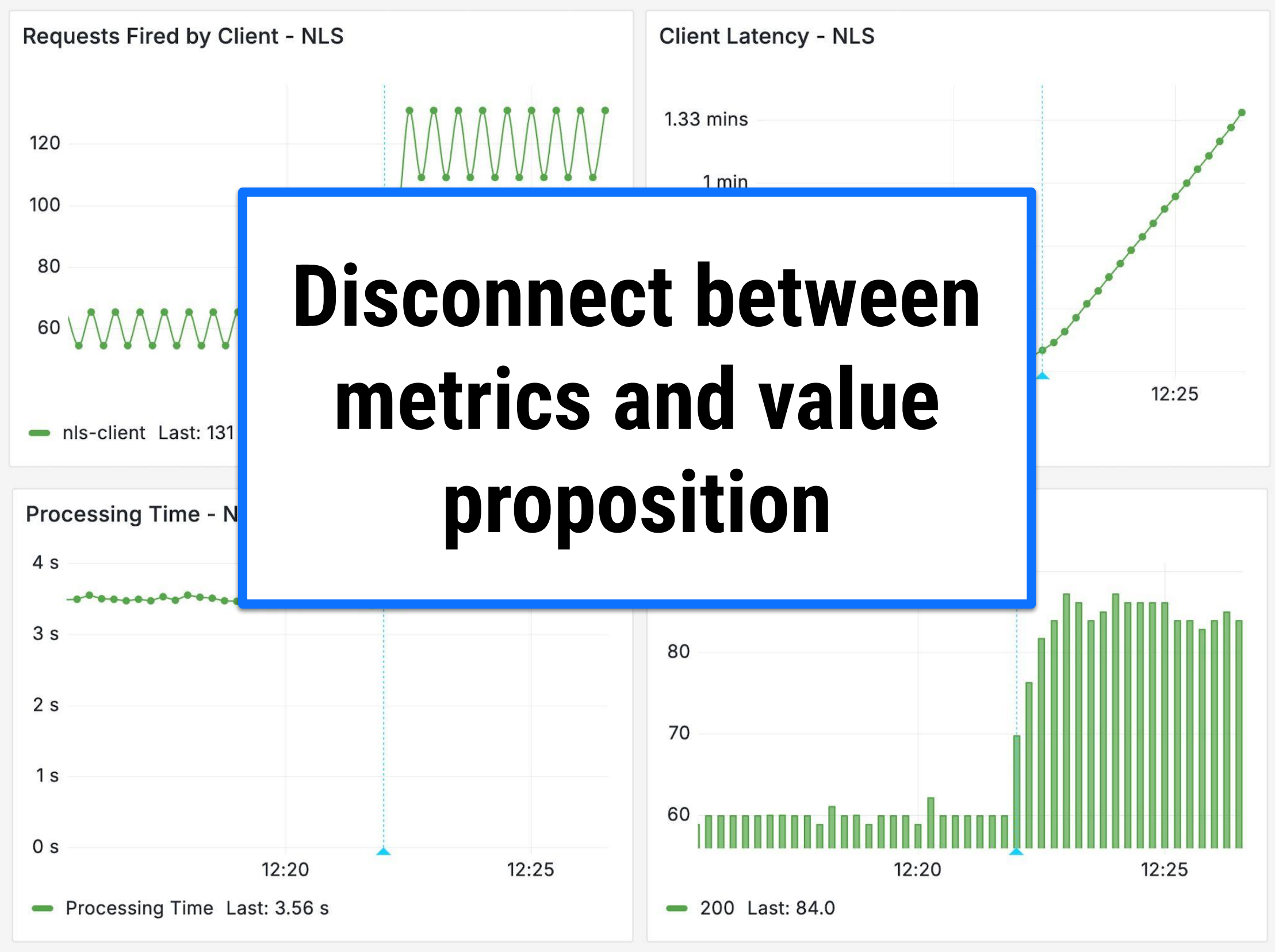
Processing Time - NLS



Response Statuses - NLS

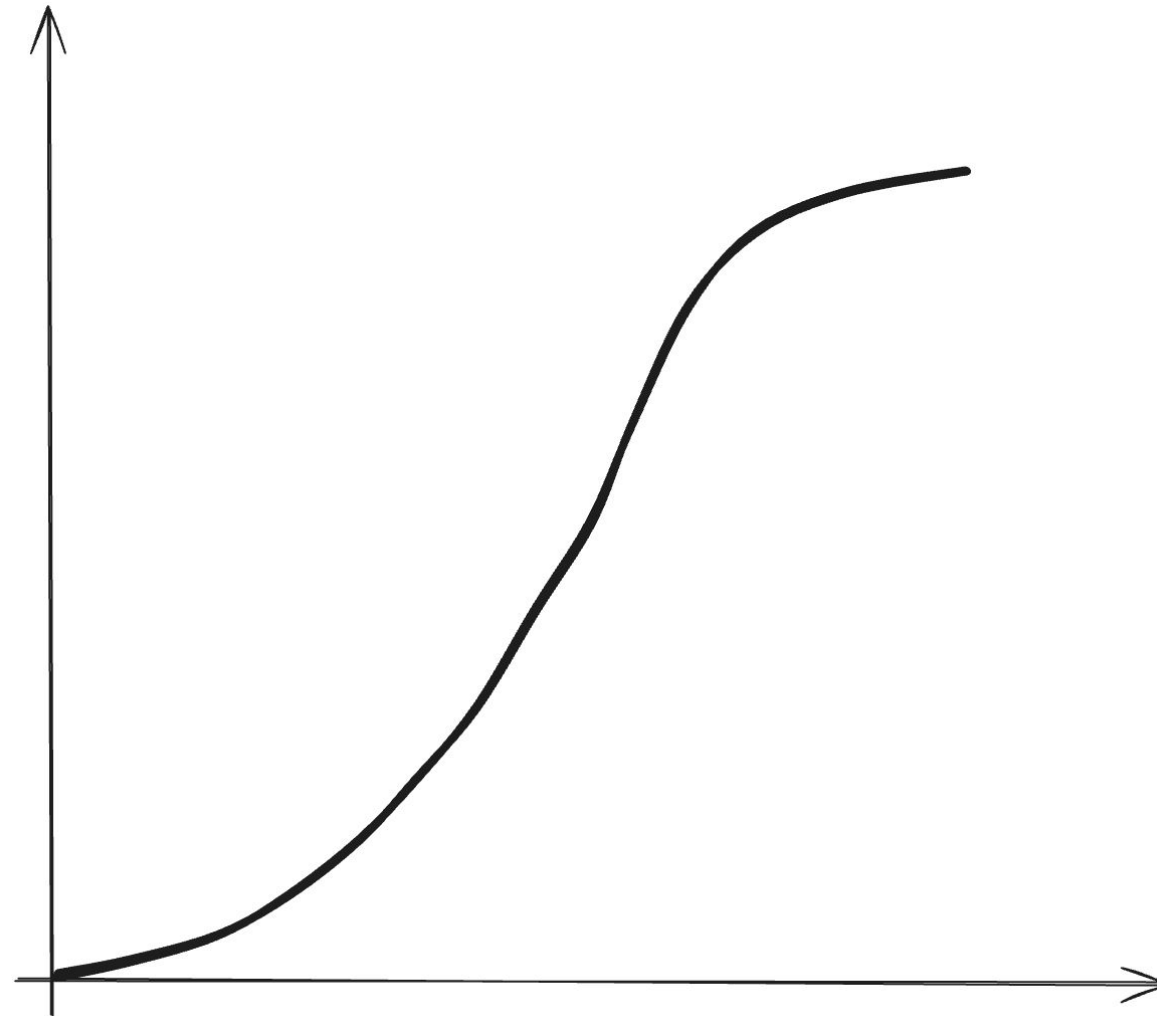


Metrics Should Help Solve Problems

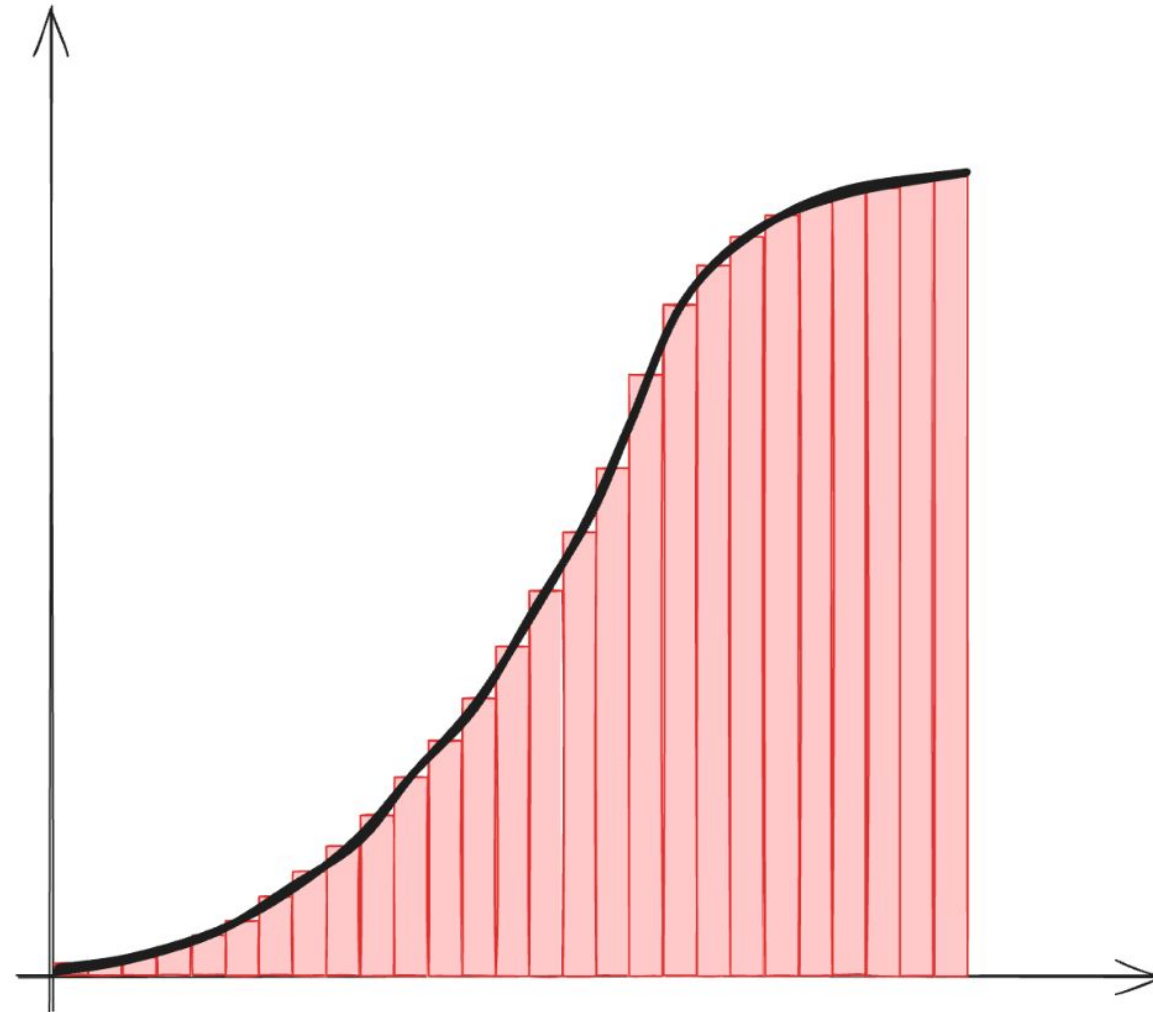


Software: A Means to Deliver Business Value

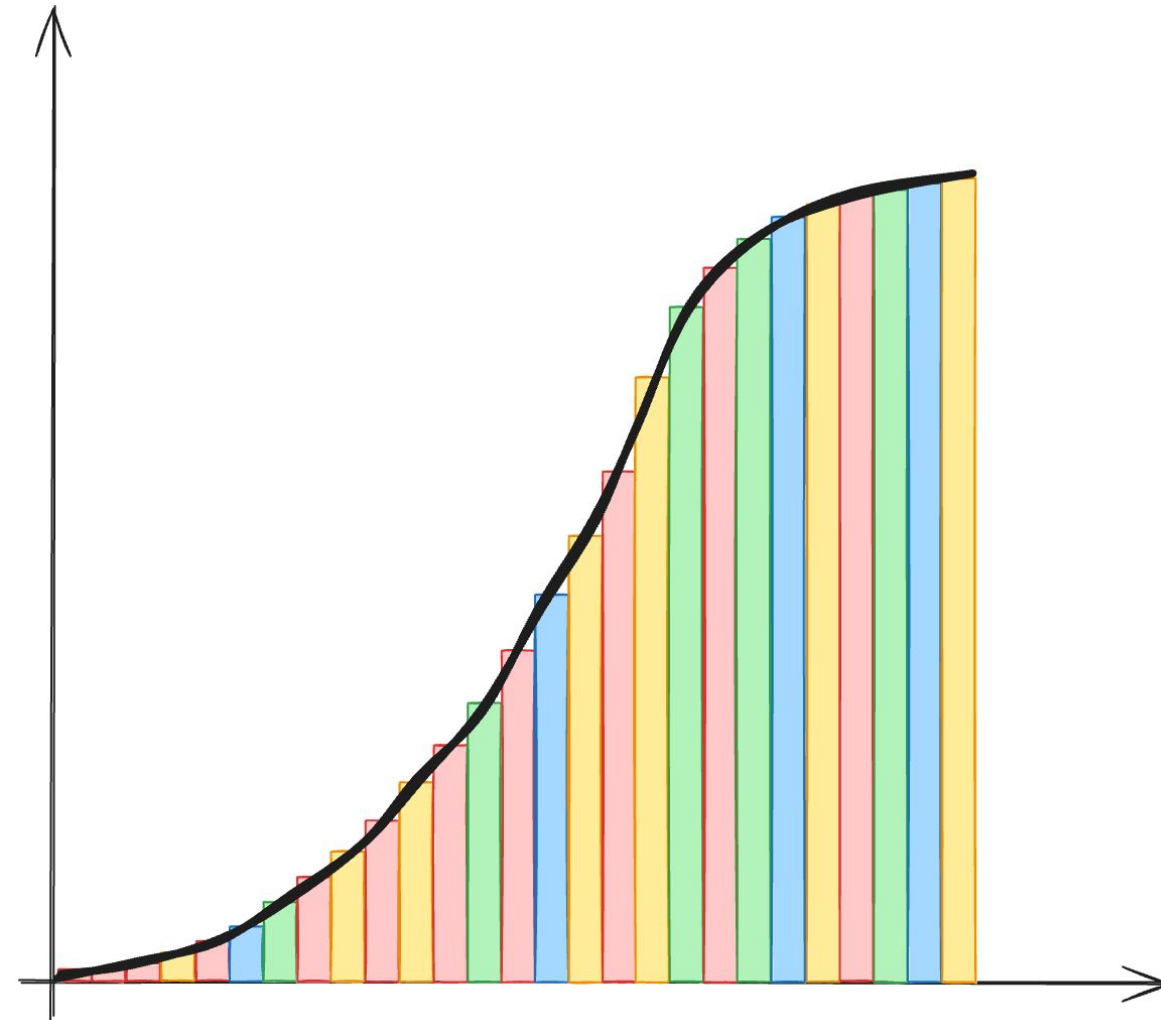
Building Software: An Analogy



Building Software: An Analogy

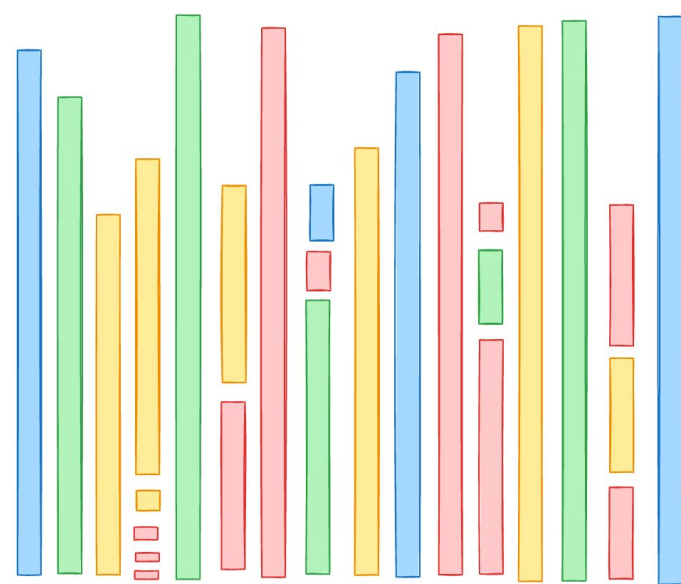


Building Software: An Analogy



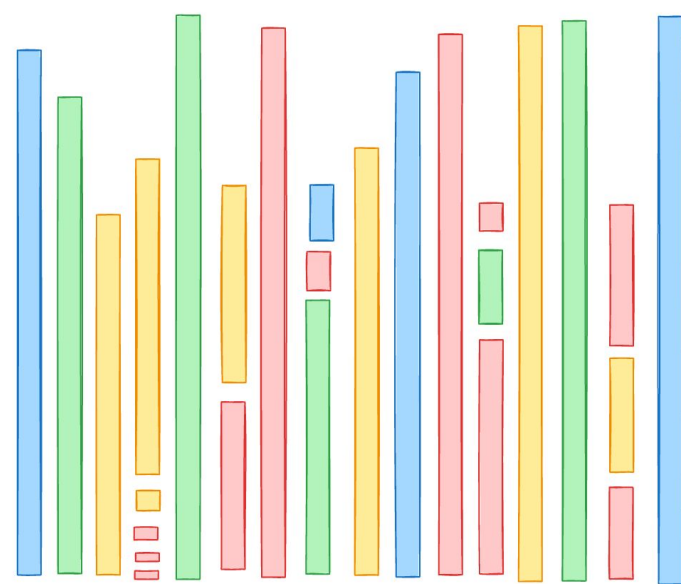
Building Software: An Analogy

Tasks to do

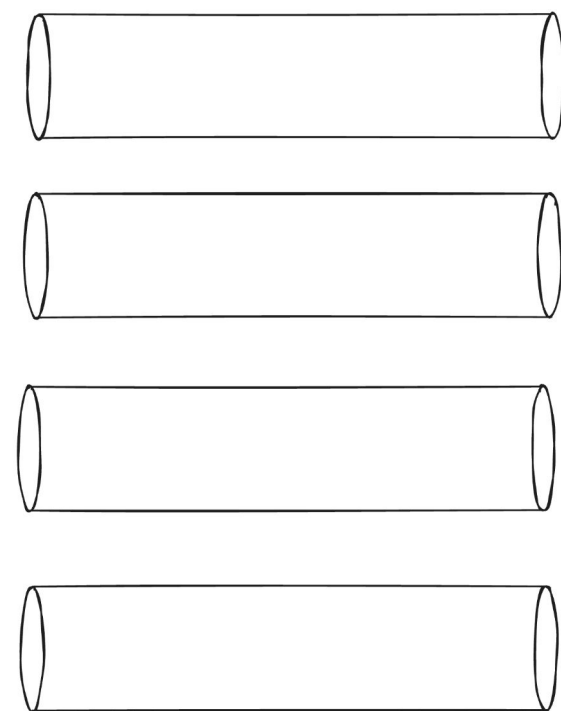


Building Software: An Analogy

Tasks to do

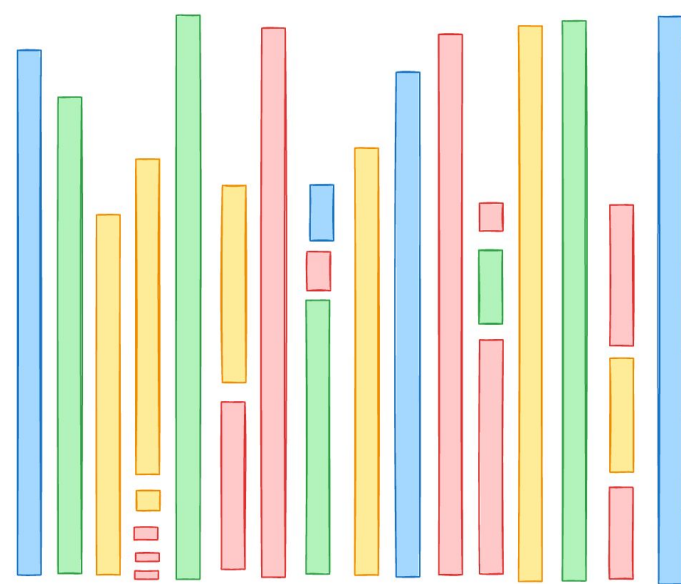


Teams work on
the tasks

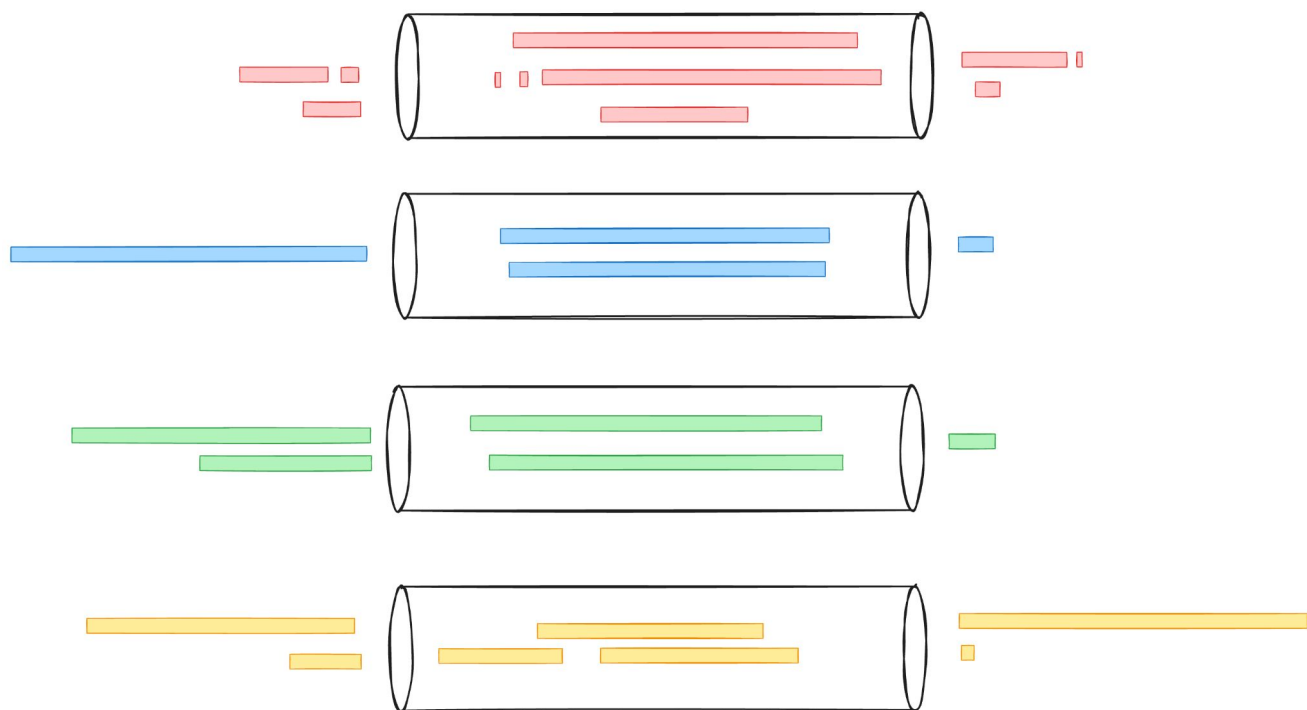


Building Software: An Analogy

Tasks to do

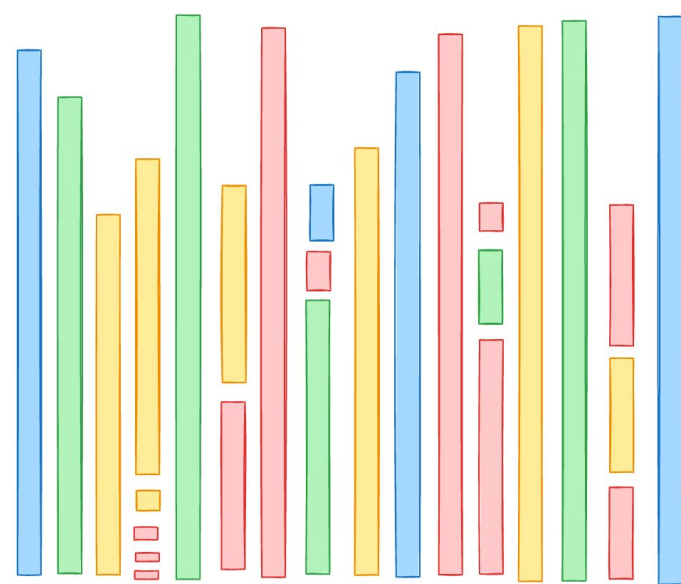


Teams work on the tasks

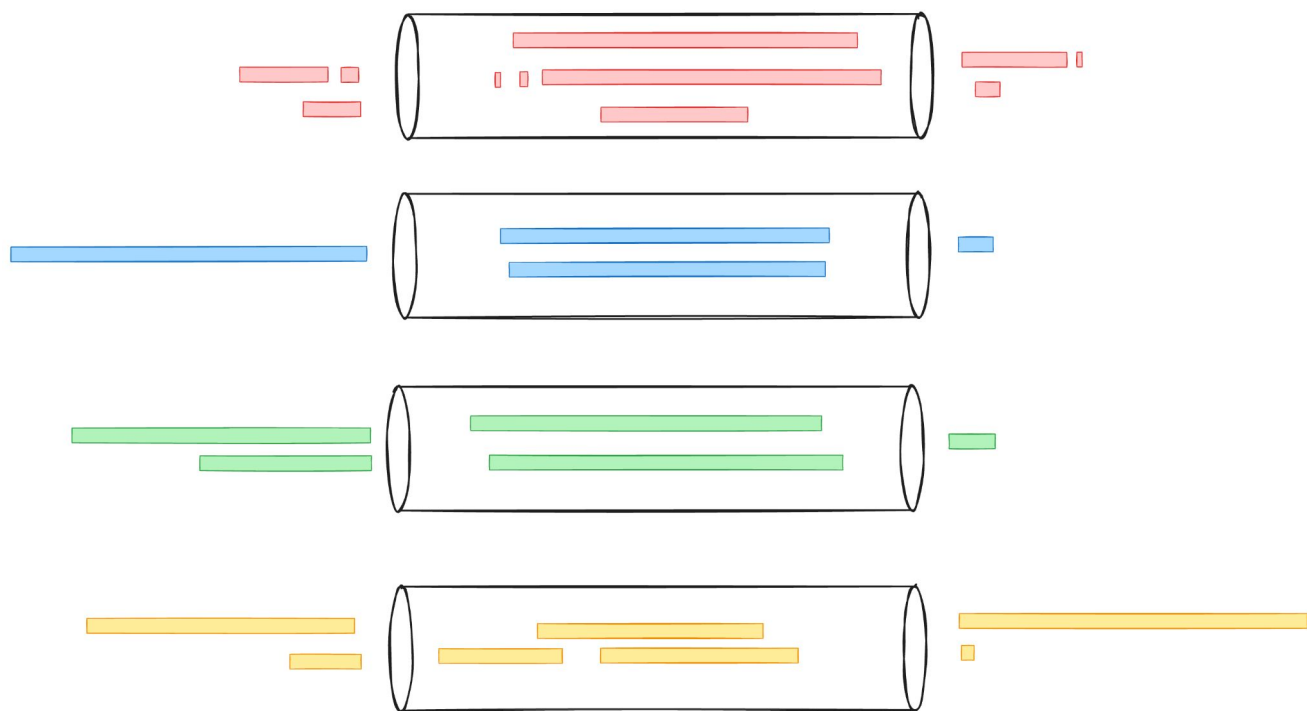


Building Software: An Analogy

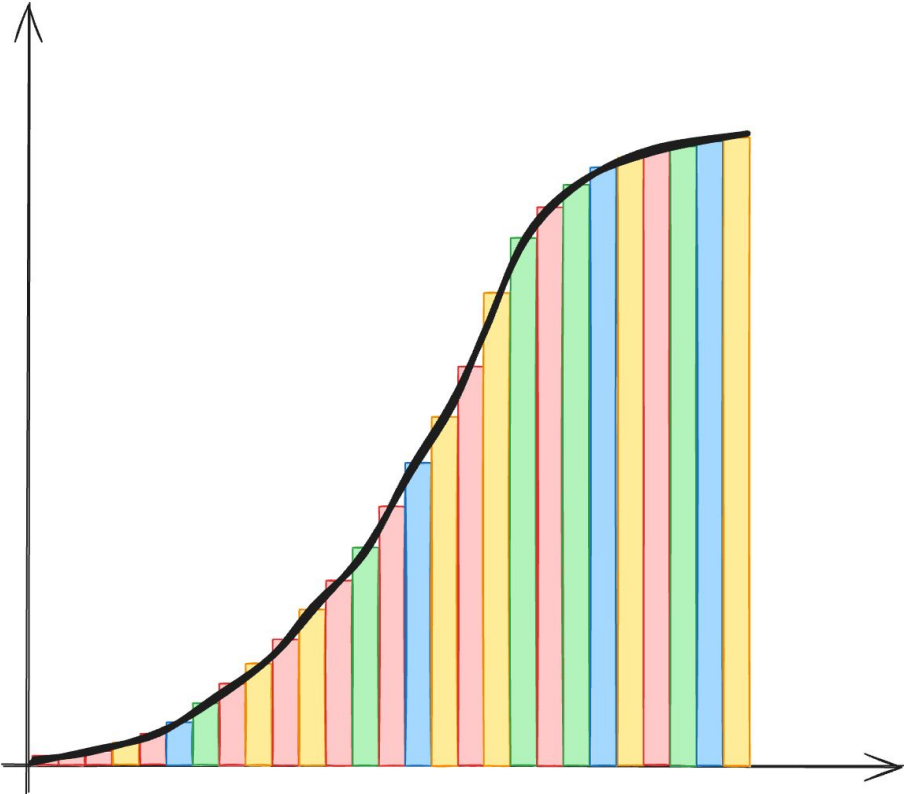
Tasks to do



Teams work on the tasks

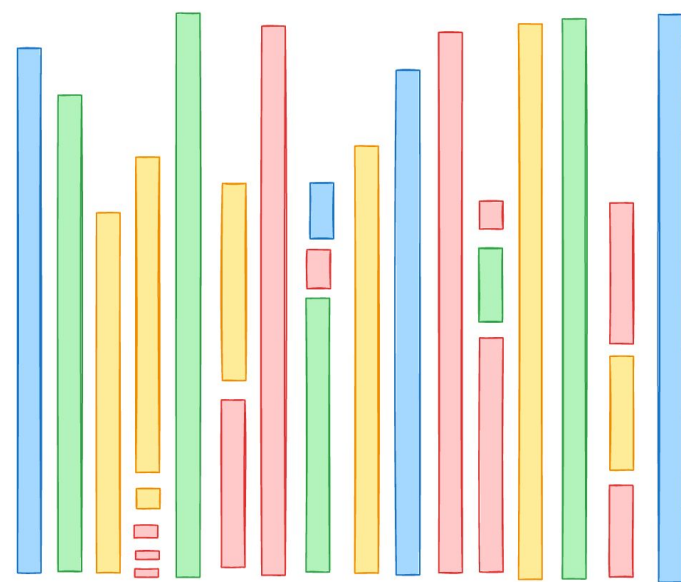


Assembled product

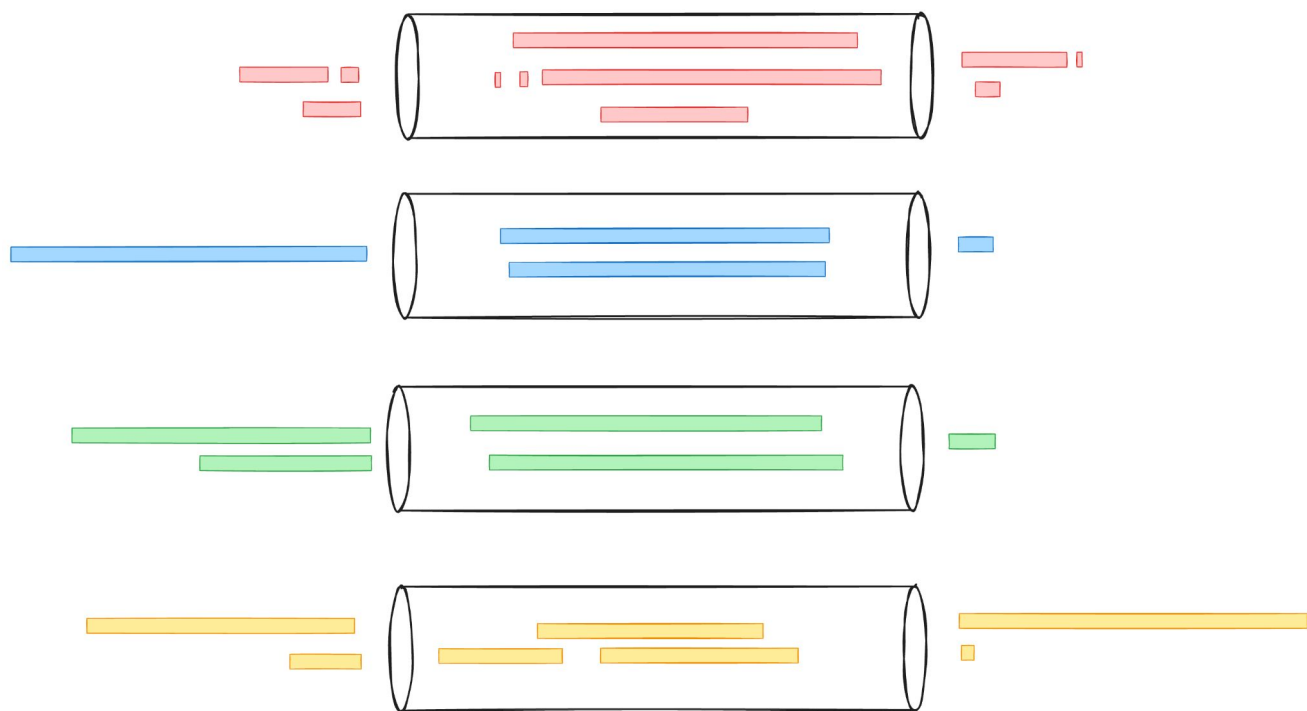


Building Software: An Analogy

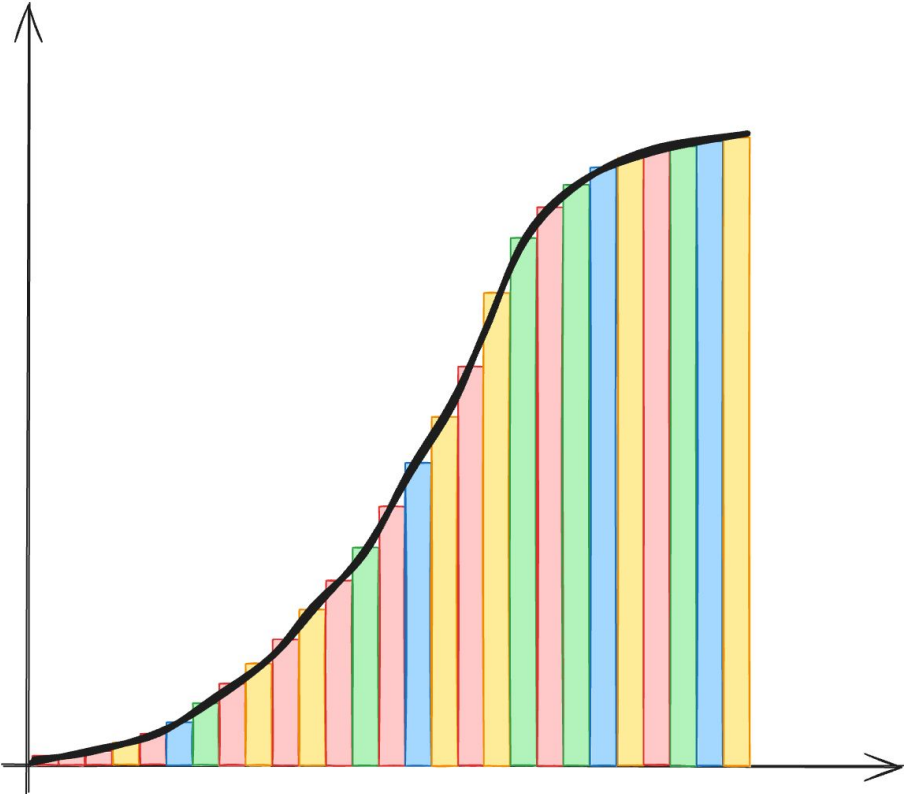
Tasks to do



Teams work on the tasks



Assembled product



Business Value

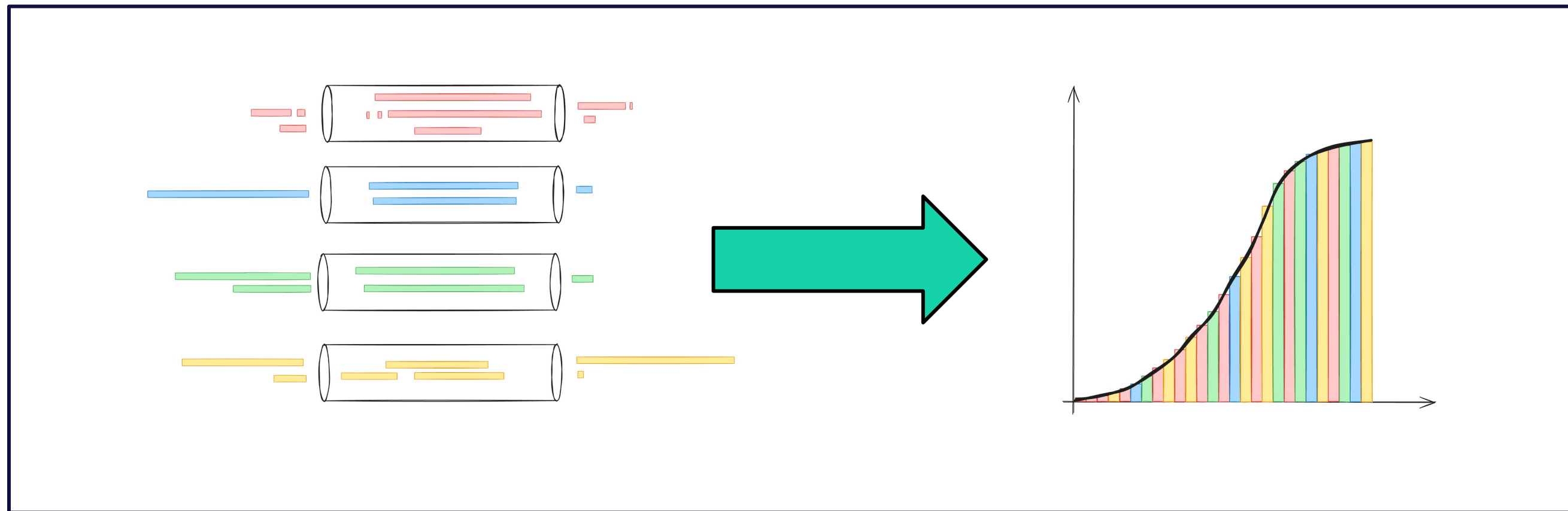
What is Flow Metrics?

Business Value Does Not Magically Appear

- The customer wants the final result

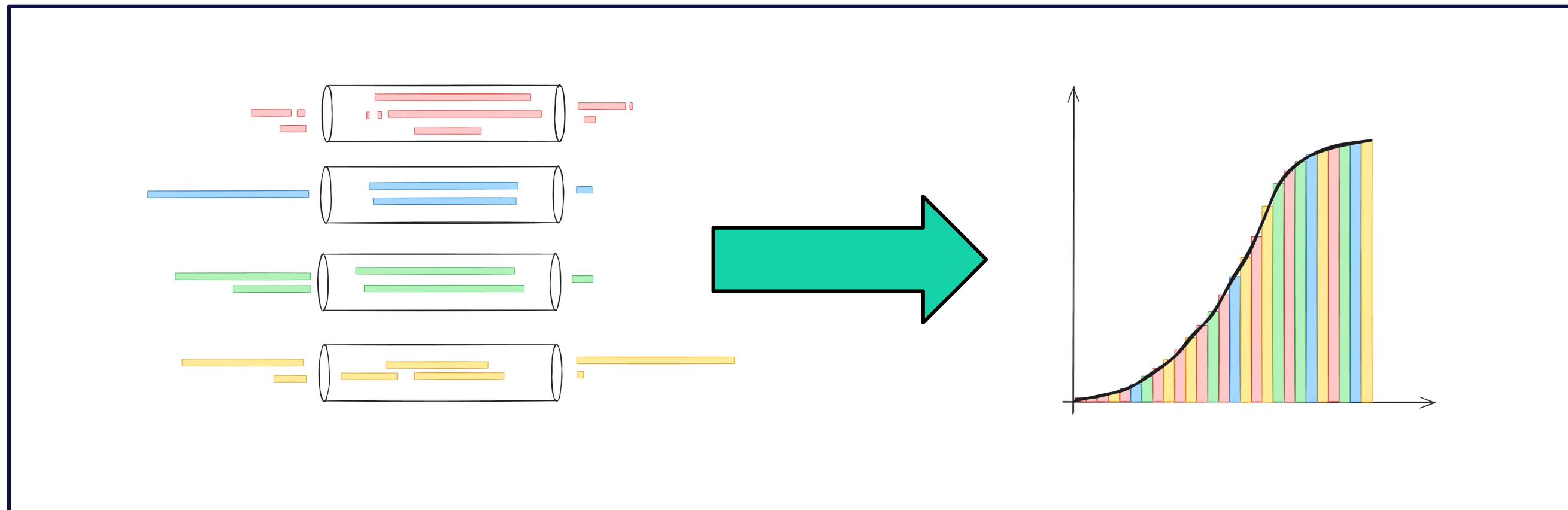
Business Value Does Not Magically Appear

- The customer wants the final result, not individual work items



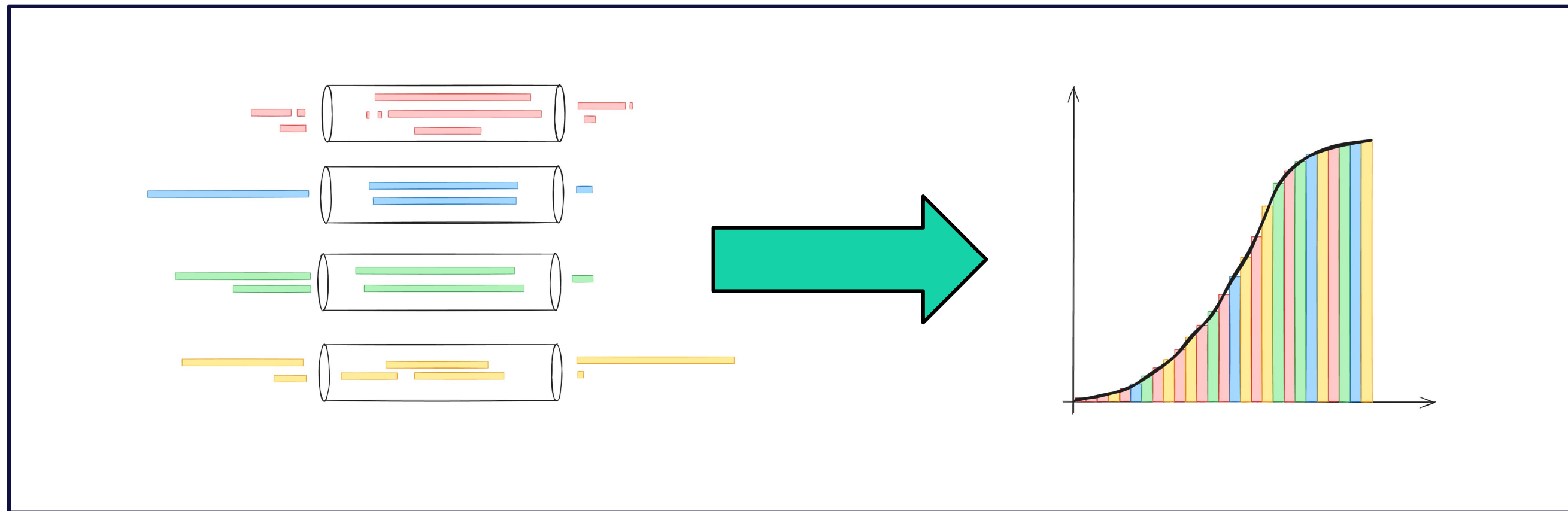
Business Value Does Not Magically Appear

- The customer wants the final result, not individual work items
- Flow is the movement of work items from **potential** to **concrete** value

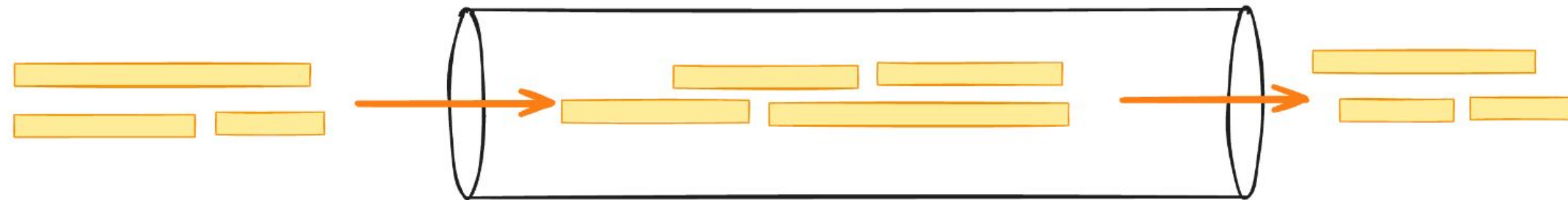


Business Value Does Not Magically Appear

- The customer wants the final result, not individual work items
- Flow is the movement of work items from **potential to concrete** value
- Flow metrics measure flow

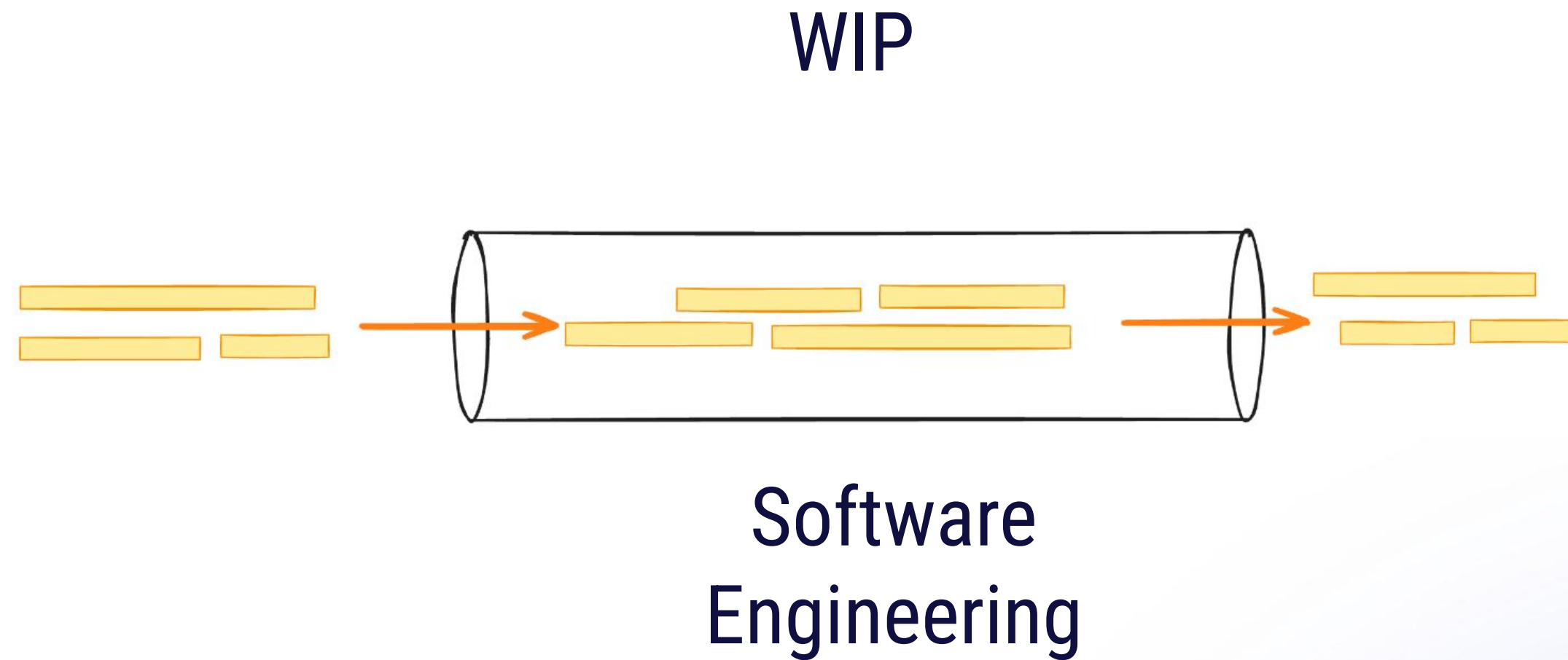


Flow Metrics in Software Engineering

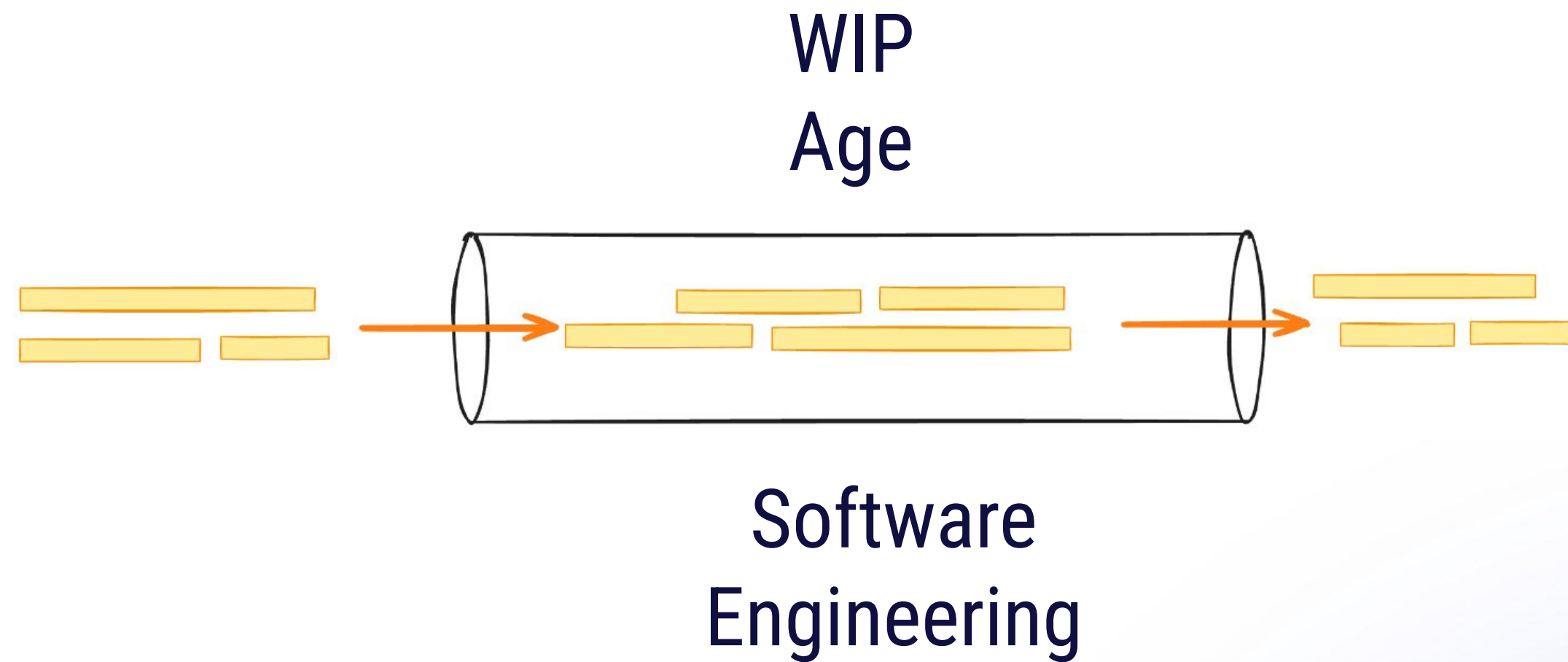


Software
Engineering

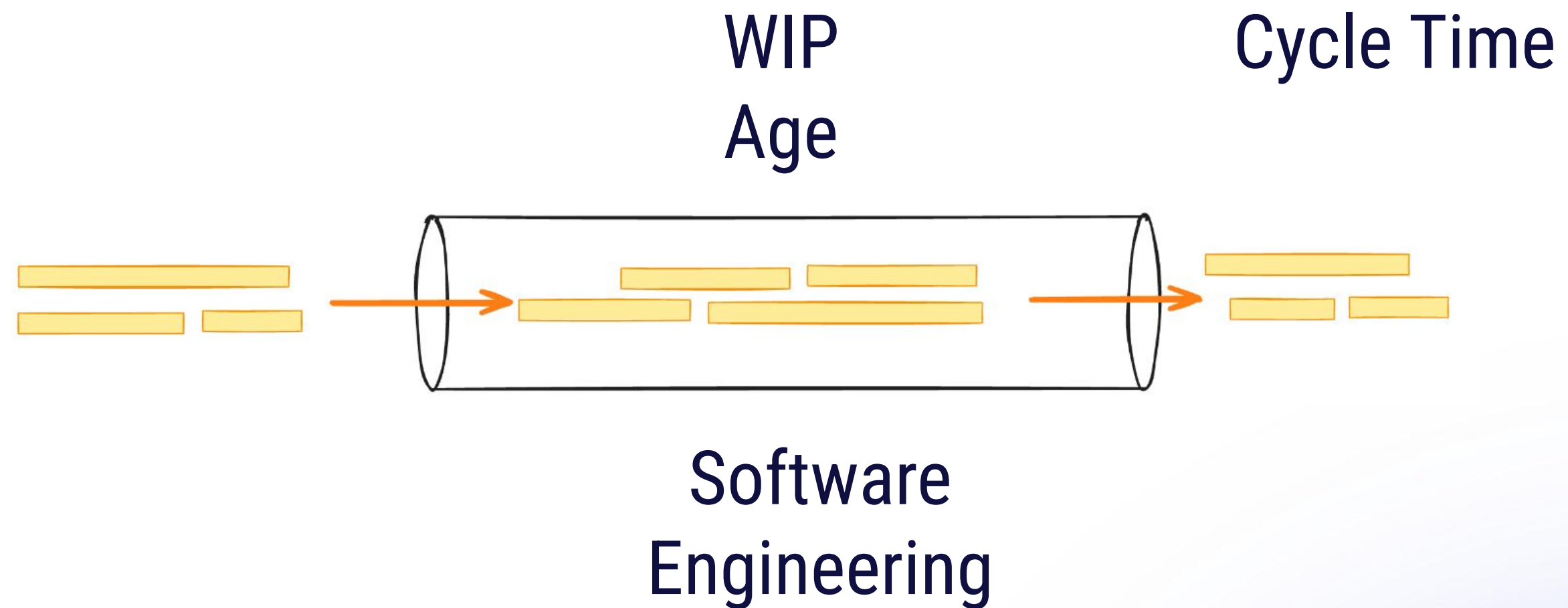
Flow Metrics in Software Engineering



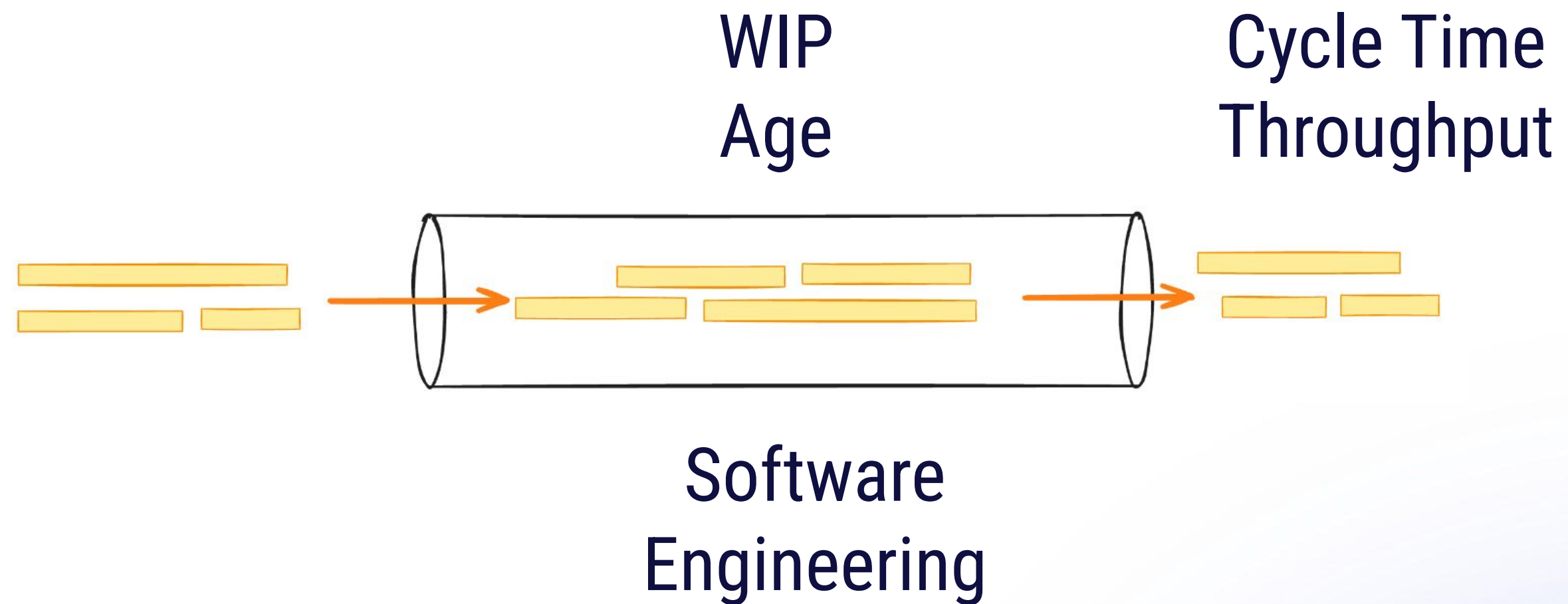
Flow Metrics in Software Engineering



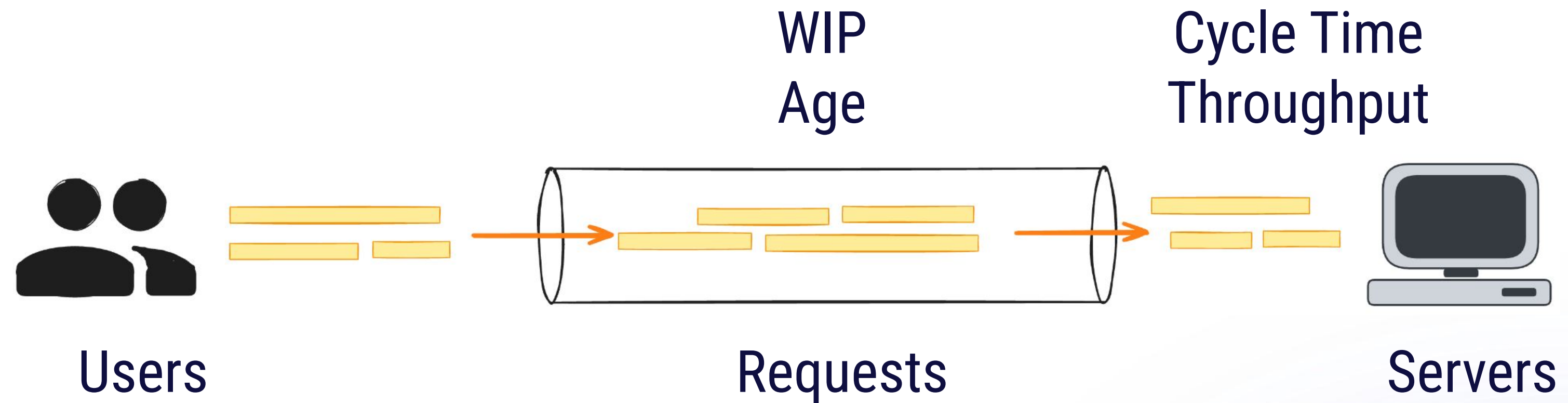
Flow Metrics in Software Engineering



Flow Metrics in Software Engineering

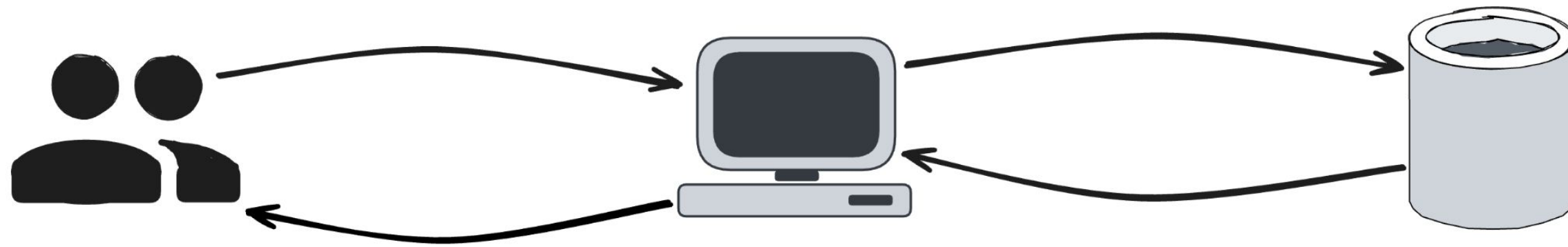


Flow Metrics in Systems

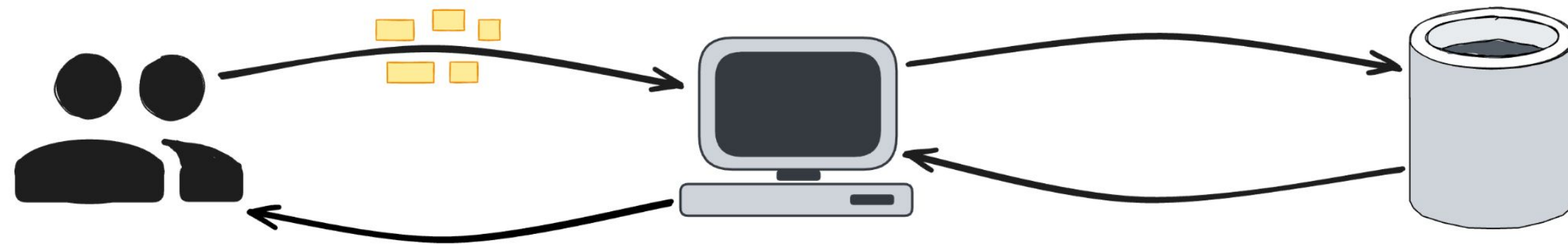


Applying Flow Metrics to Software Systems

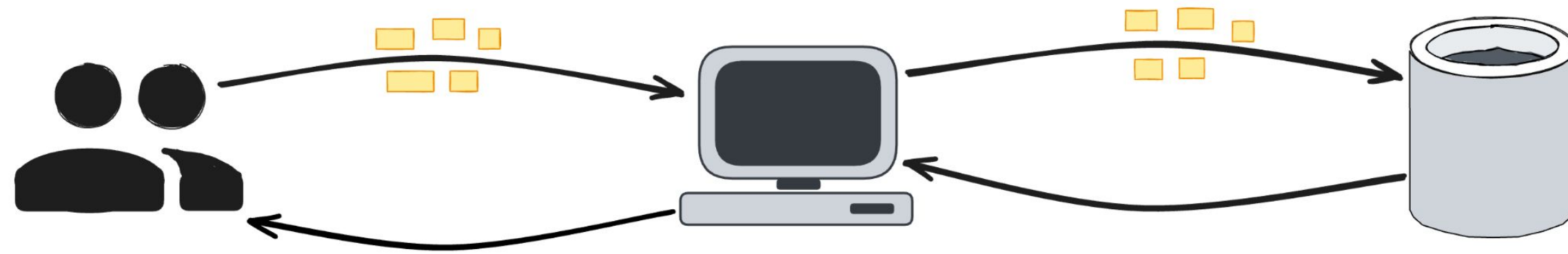
Flow Metrics Applied to Systems



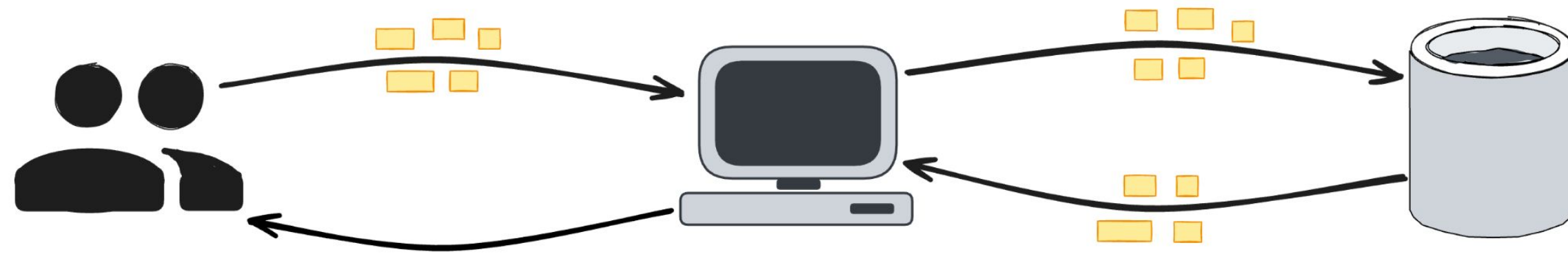
Flow Metrics Applied to Systems



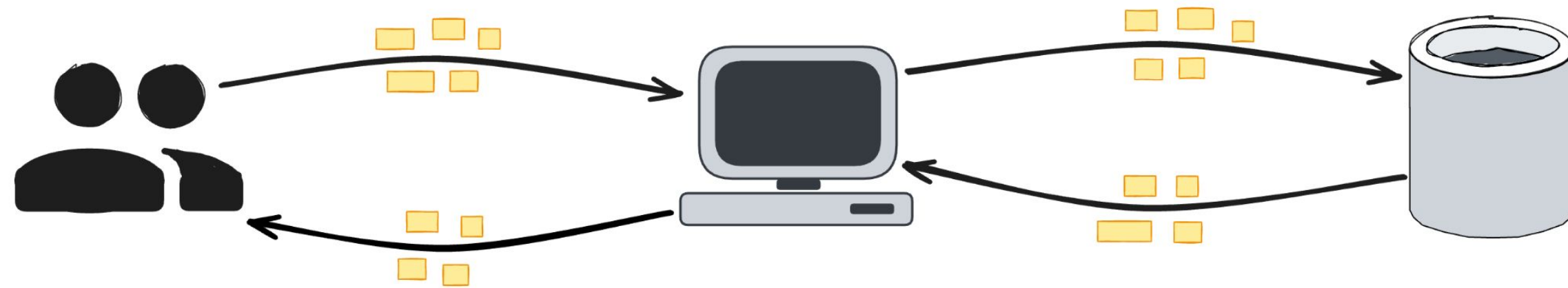
Flow Metrics Applied to Systems



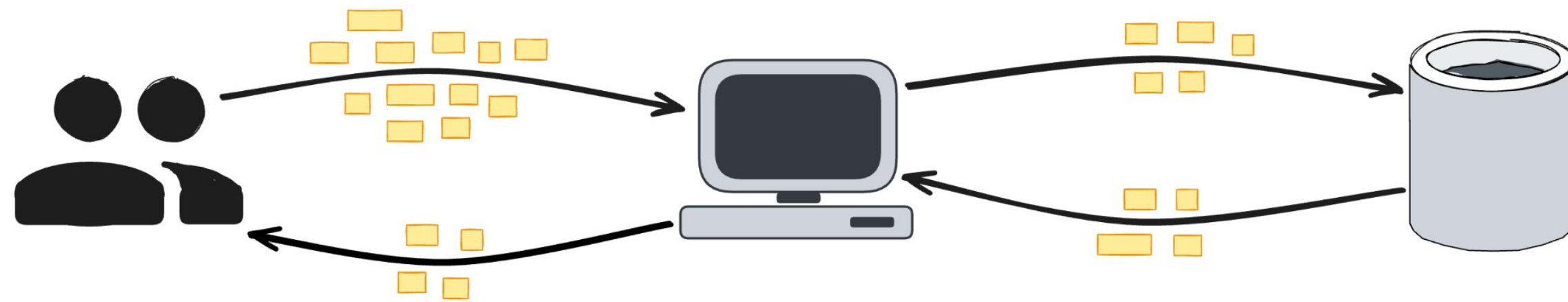
Flow Metrics Applied to Systems



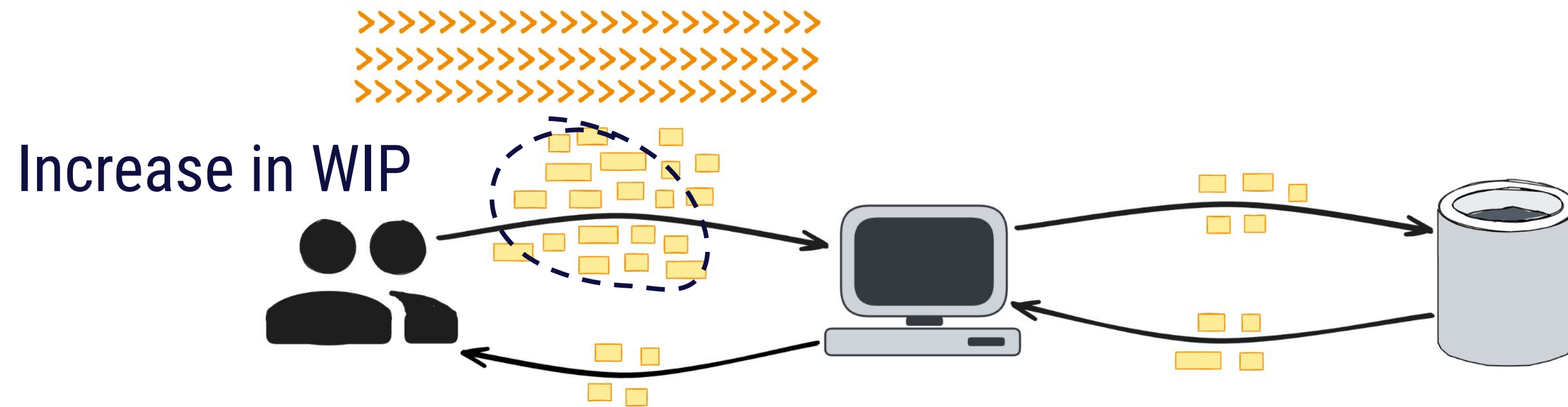
Flow Metrics Applied to Systems



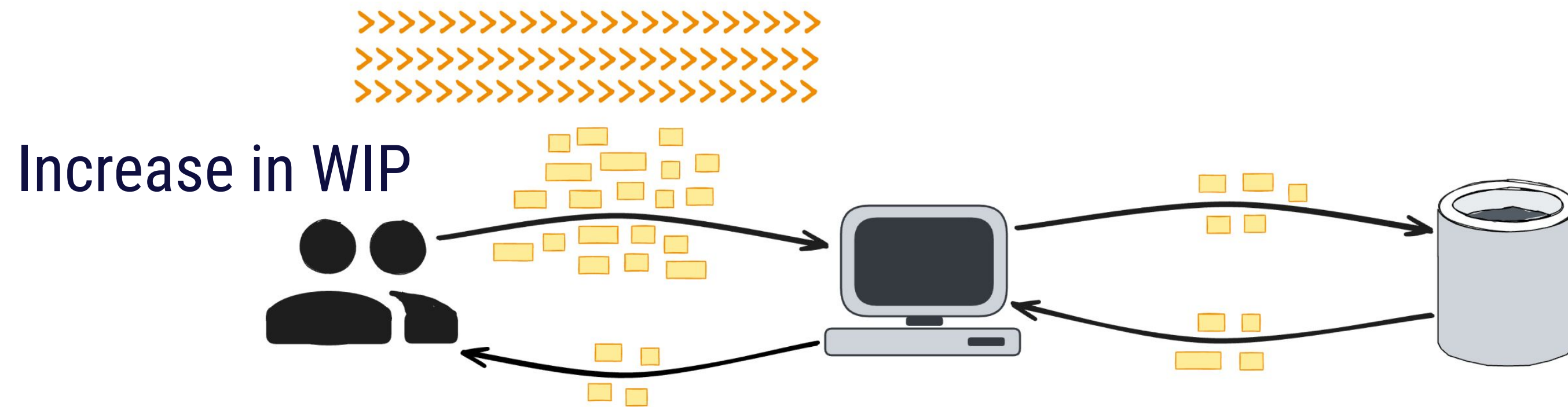
Scenario 1: Spike in Incoming Requests



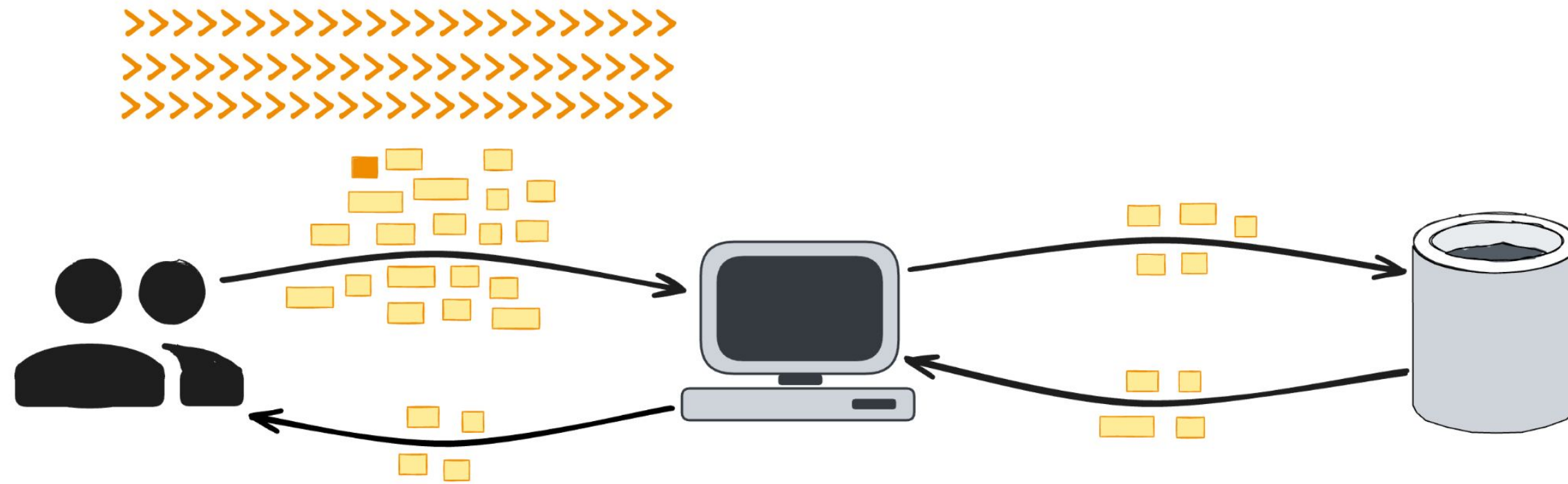
Scenario 1: Spike in Incoming Requests



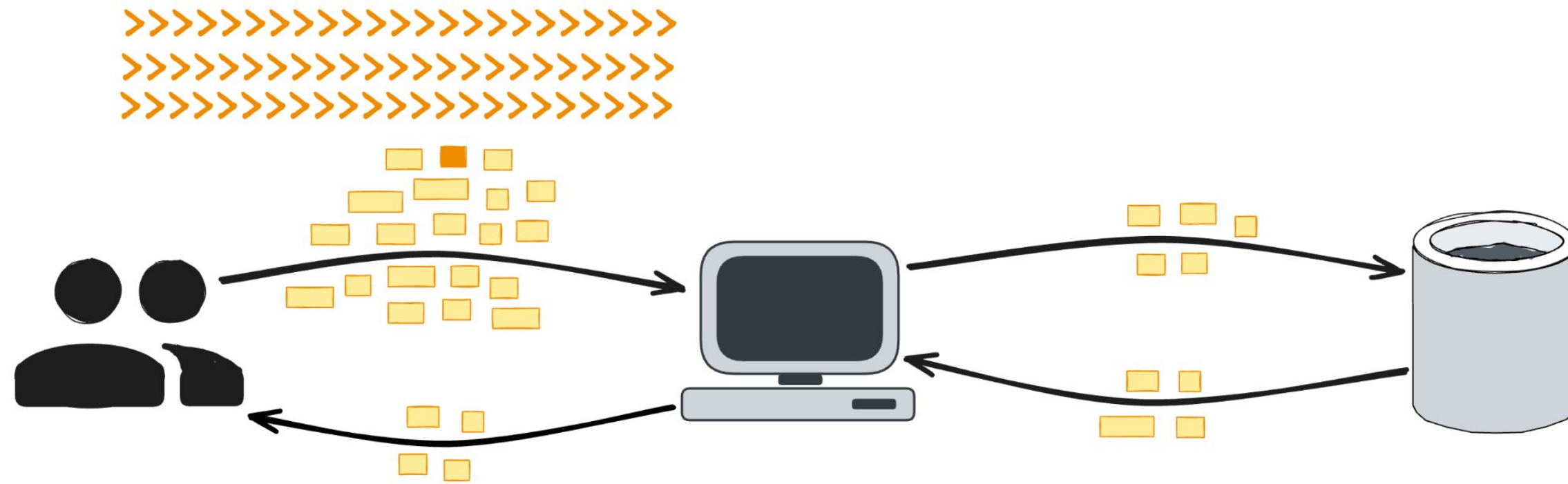
Scenario 1: Spike in Incoming Requests



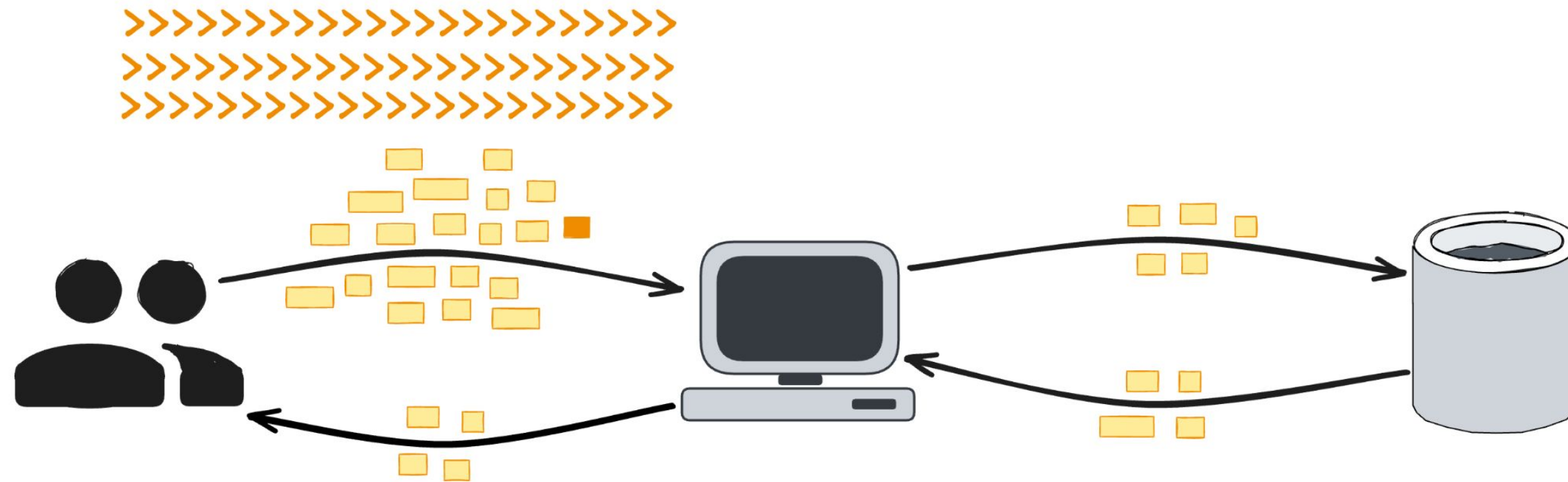
Scenario 1: Spike in Incoming Requests



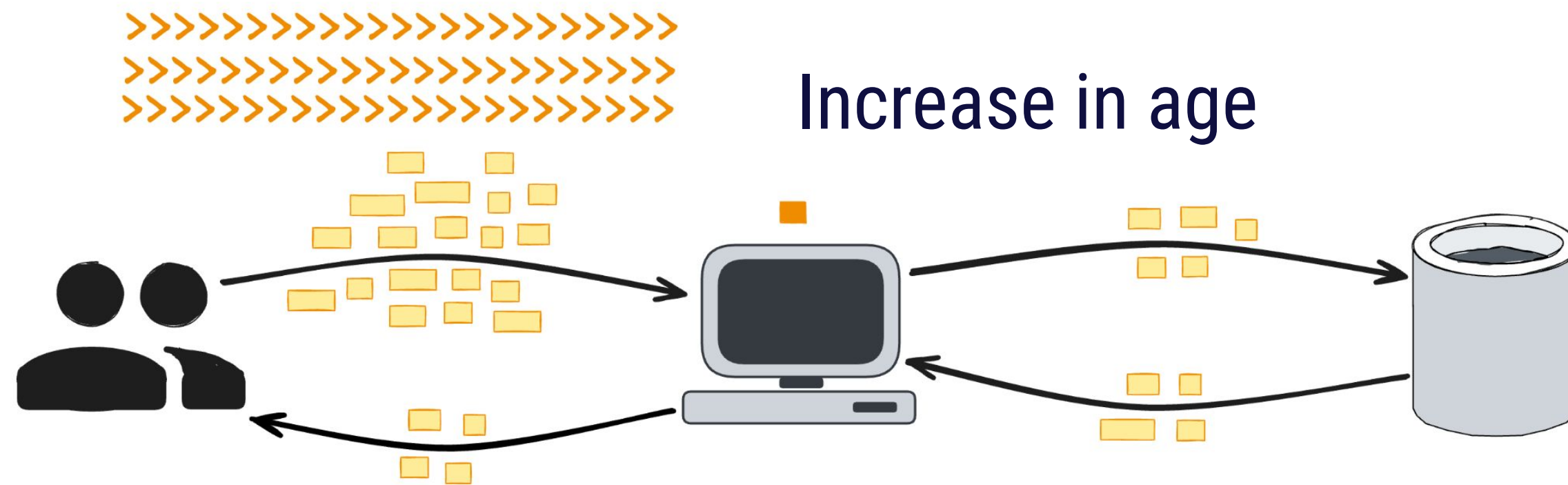
Scenario 1: Spike in Incoming Requests



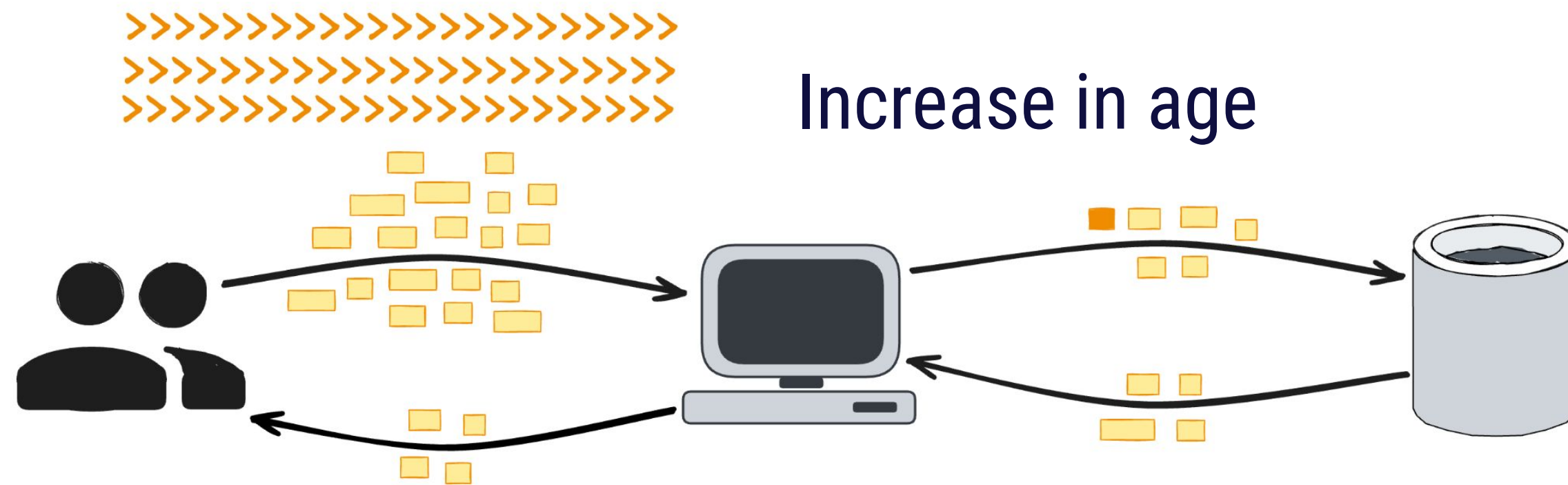
Scenario 1: Spike in Incoming Requests



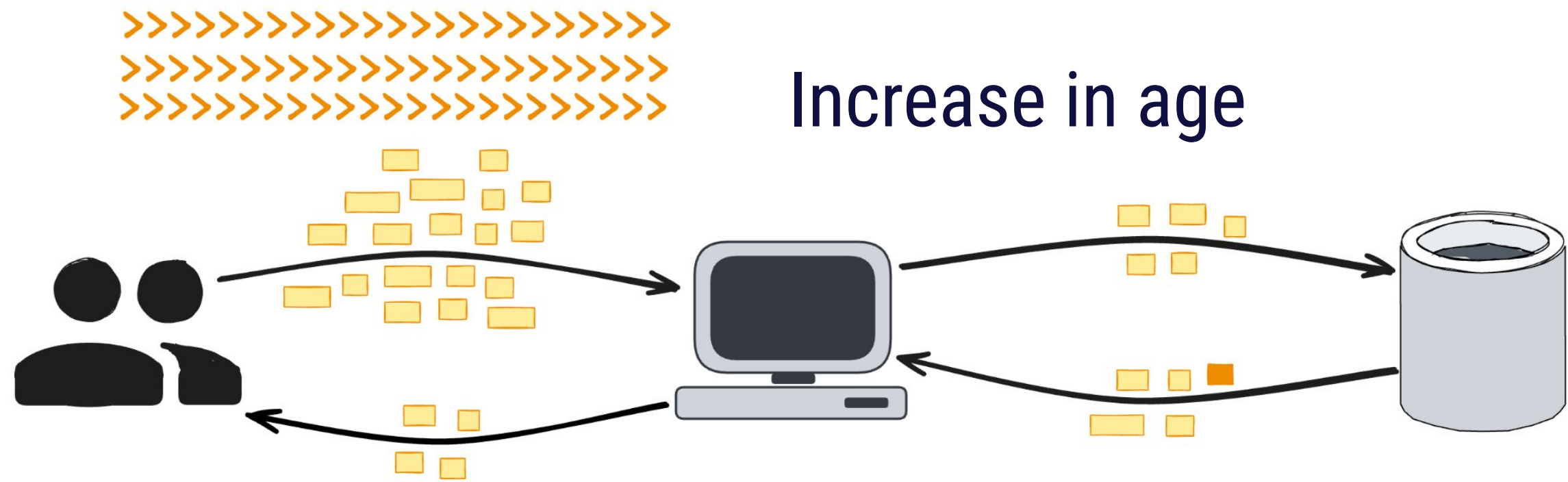
Scenario 1: Spike in Incoming Requests



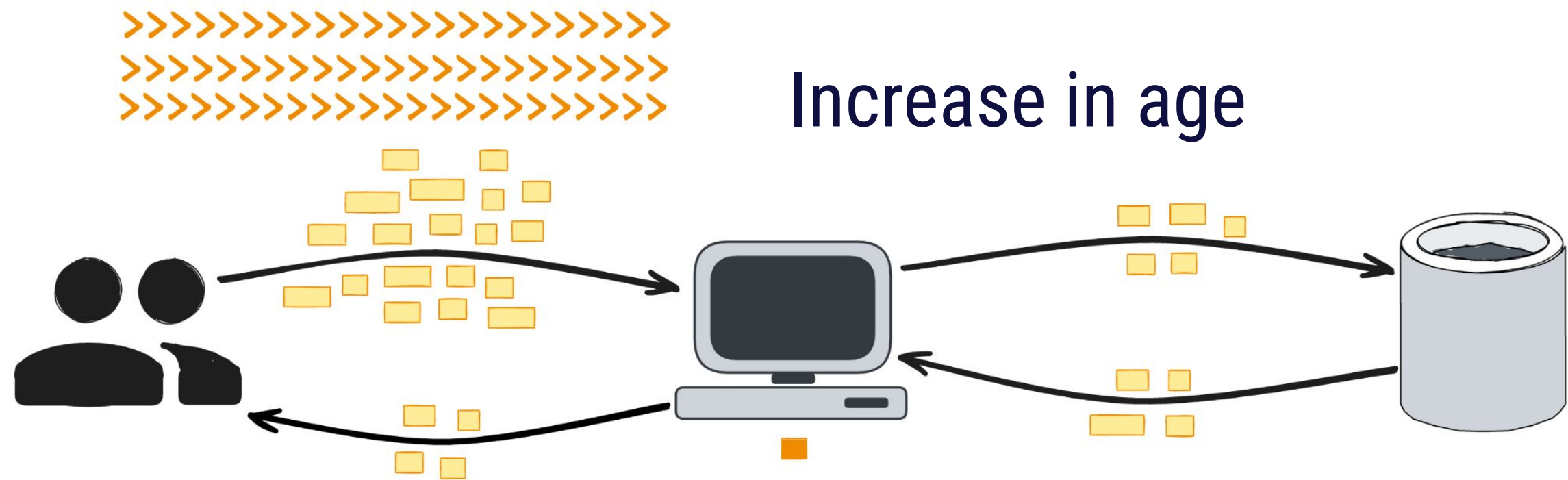
Scenario 1: Spike in Incoming Requests



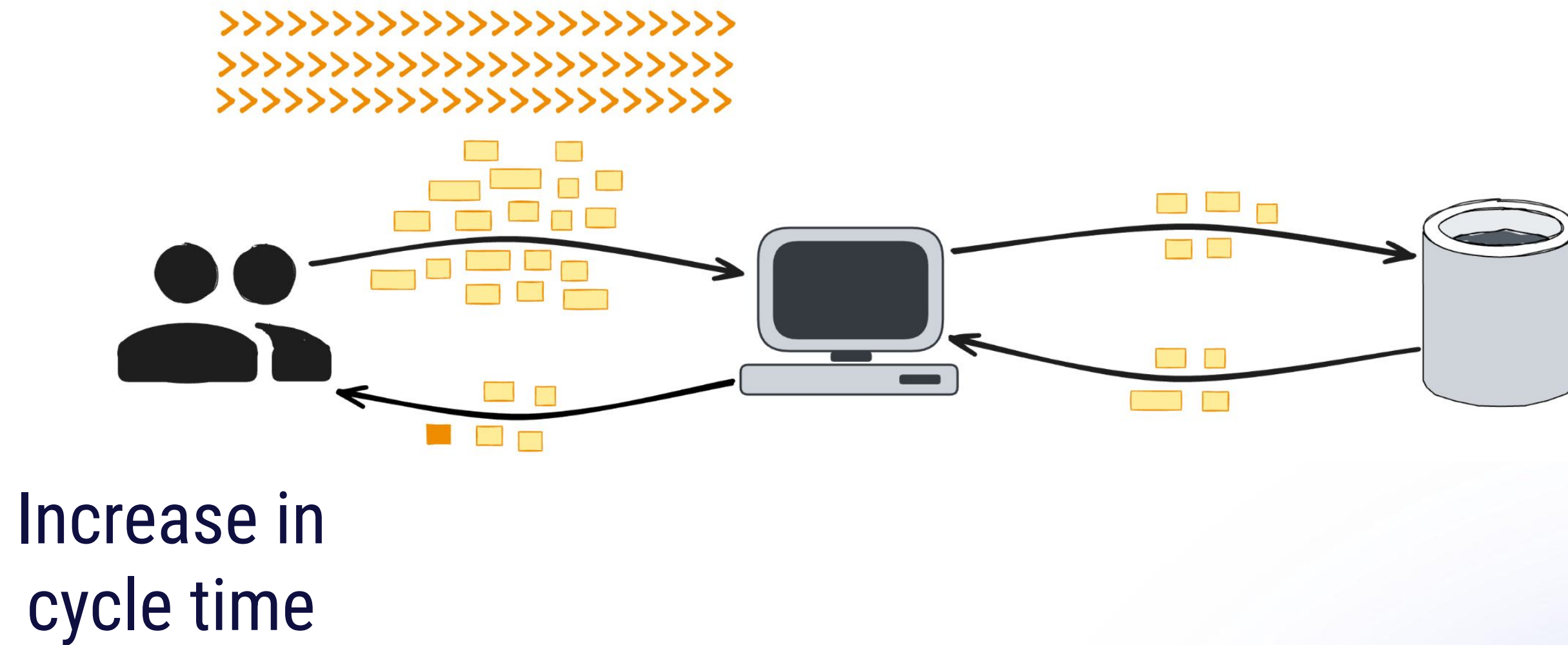
Scenario 1: Spike in Incoming Requests



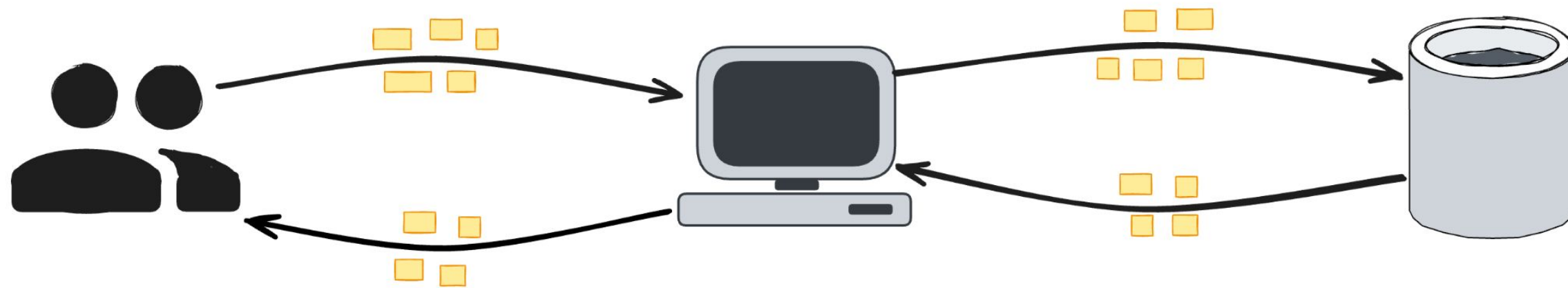
Scenario 1: Spike in Incoming Requests



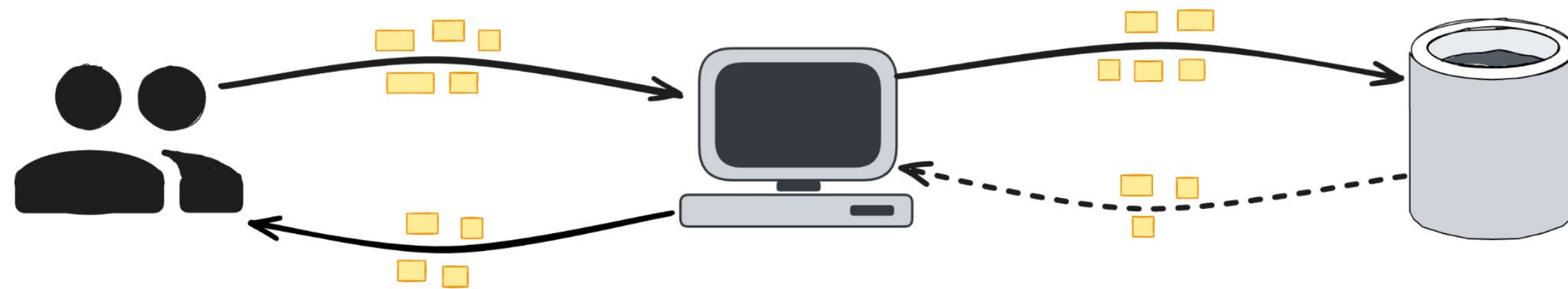
Scenario 1: Spike in Incoming Requests



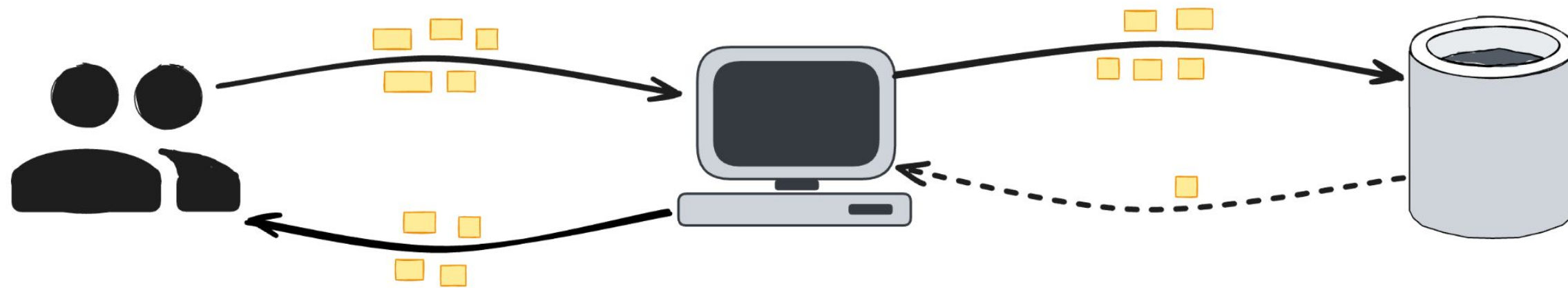
Scenario 2: Degraded Dependency



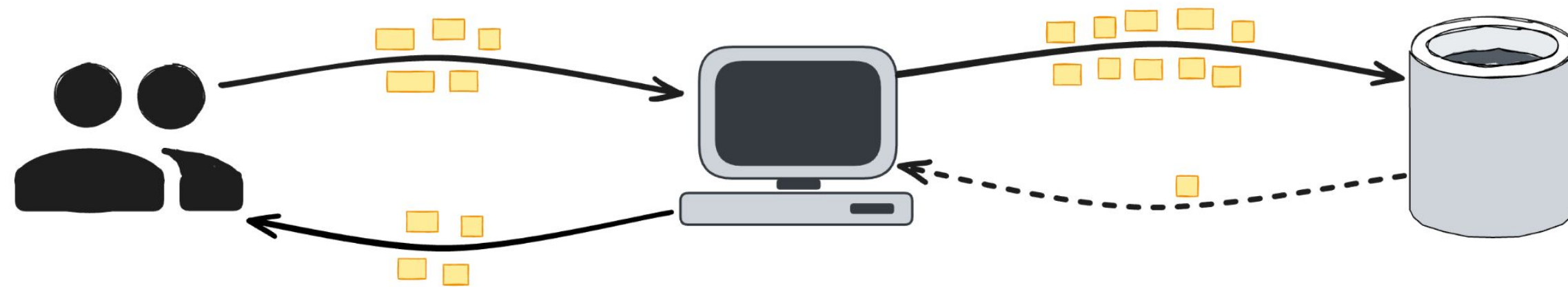
Scenario 2: Degraded Dependency



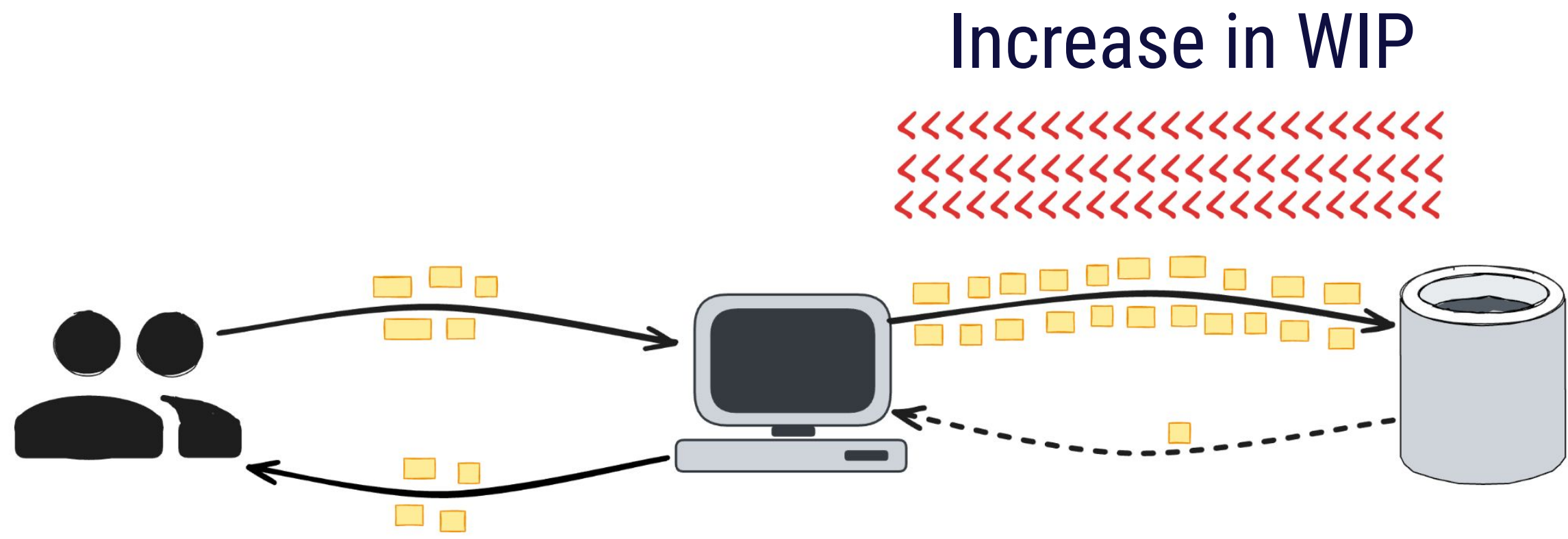
Scenario 2: Degraded Dependency



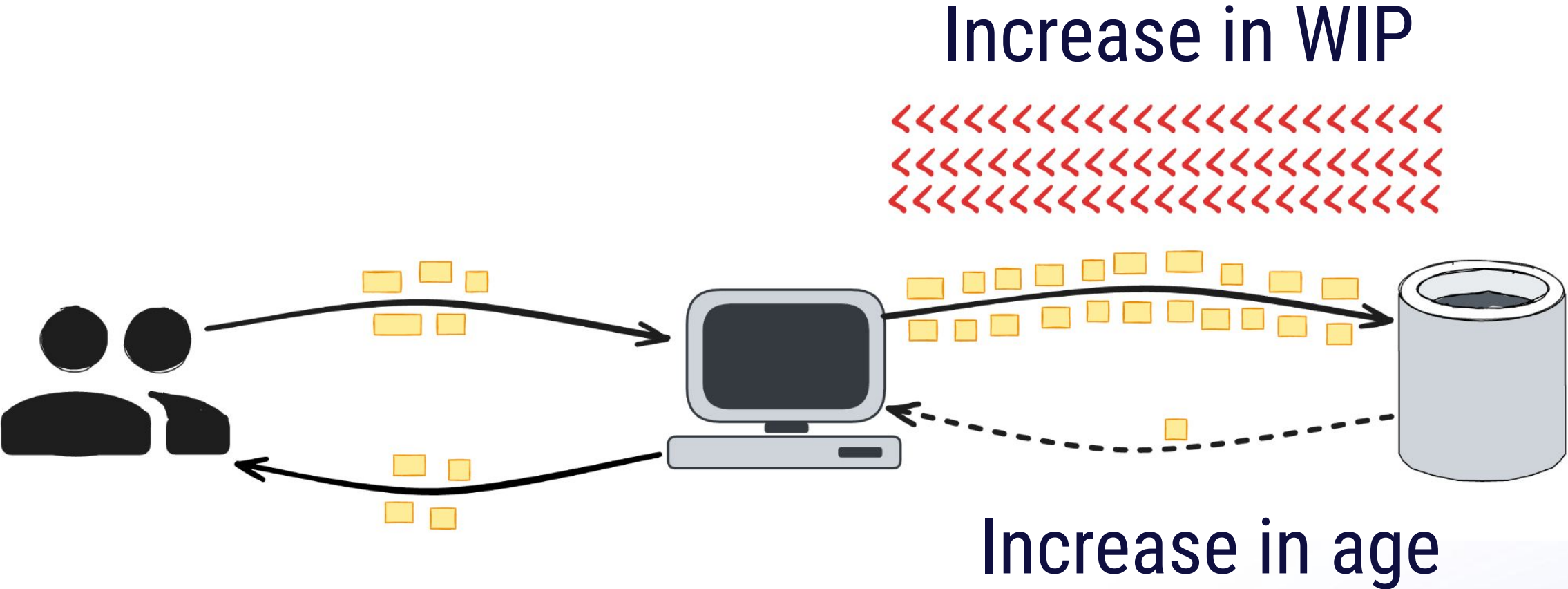
Scenario 2: Degraded Dependency



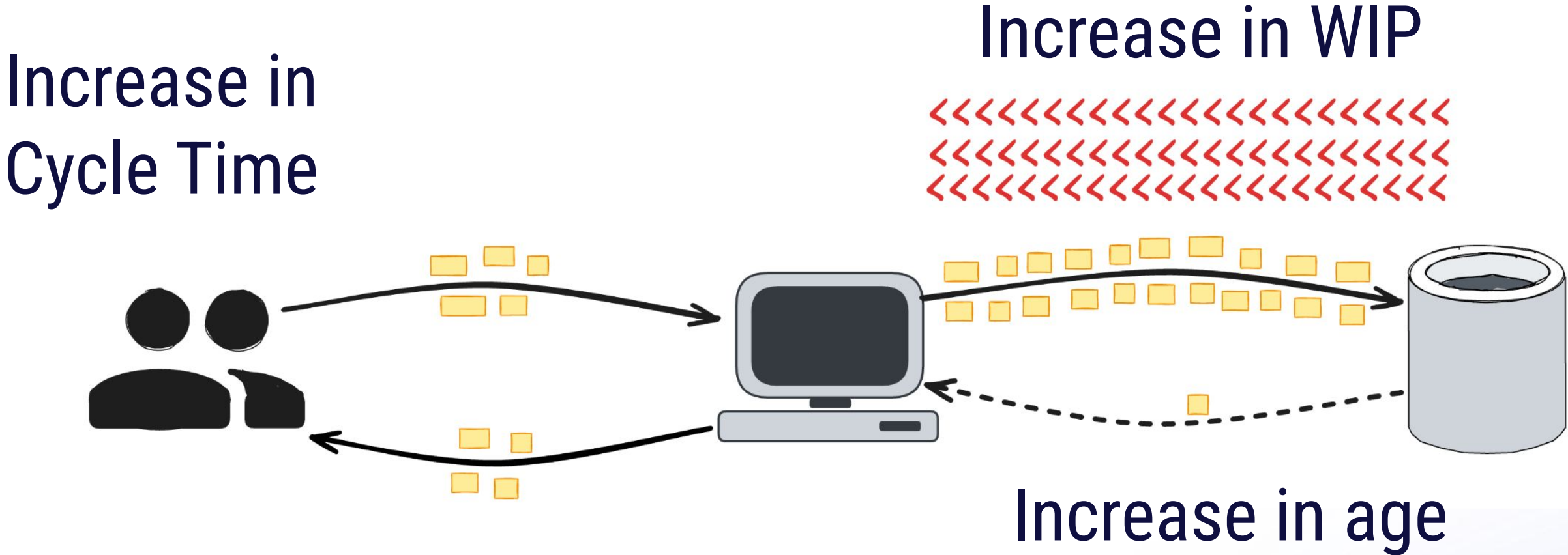
Scenario 2: Degraded Dependency



Scenario 2: Degraded Dependency



Scenario 2: Degraded Dependency



Flow Metrics Applied to Systems

- If there is **congestion**
 - WIP increases
 - Age increases
 - Cycle time will increase

Flow Metrics Applied to Systems

- If there is **congestion**

- WIP increases
- Age increases

Leading indicators

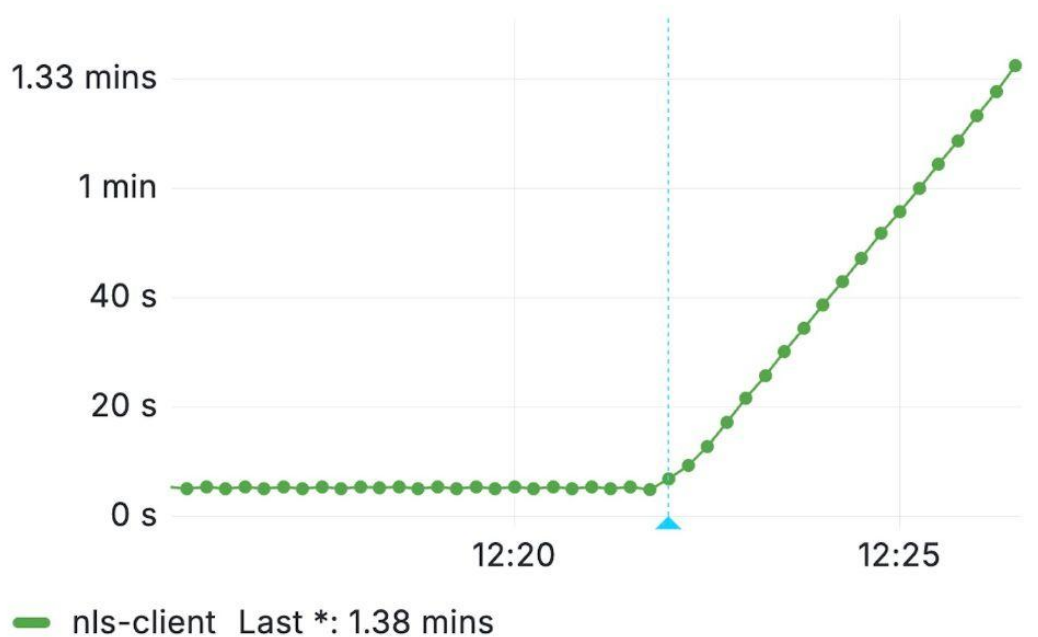
- Cycle time will increase

Detecting Congestion with Flow Metrics

Requests Fired by Client - NLS



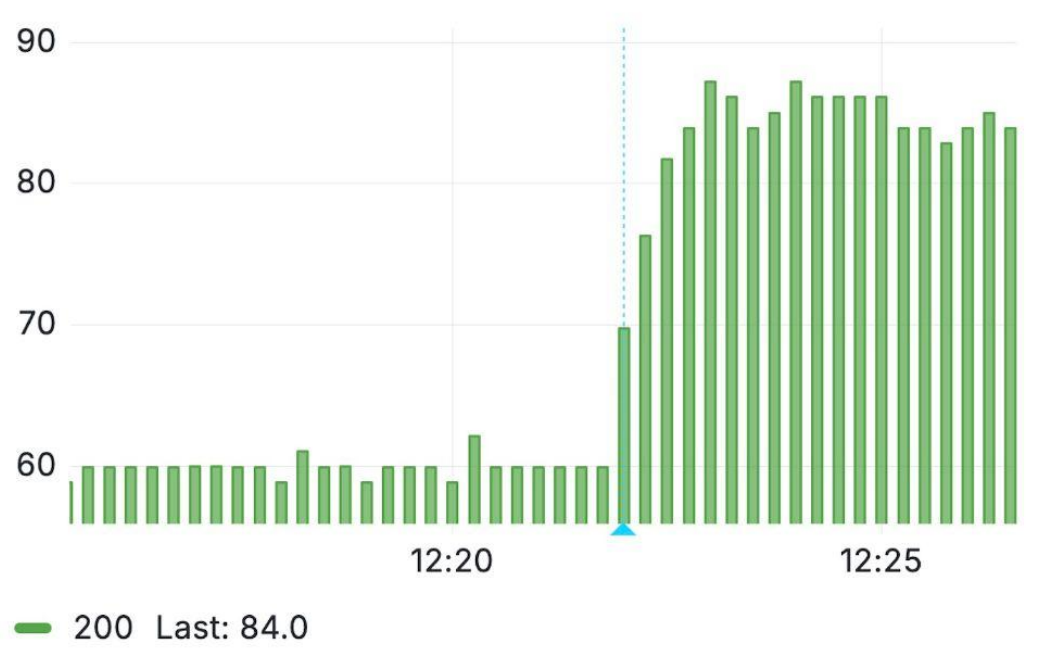
Client Latency - NLS



Processing Time - NLS



Response Statuses - NLS

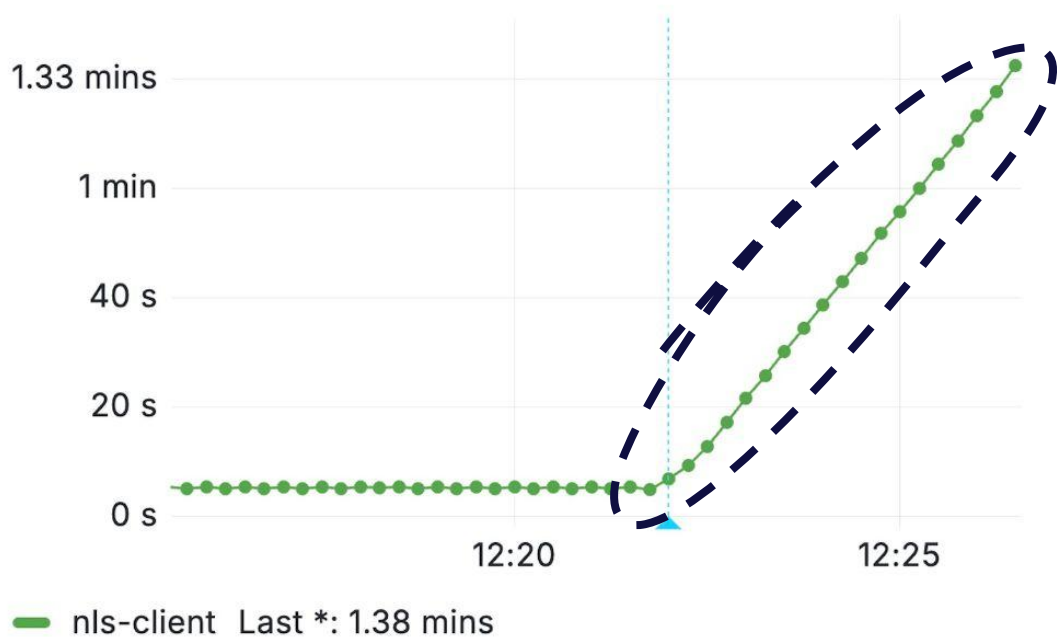


Detecting Congestion with Flow Metrics

Requests Fired by Client - NLS



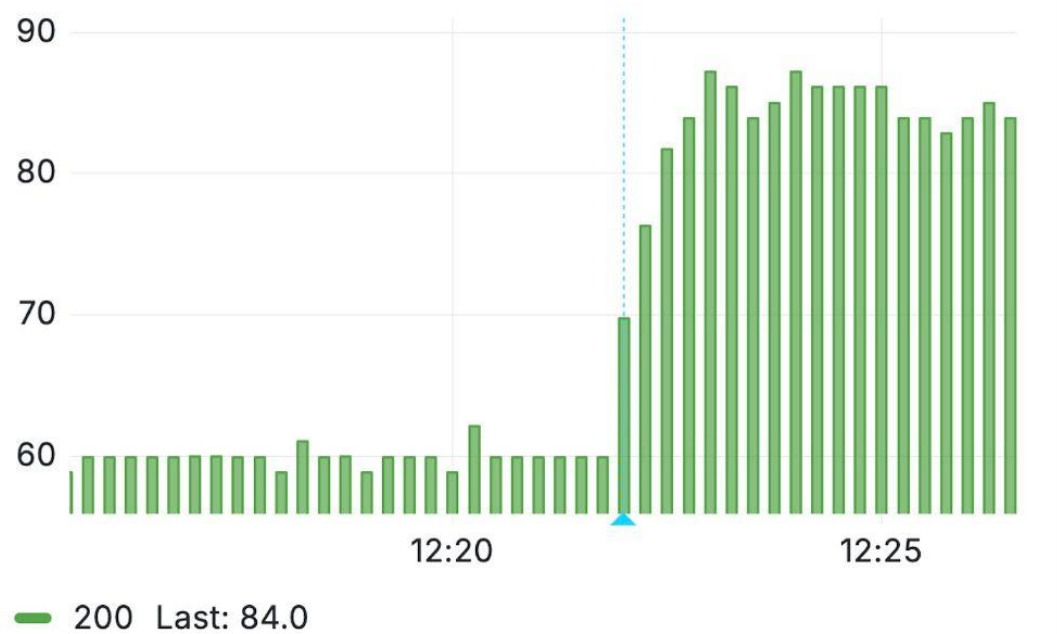
Client Latency - NLS



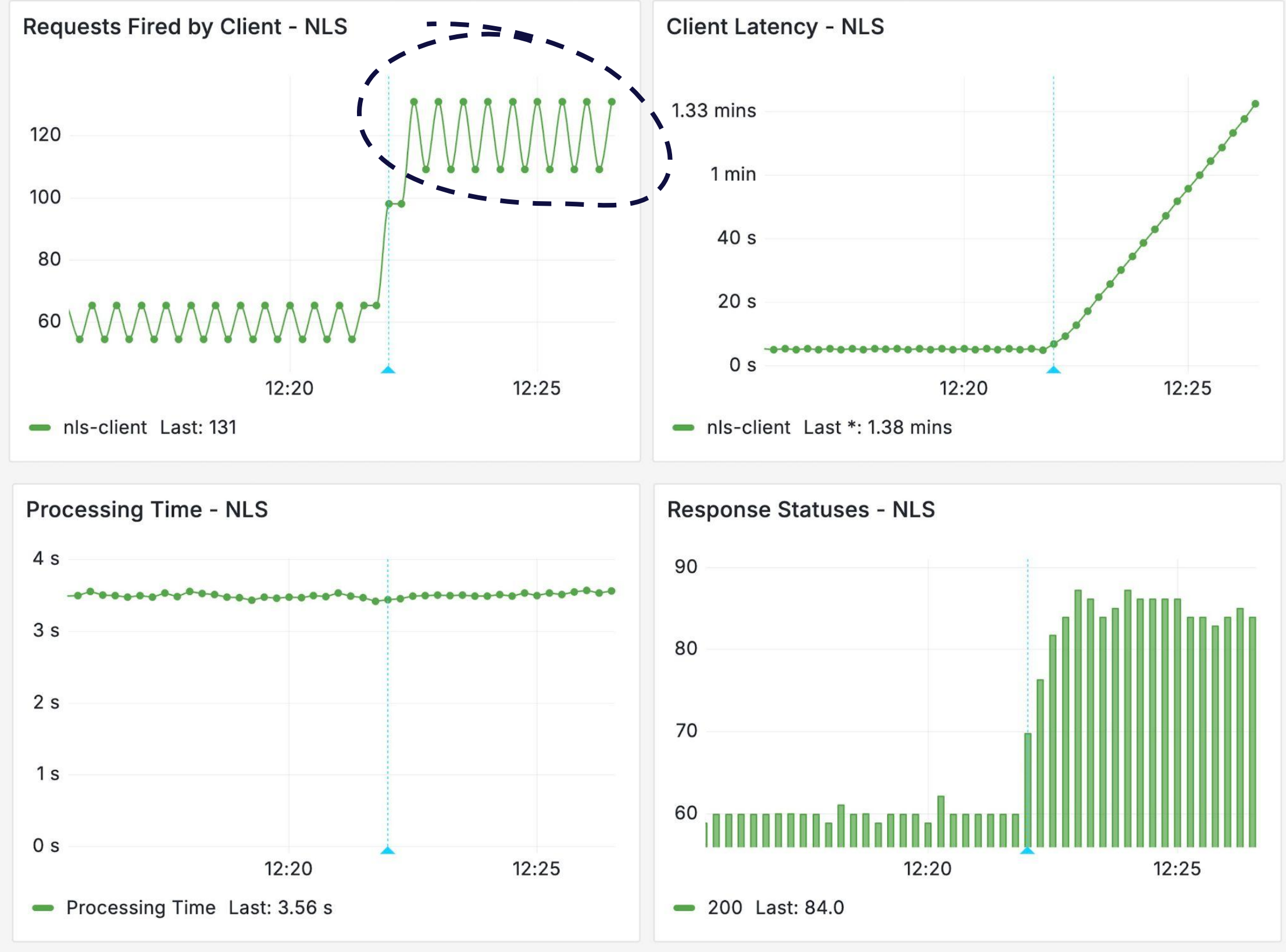
Processing Time - NLS



Response Statuses - NLS



Detecting Congestion with Flow Metrics



Detecting Congestion with Flow Metrics

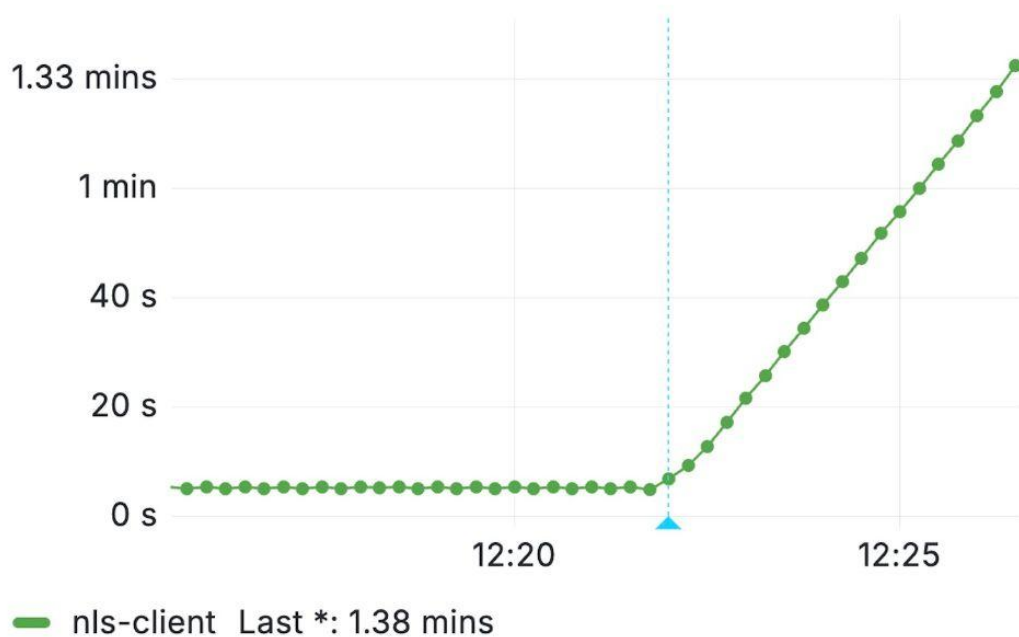


Detecting Congestion with Flow Metrics

Requests Fired by Client - NLS



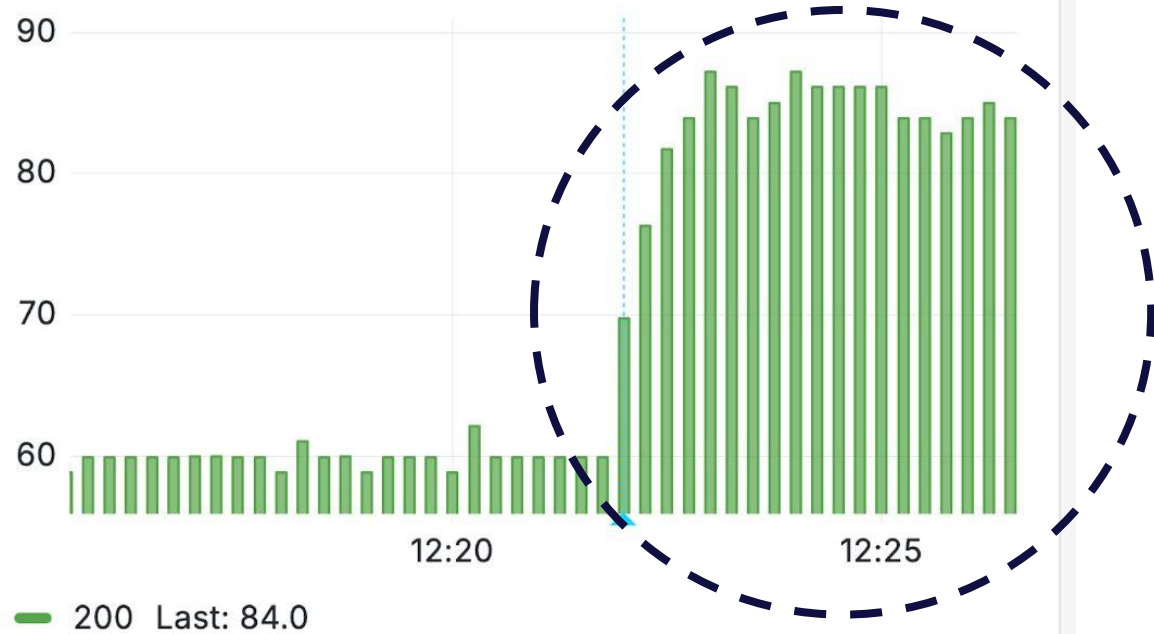
Client Latency - NLS



Processing Time - NLS

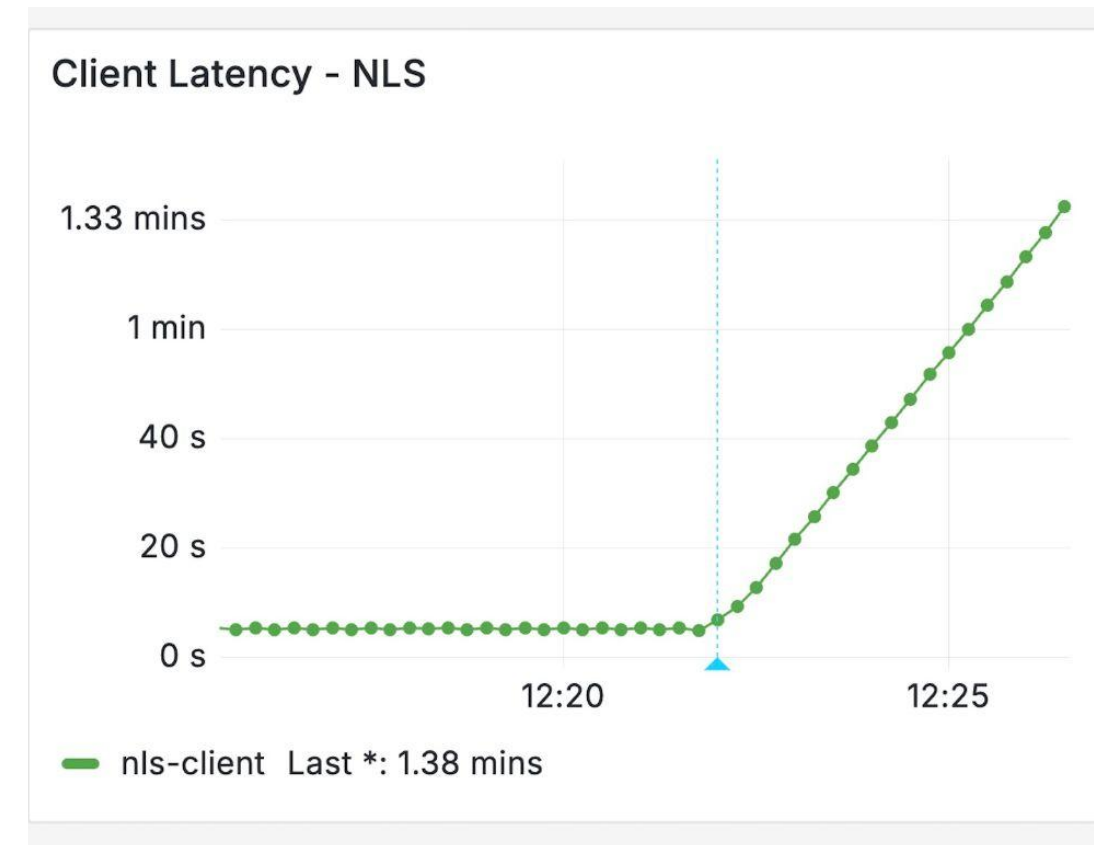


Response Statuses - NLS



Detecting Congestion with Flow Metrics

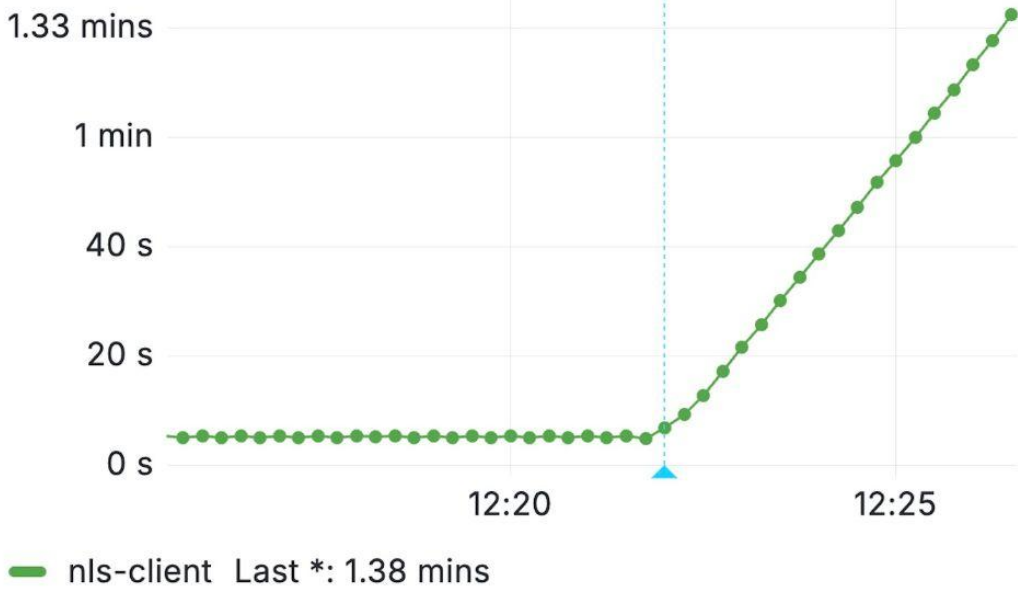
**Impact observed
by the client**



Detecting Congestion with Flow Metrics

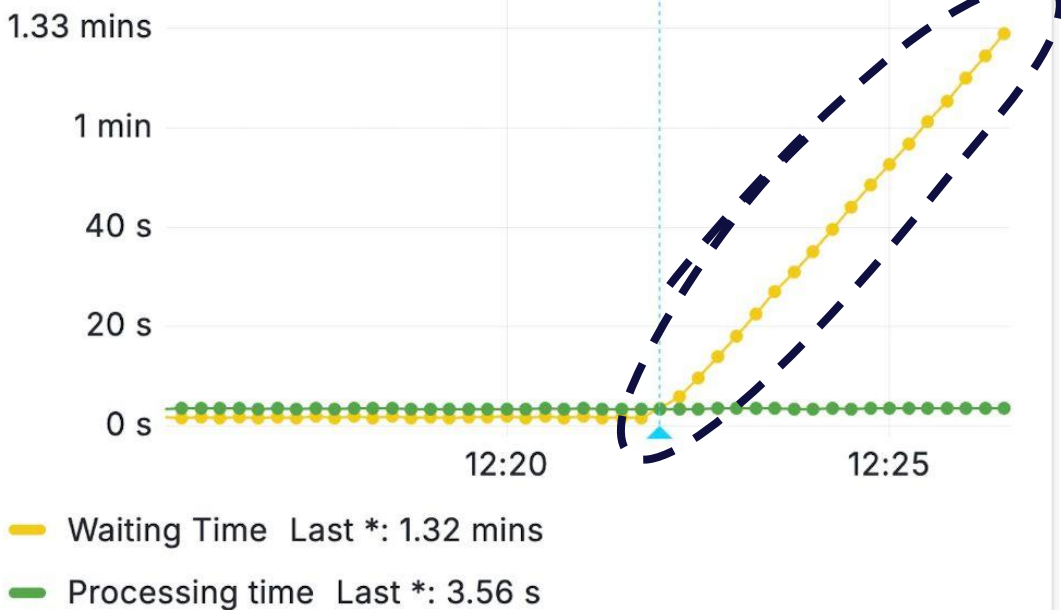
Impact observed
by the client

Client Latency - NLS



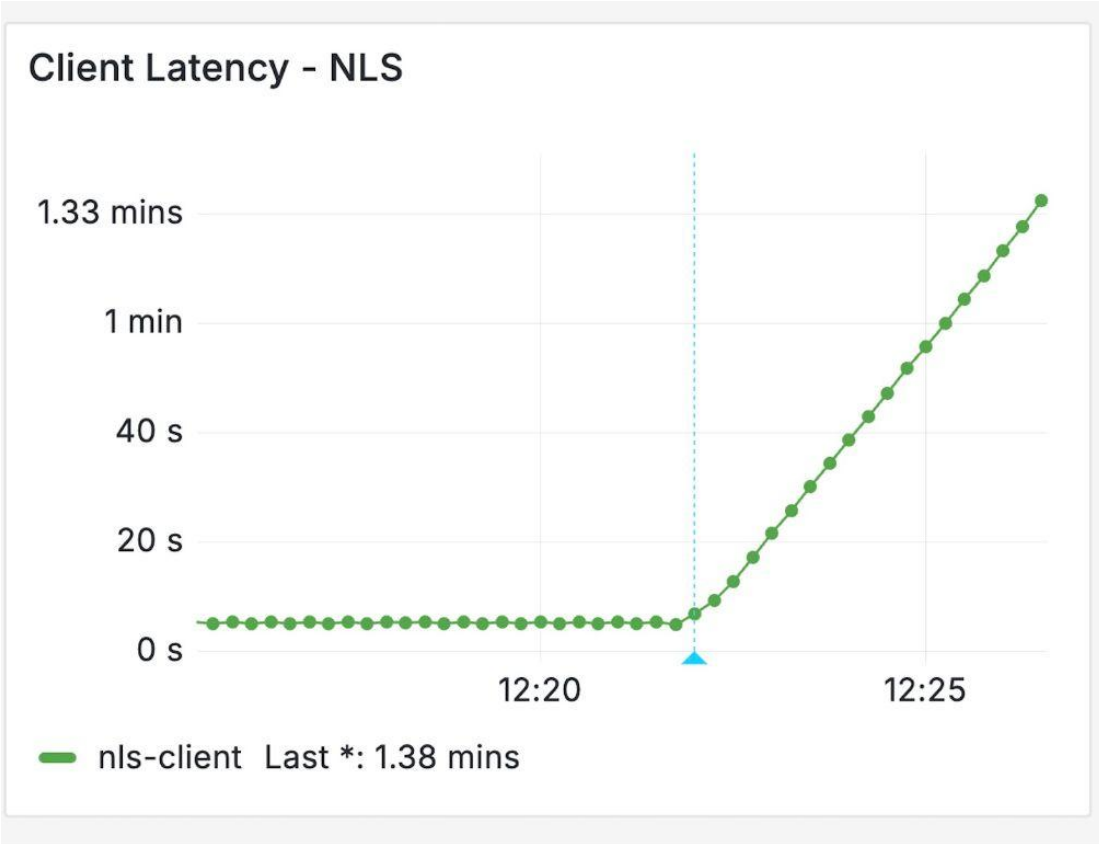
Leading
Indicators

Age - NLS



Detecting Congestion with Flow Metrics

Impact observed
by the client



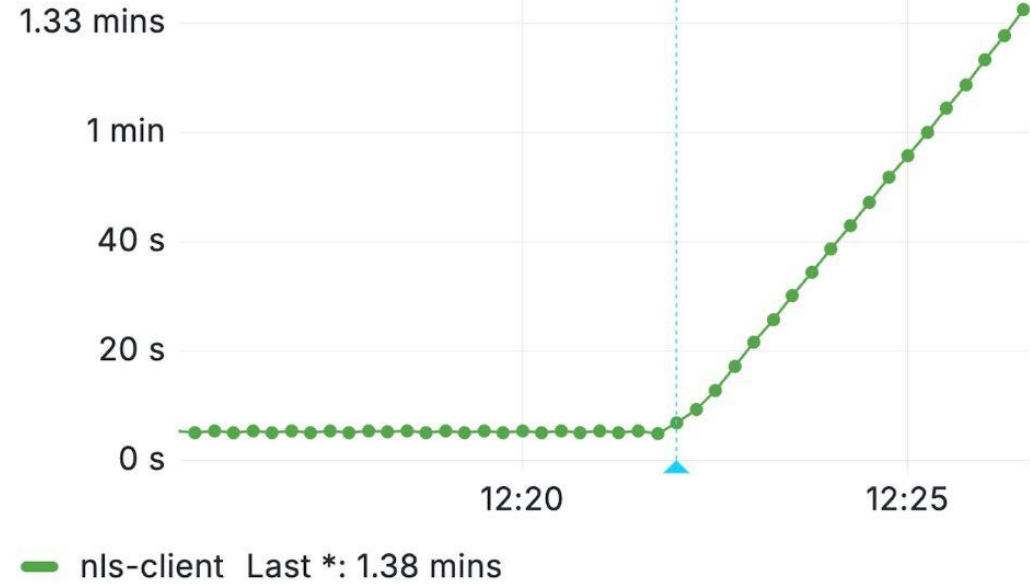
Leading
Indicators



Detecting Congestion with Flow Metrics

Impact observed
by the client

Client Latency - NLS



Requests spend
most of their
age just waiting

Age - NLS



Waiting Time Last *: 1.32 mins

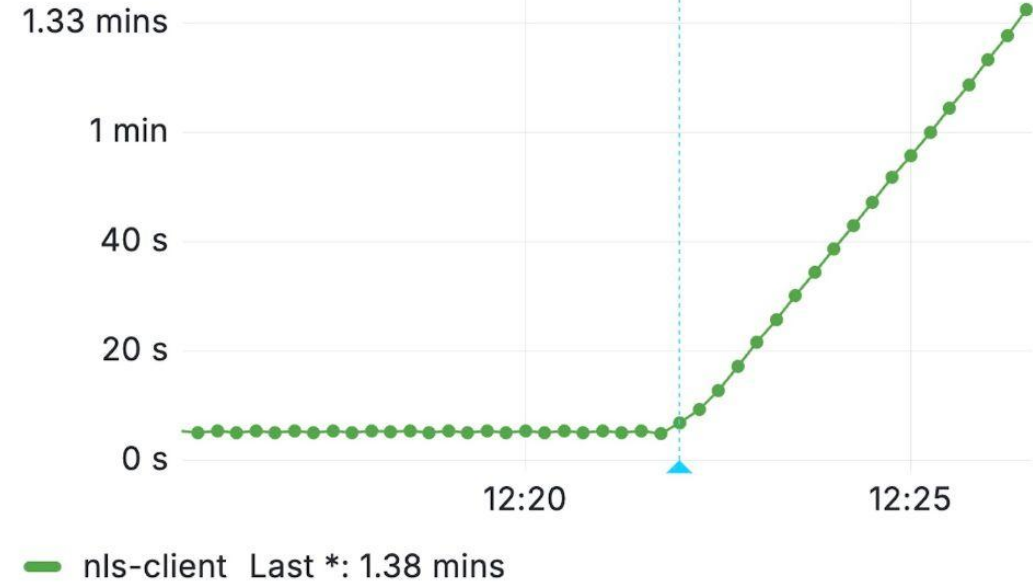
Processing time Last *: 3.56 s

Leading
Indicators

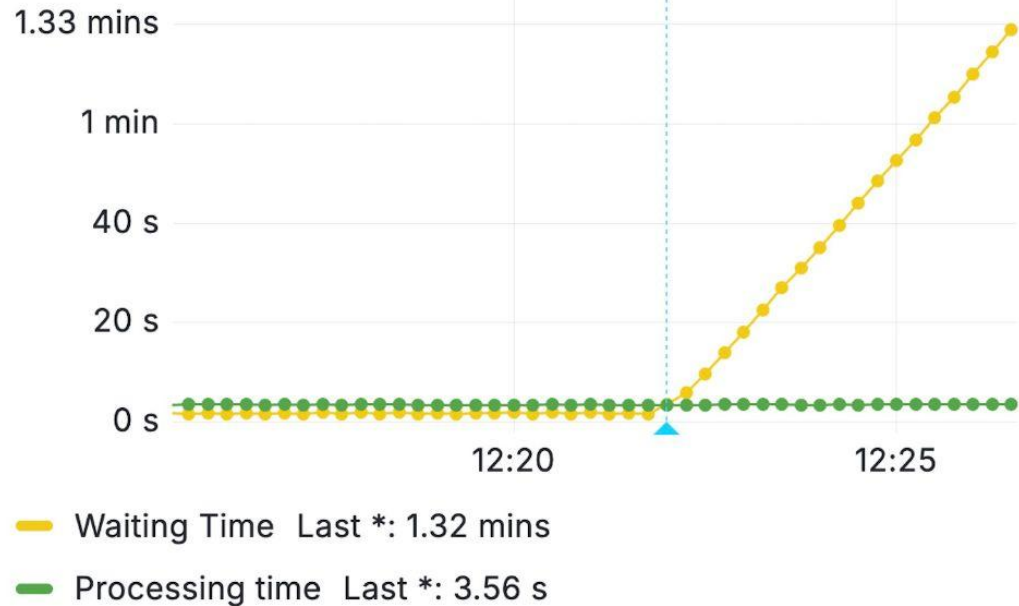
Detecting Congestion with Flow Metrics

Impact observed
by the client

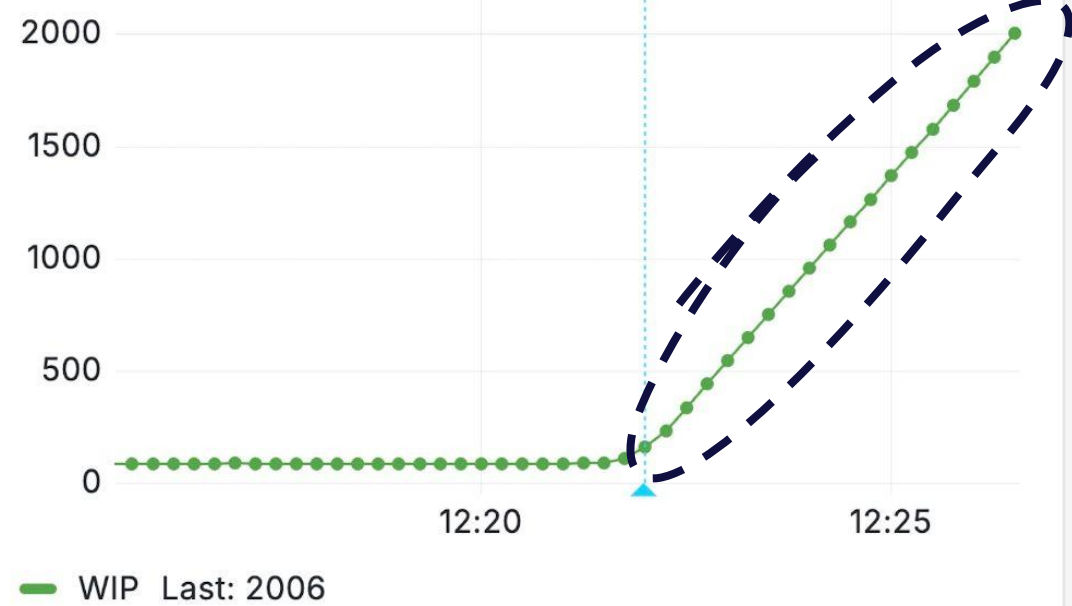
Client Latency - NLS



Age - NLS



In flight Requests (WIP) - NLS



Leading
Indicators

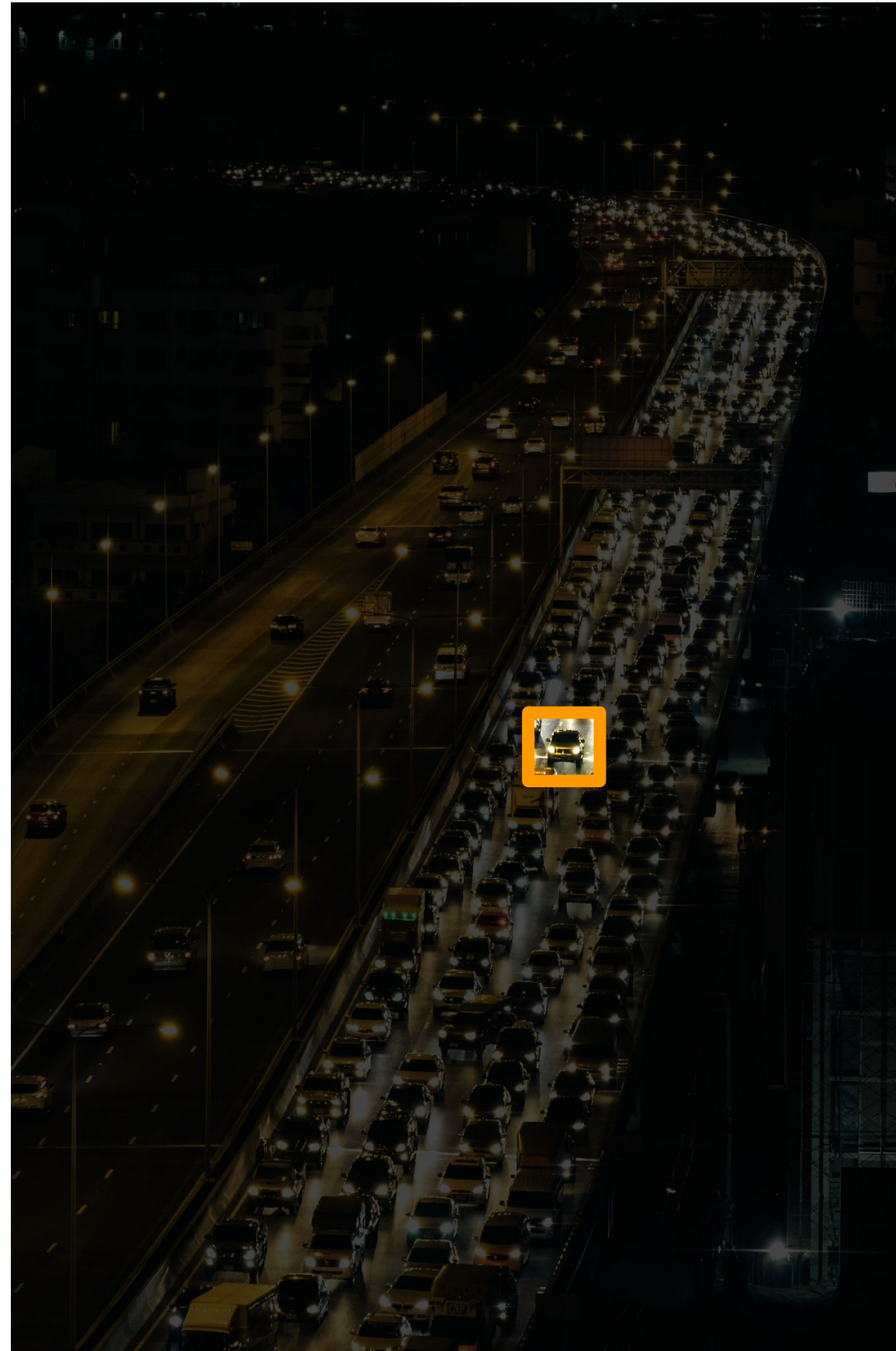
There are
too many
requests in
the system

Detecting Congestion with Flow Metrics

Source



Detecting Congestion with Flow Metrics



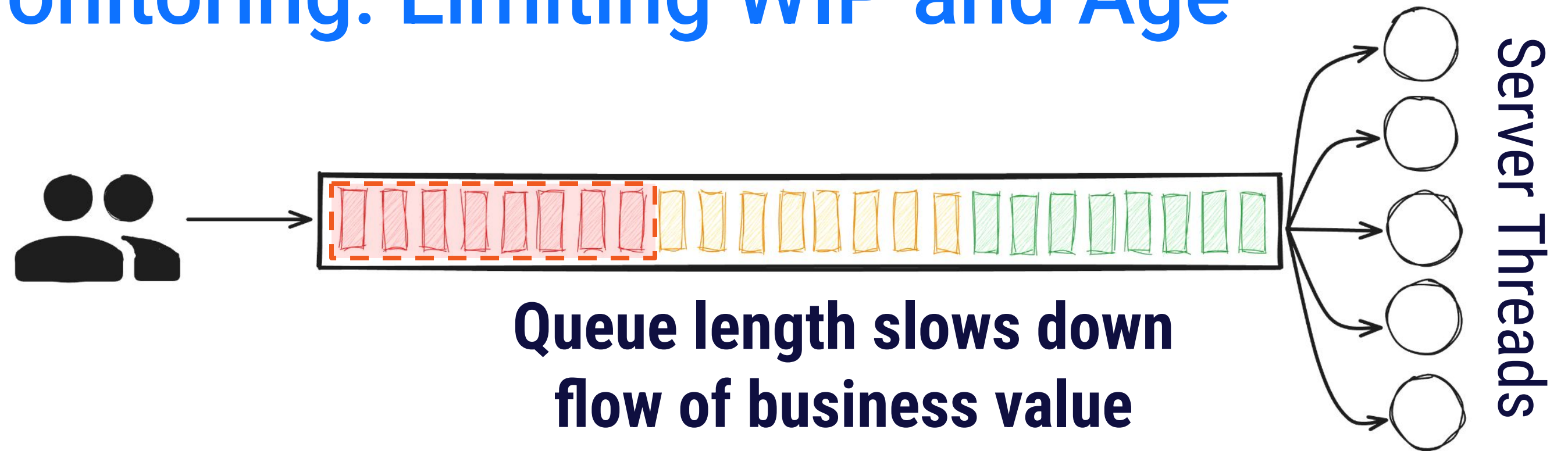
Just detecting
congestion is not
good enough

On-the-fly Resilience with Flow Metrics

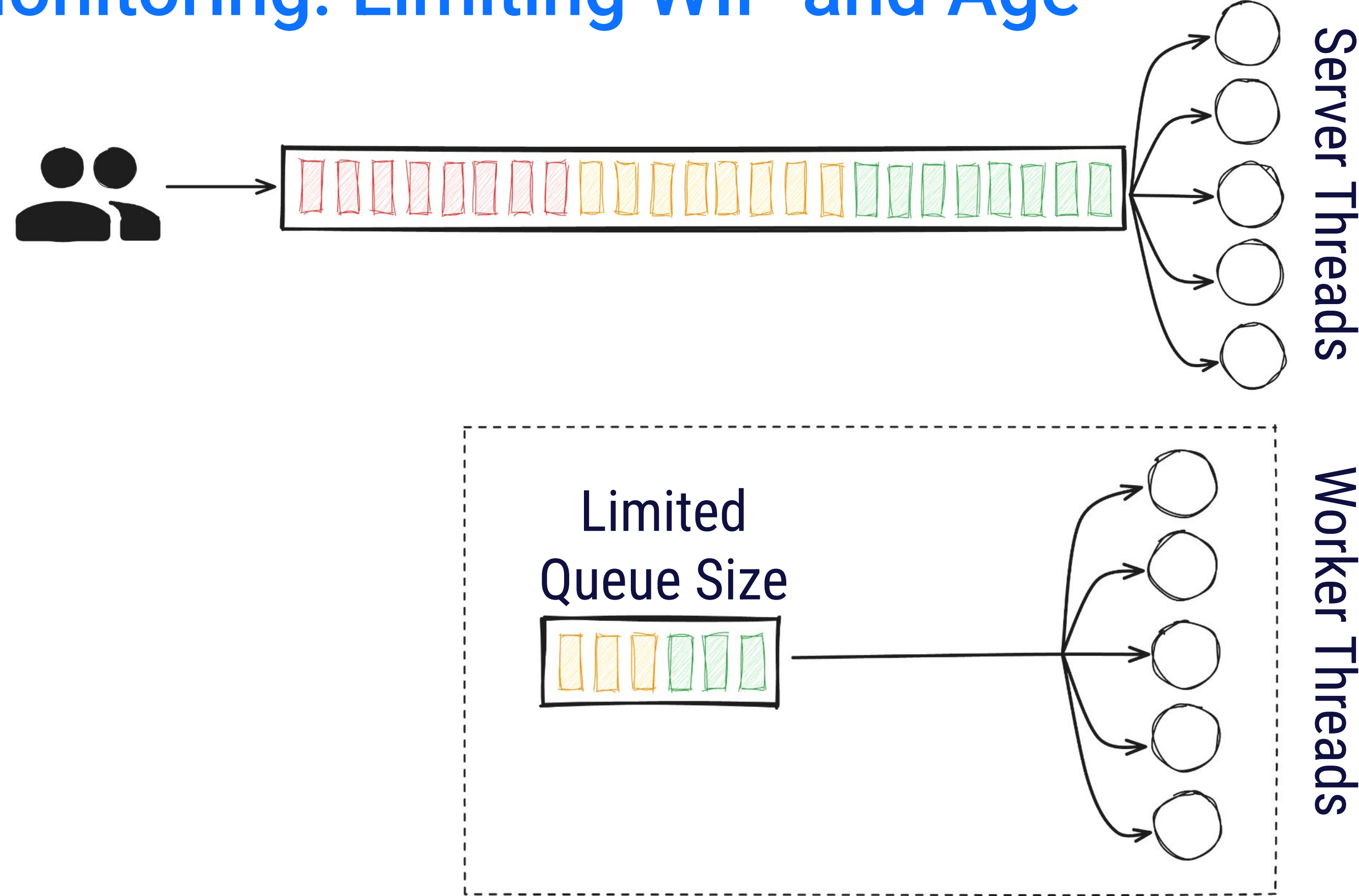
Beyond Monitoring: Limiting WIP and Age



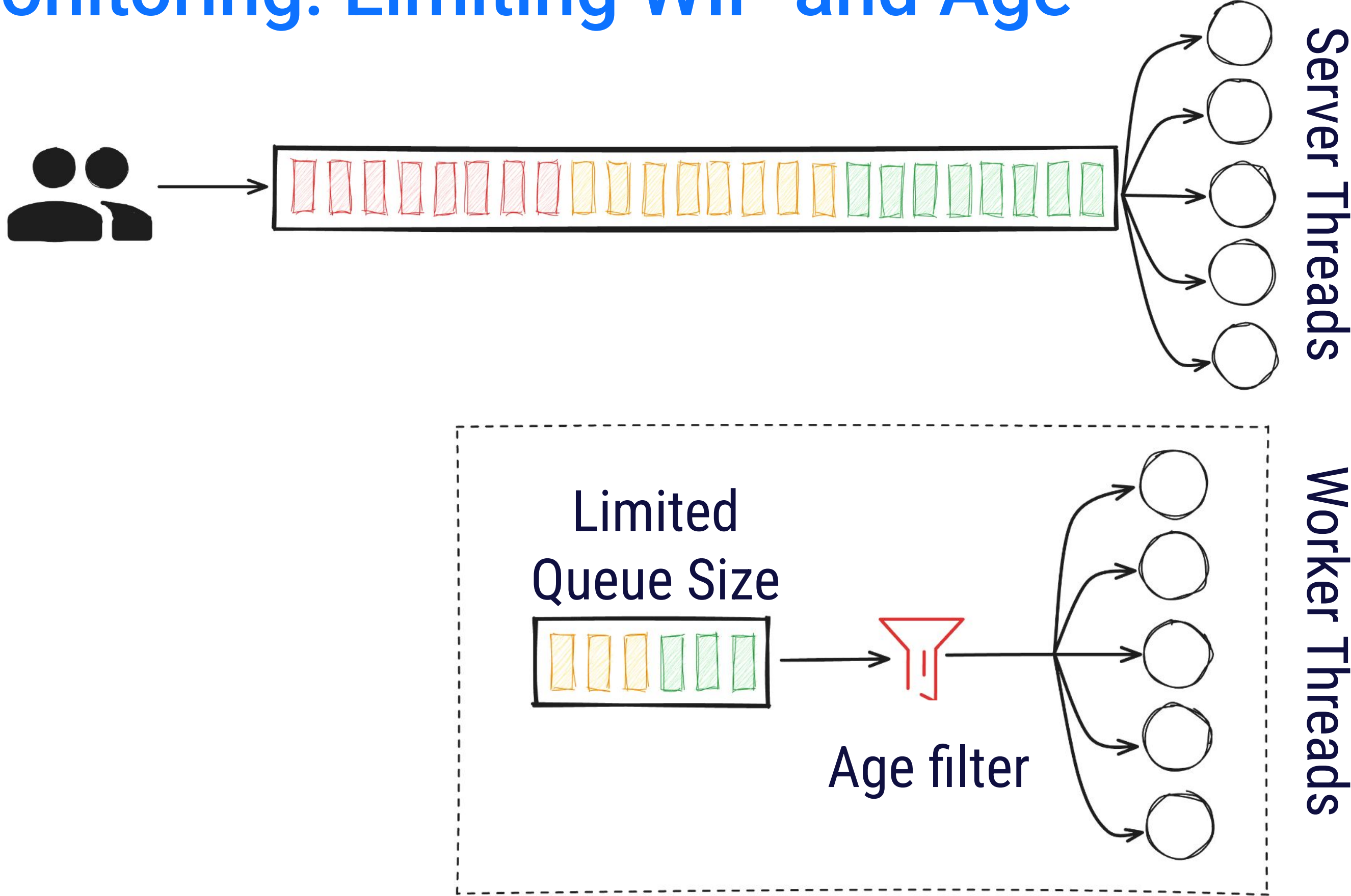
Beyond Monitoring: Limiting WIP and Age



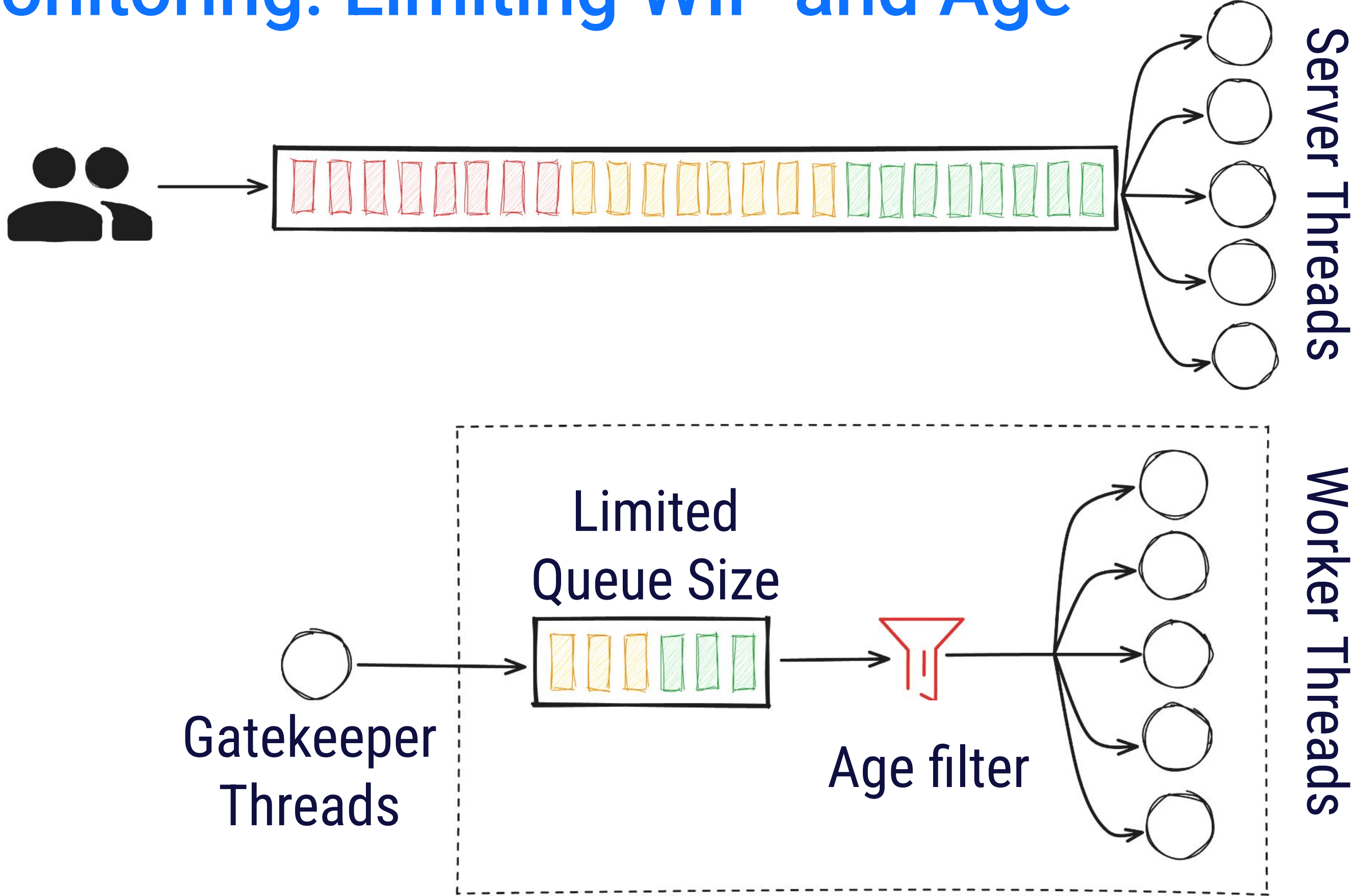
Beyond Monitoring: Limiting WIP and Age



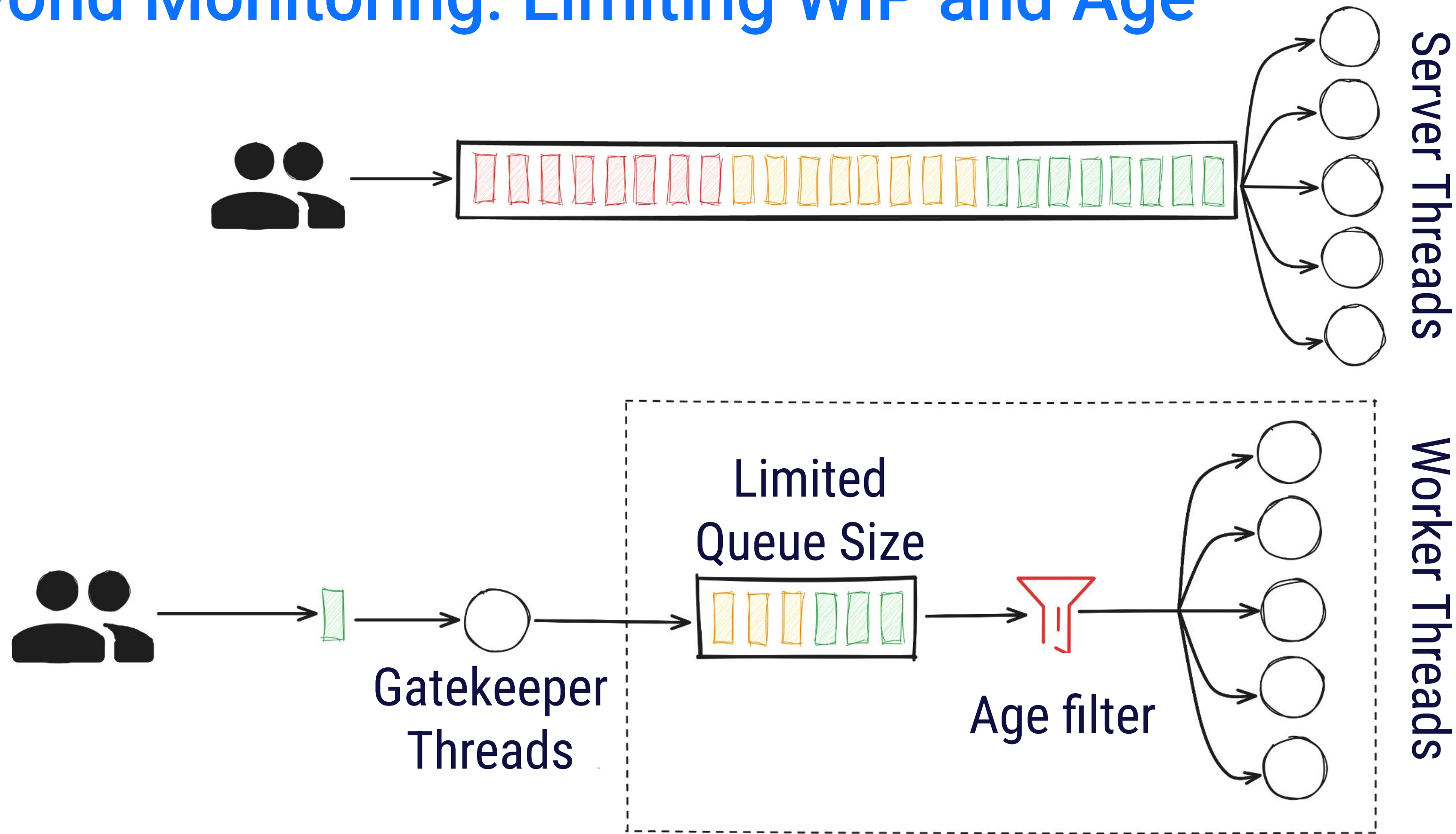
Beyond Monitoring: Limiting WIP and Age



Beyond Monitoring: Limiting WIP and Age

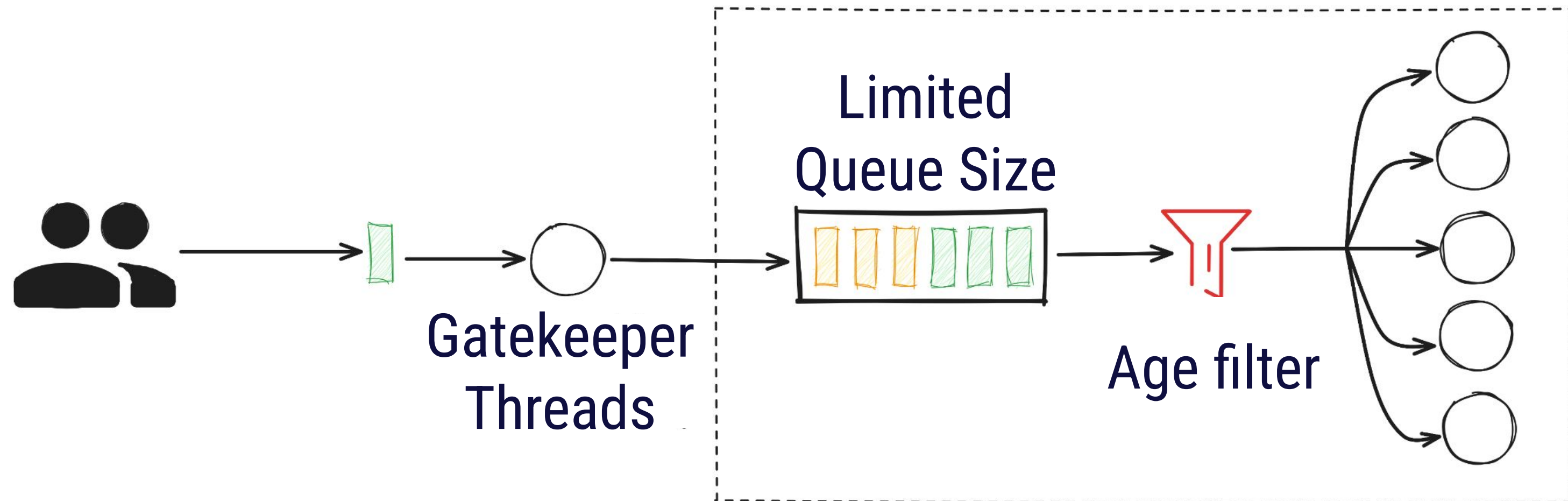


Beyond Monitoring: Limiting WIP and Age



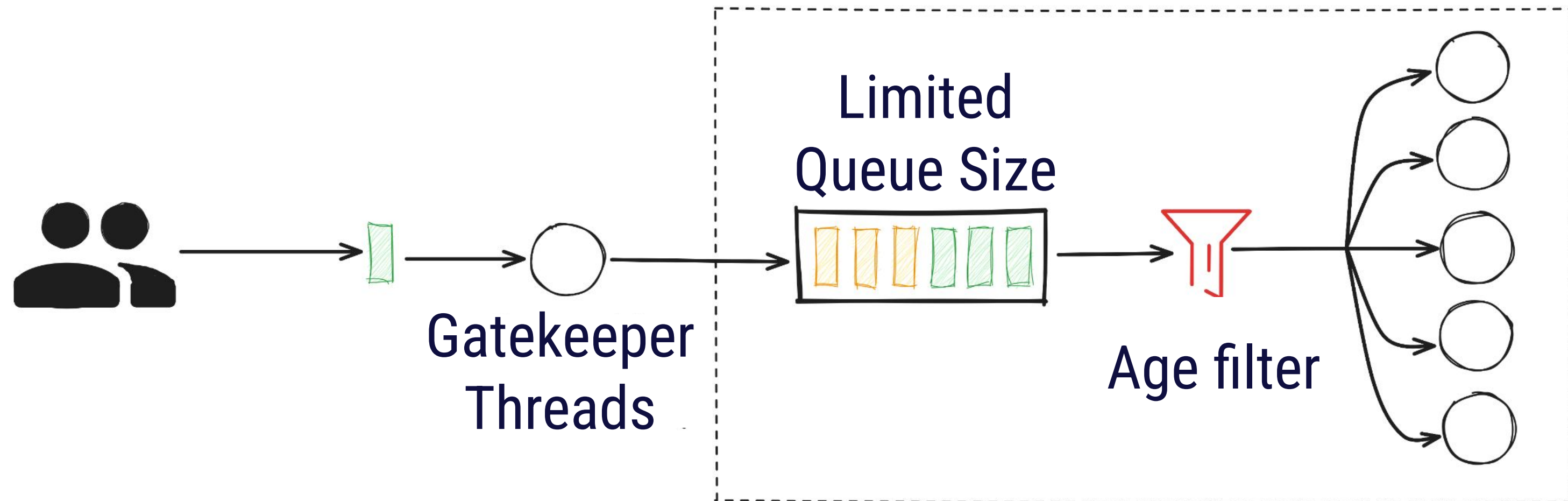
Why Limit WIP and Age?

- **Bounded wait-time for clients**



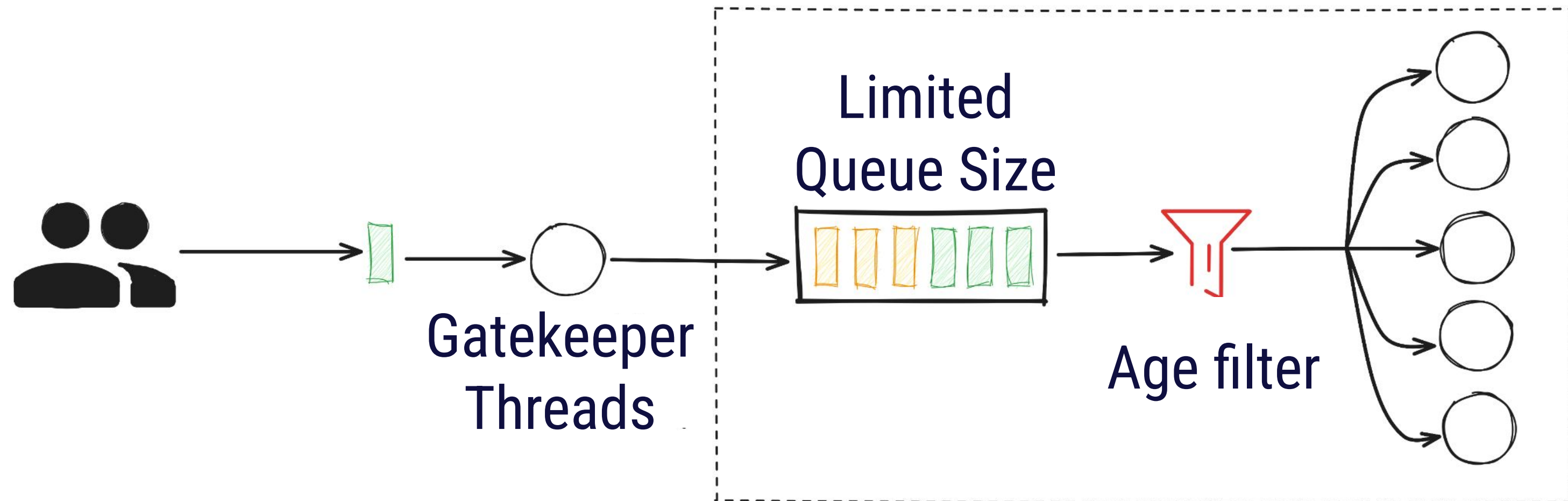
Why Limit WIP and Age?

- **Bounded wait-time** for clients
- **Effective communication** when there is congestion



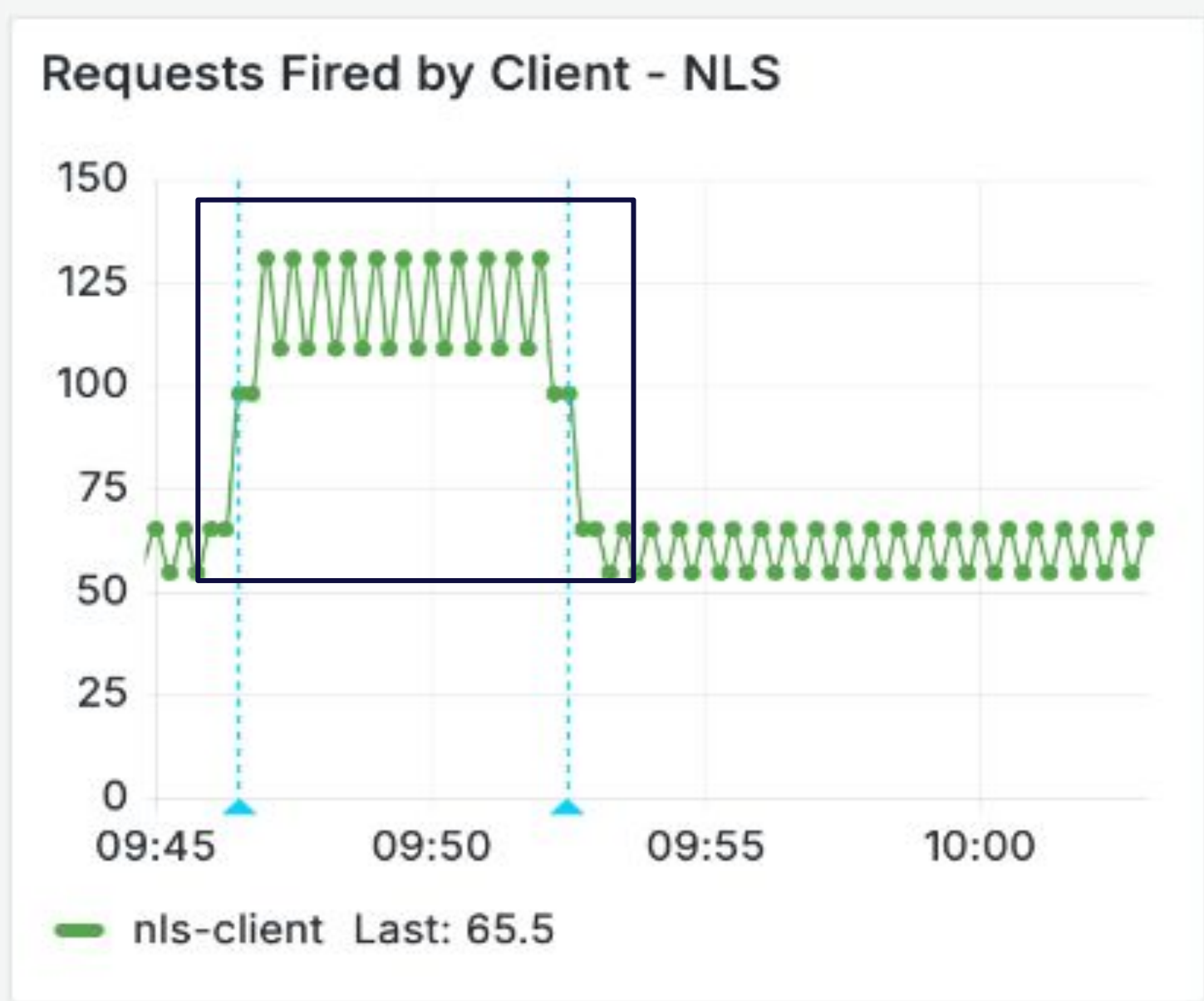
Why Limit WIP and Age?

- **Bounded wait-time** for clients
- **Effective communication** when there is congestion
- **Faster recovery** after incident



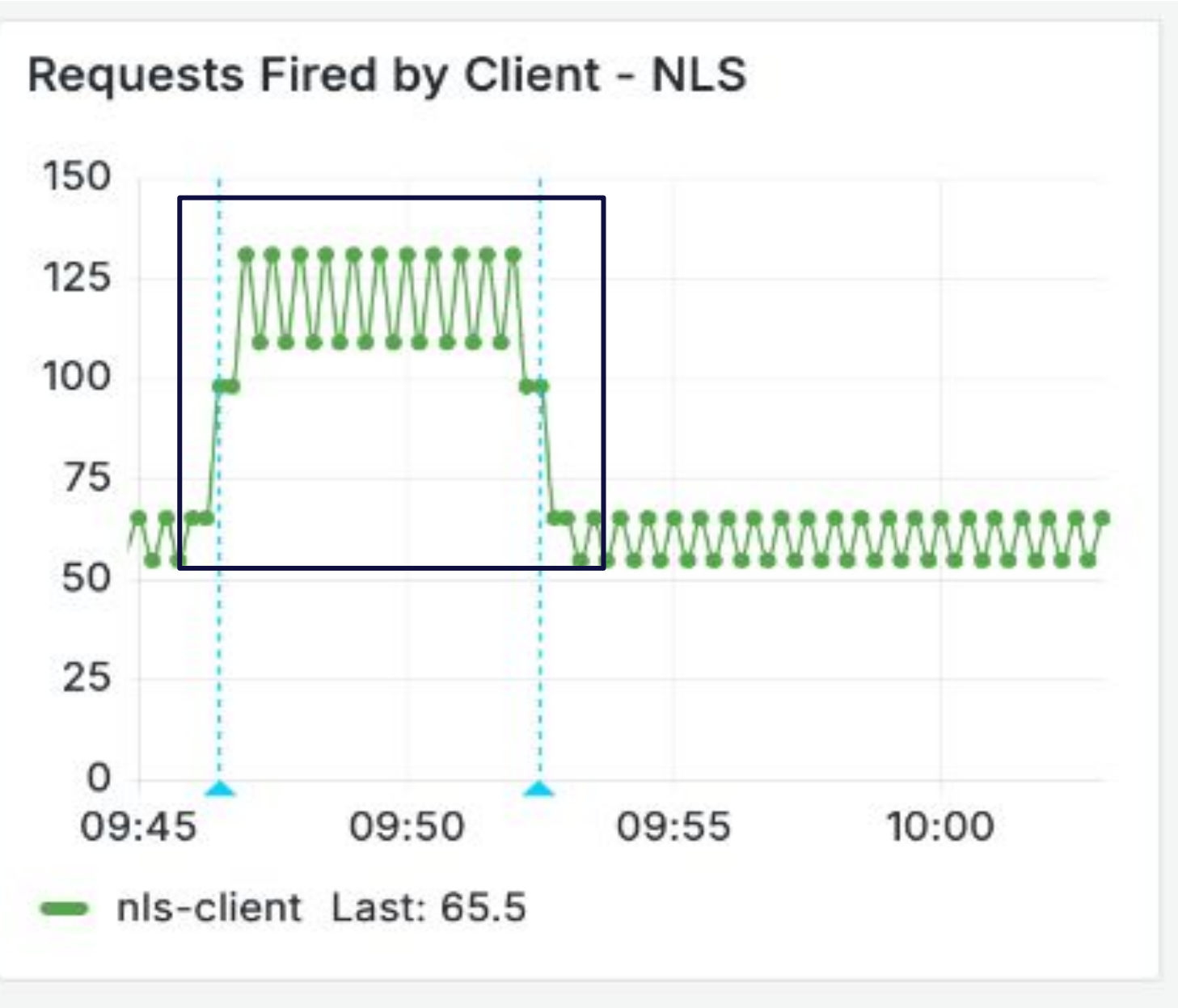
Result 1: Spike in Incoming Requests

Default Spring Boot Server



Result 1: Spike in Incoming Requests

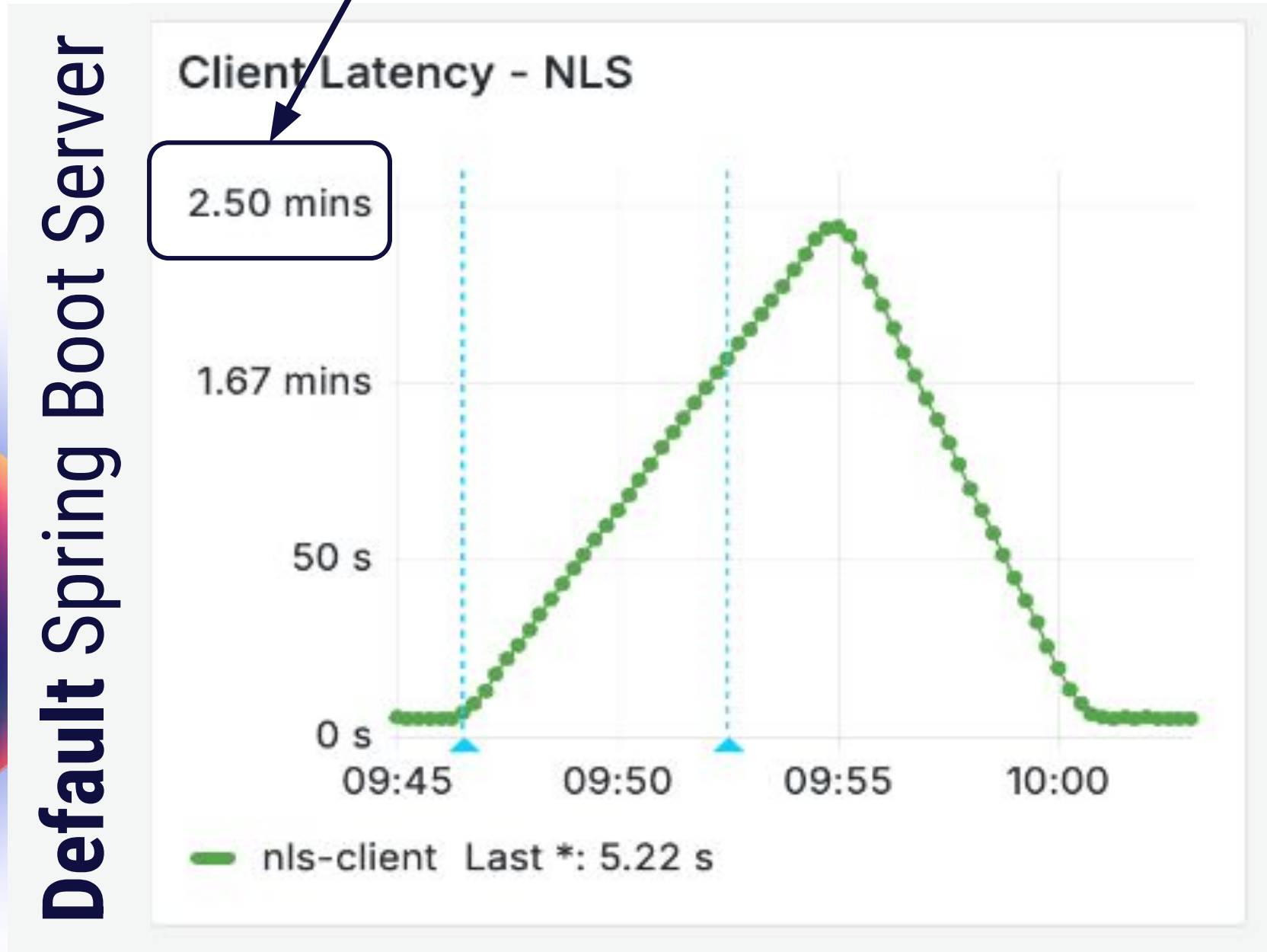
Default Spring Boot Server



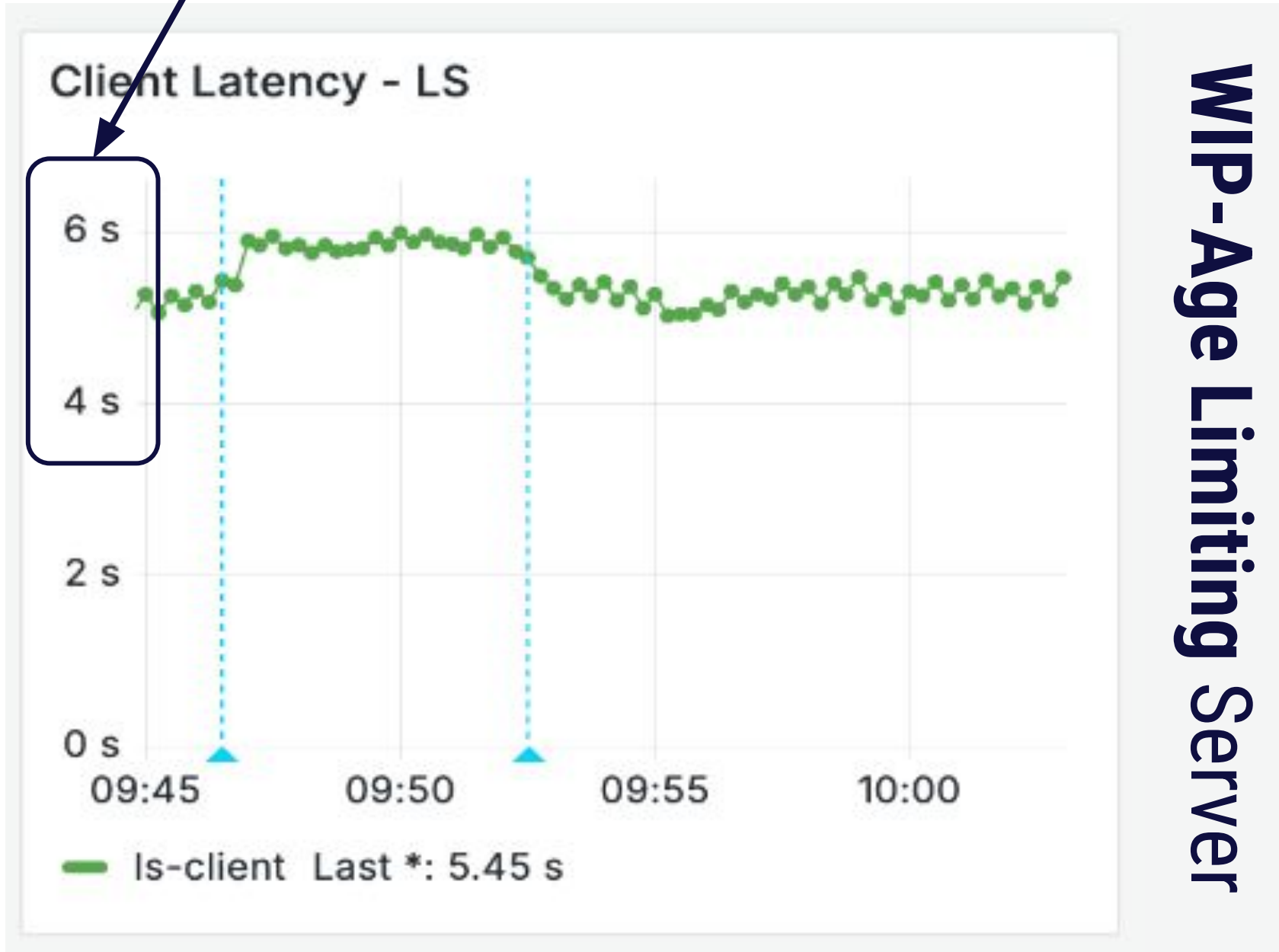
WIP-Age Limiting Server

Result 1: Spike in Incoming Requests

Unresponsive



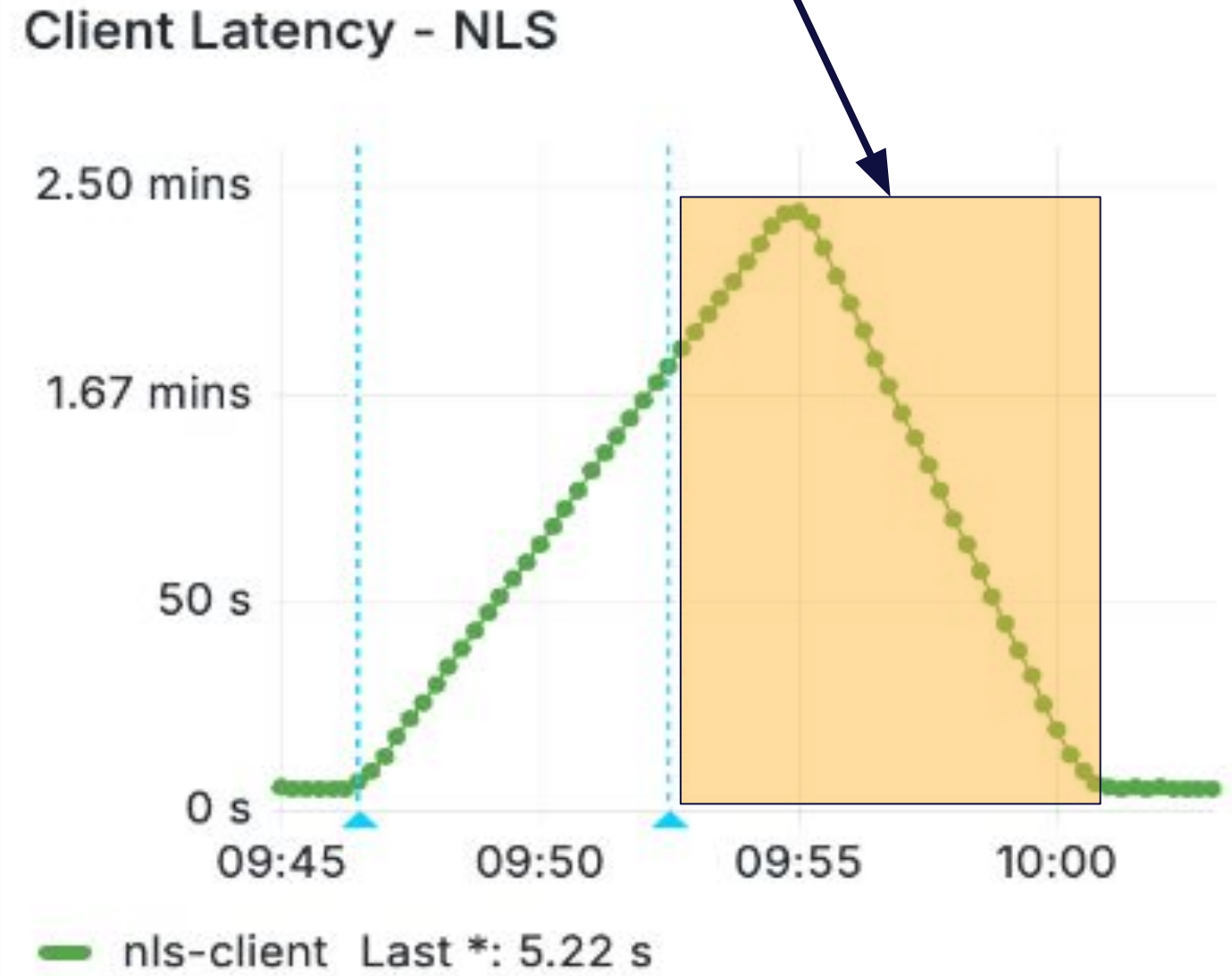
Responds in time



Result 1: Spike in Incoming Requests

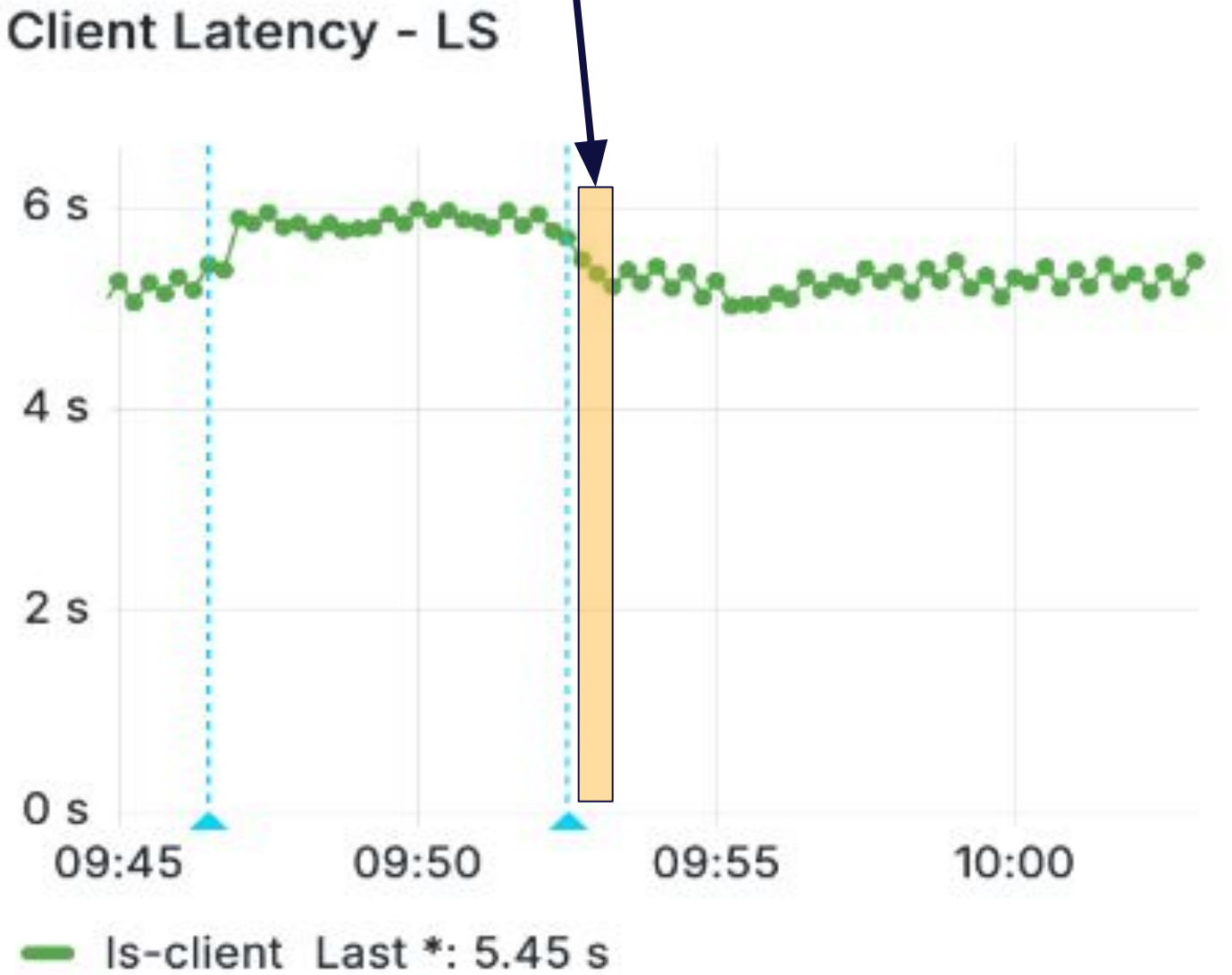
Time to recover is longer

Default Spring Boot Server



Recovers immediately

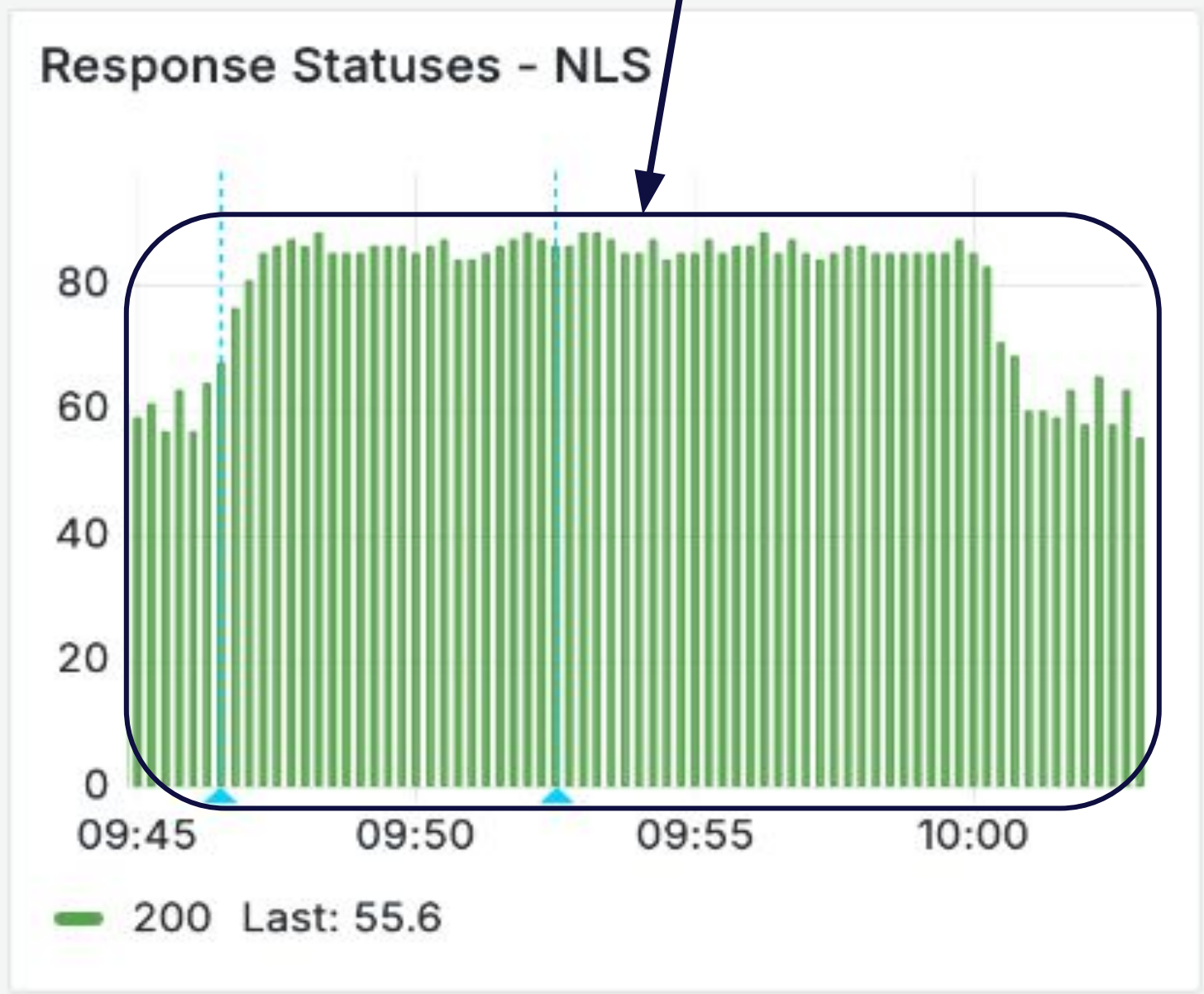
WIP-Age Limiting Server



Result 1: Spike in Incoming Requests

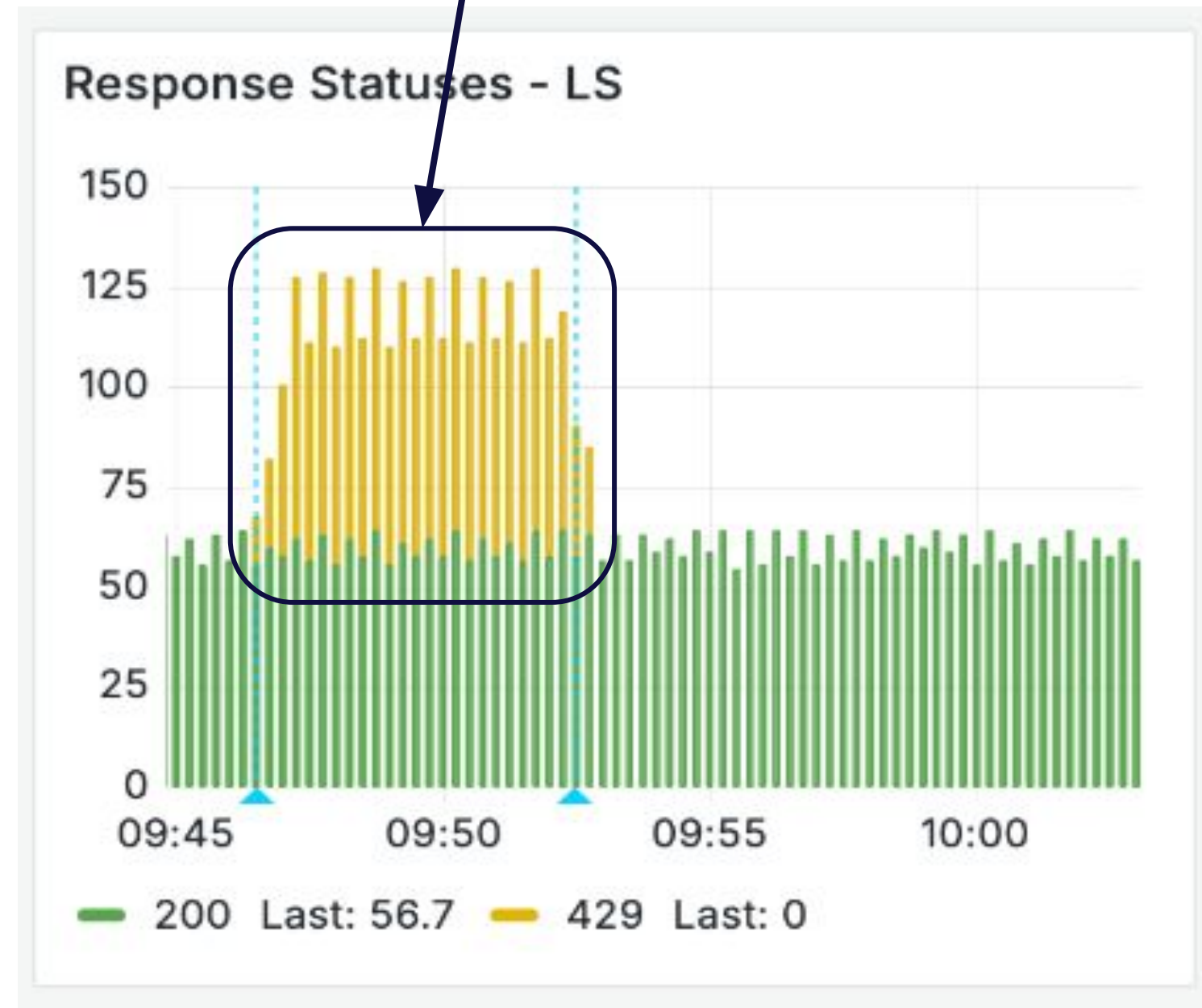
Pretends everything is 200 OK

Default Spring Boot Server

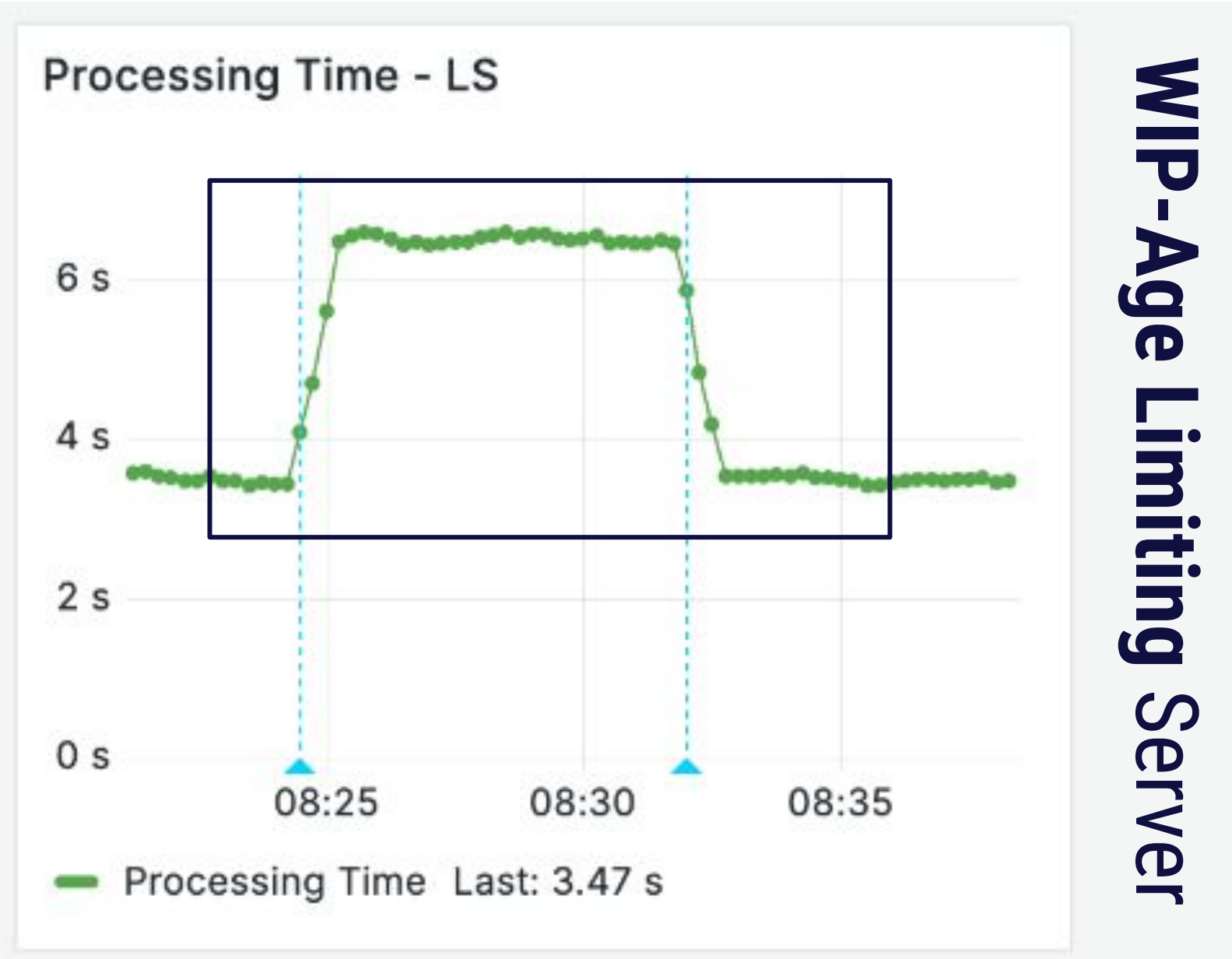


Graceful degradation

WIP-Age Limiting Server



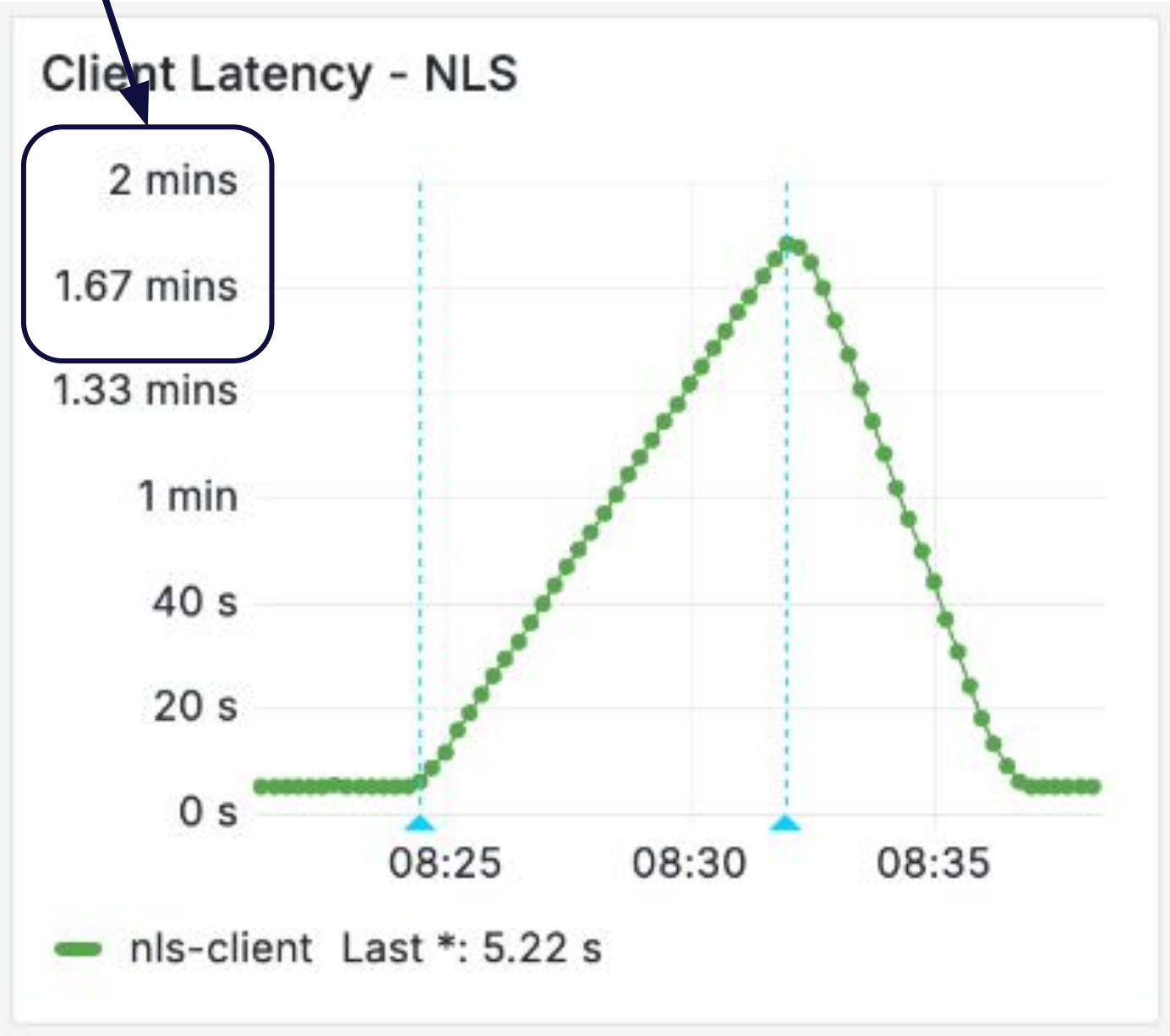
Result 2: Degraded Dependency



Result 2: Degraded Dependency

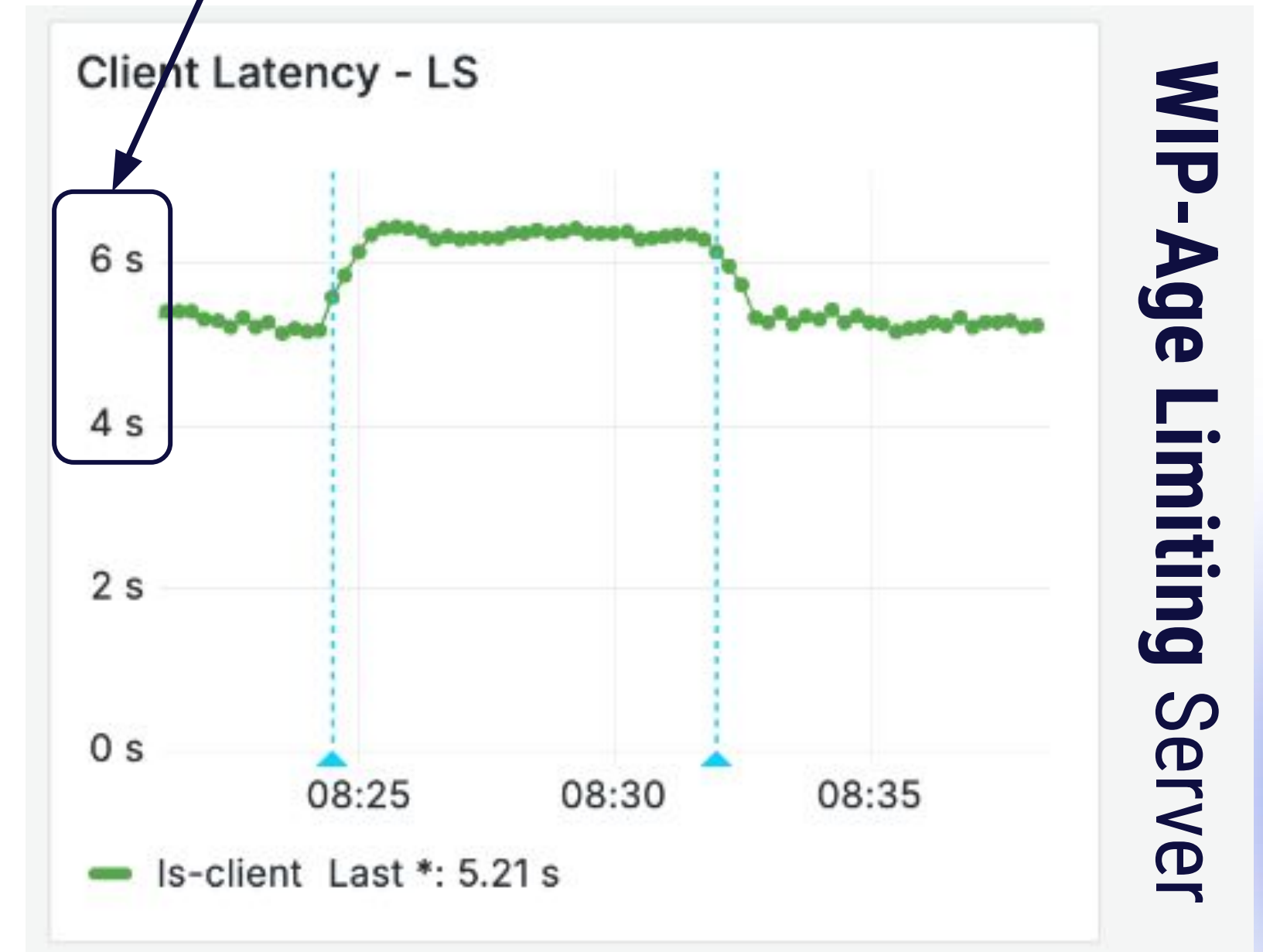
Unresponsive

Default Spring Boot Server



Responds in time

WIP-Age Limiting Server

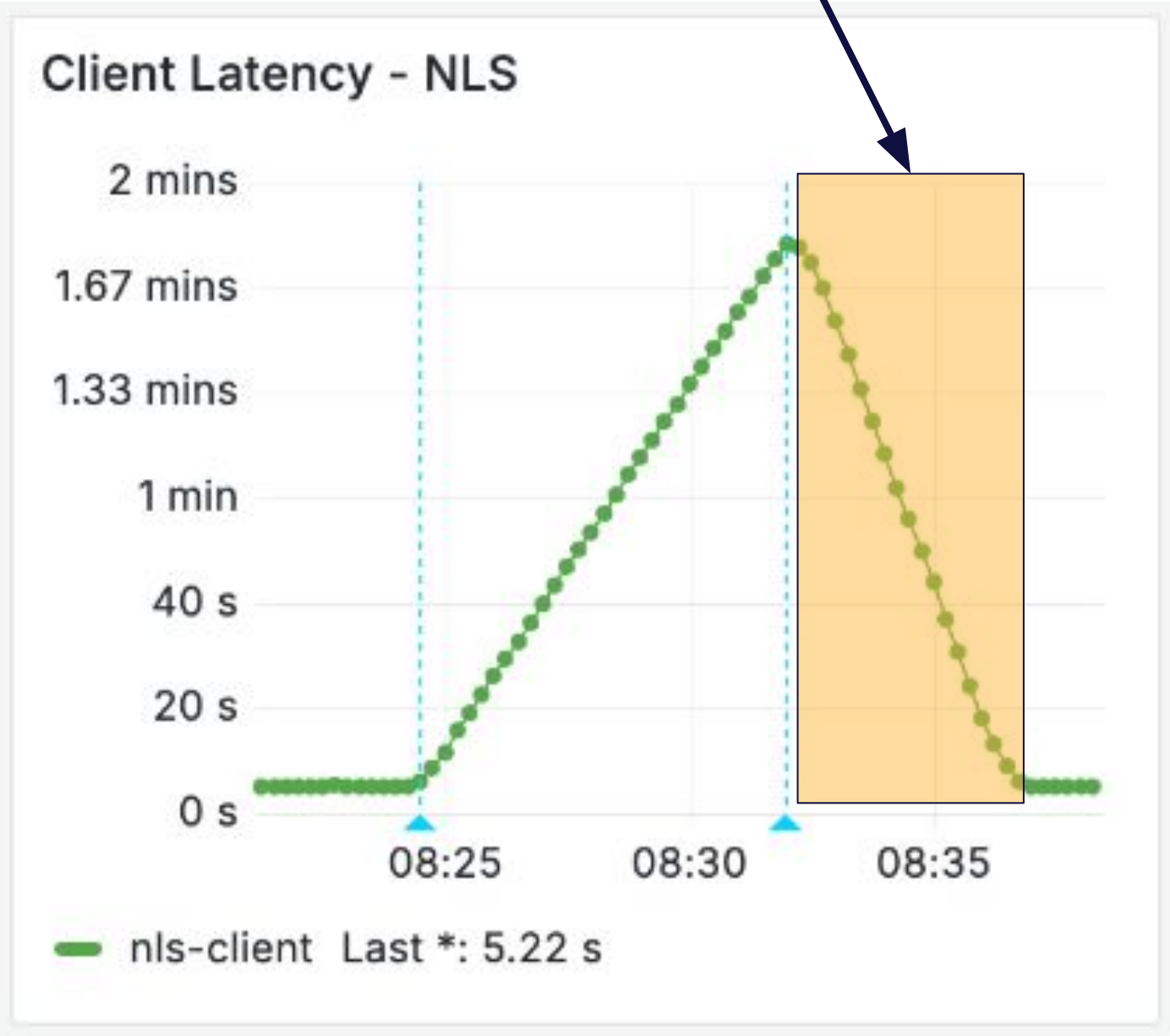


Result 2: Degraded Dependency

Time to recover is longer



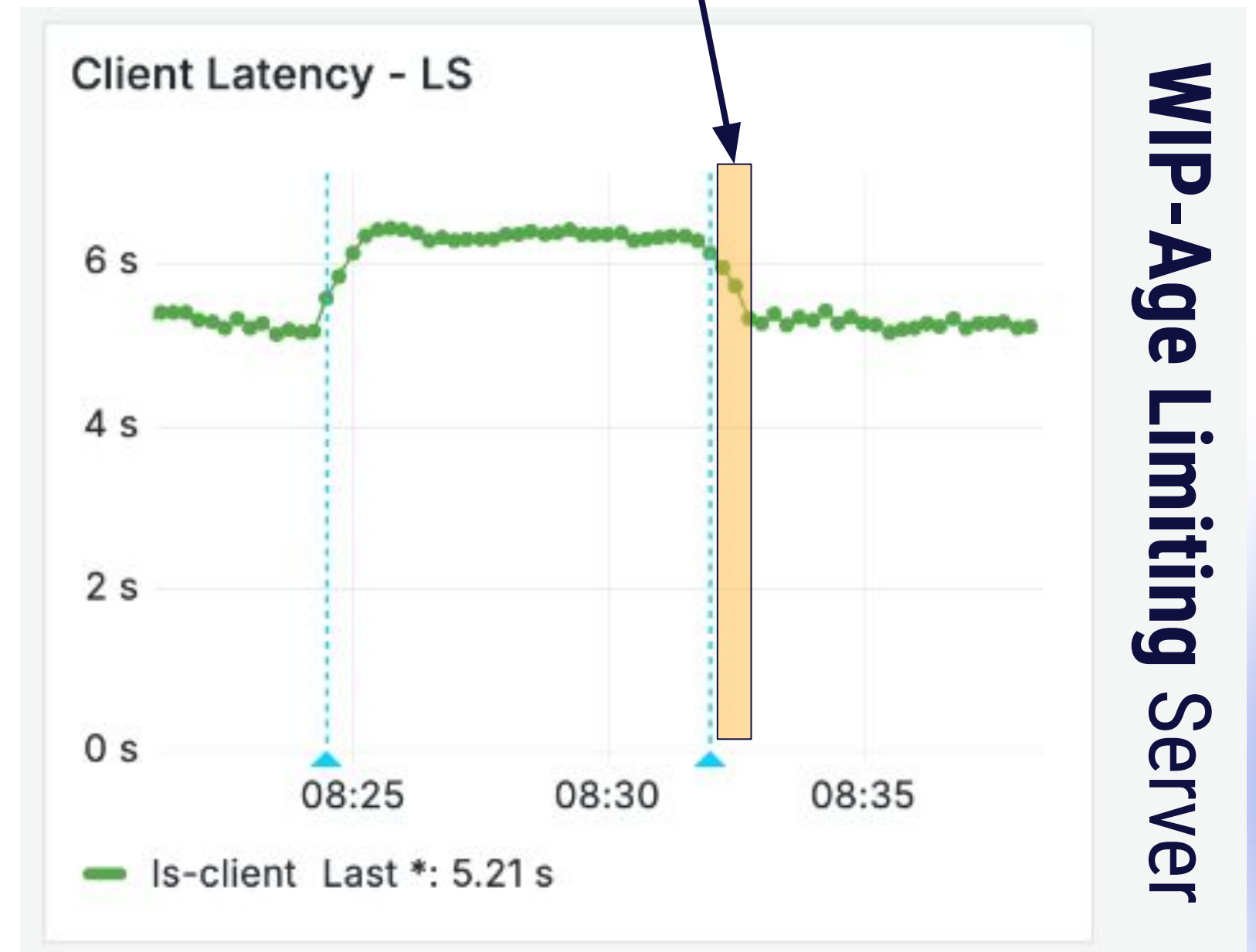
Default Spring Boot Server



Recovers immediately



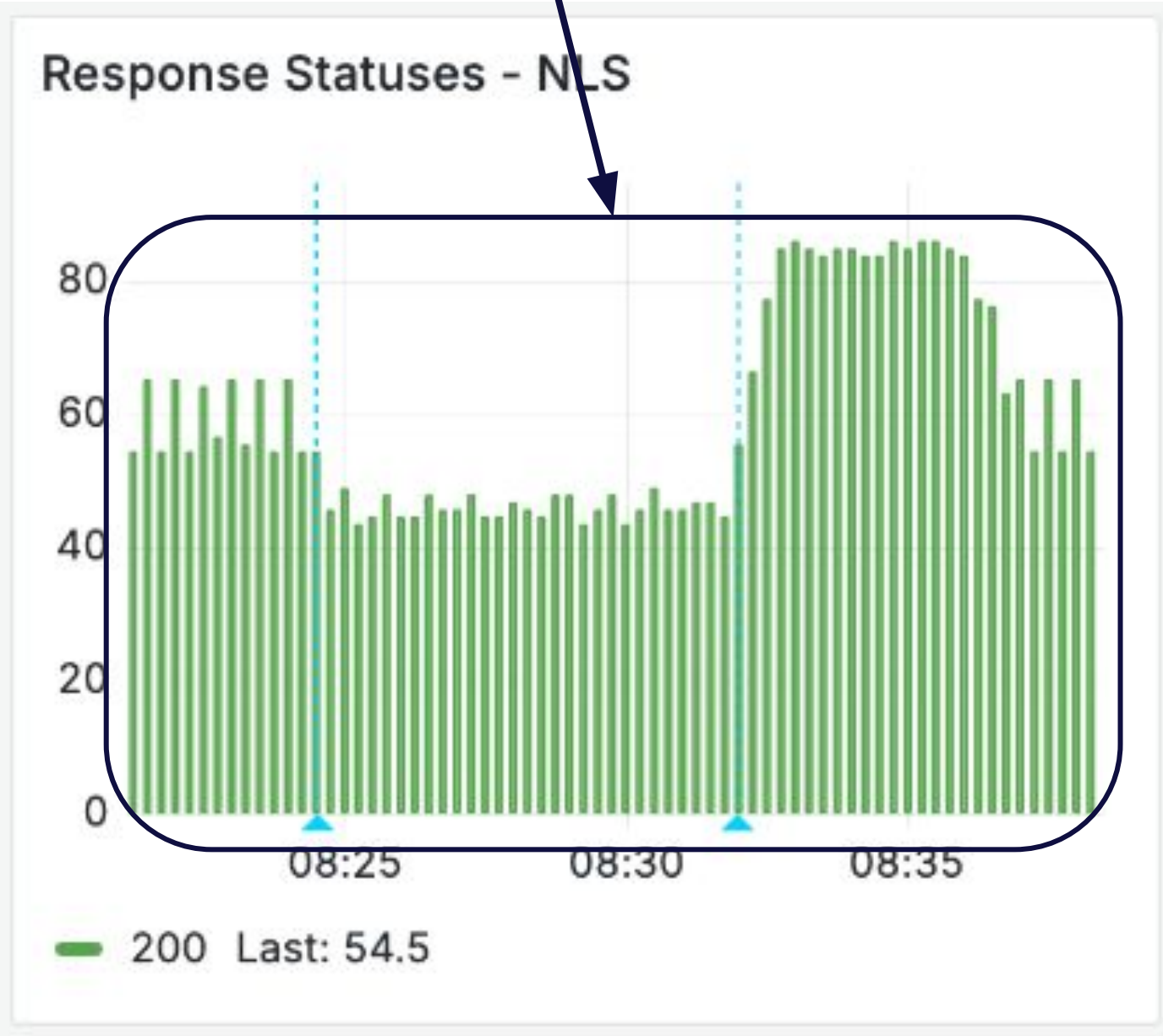
WIP-Age Limiting Server



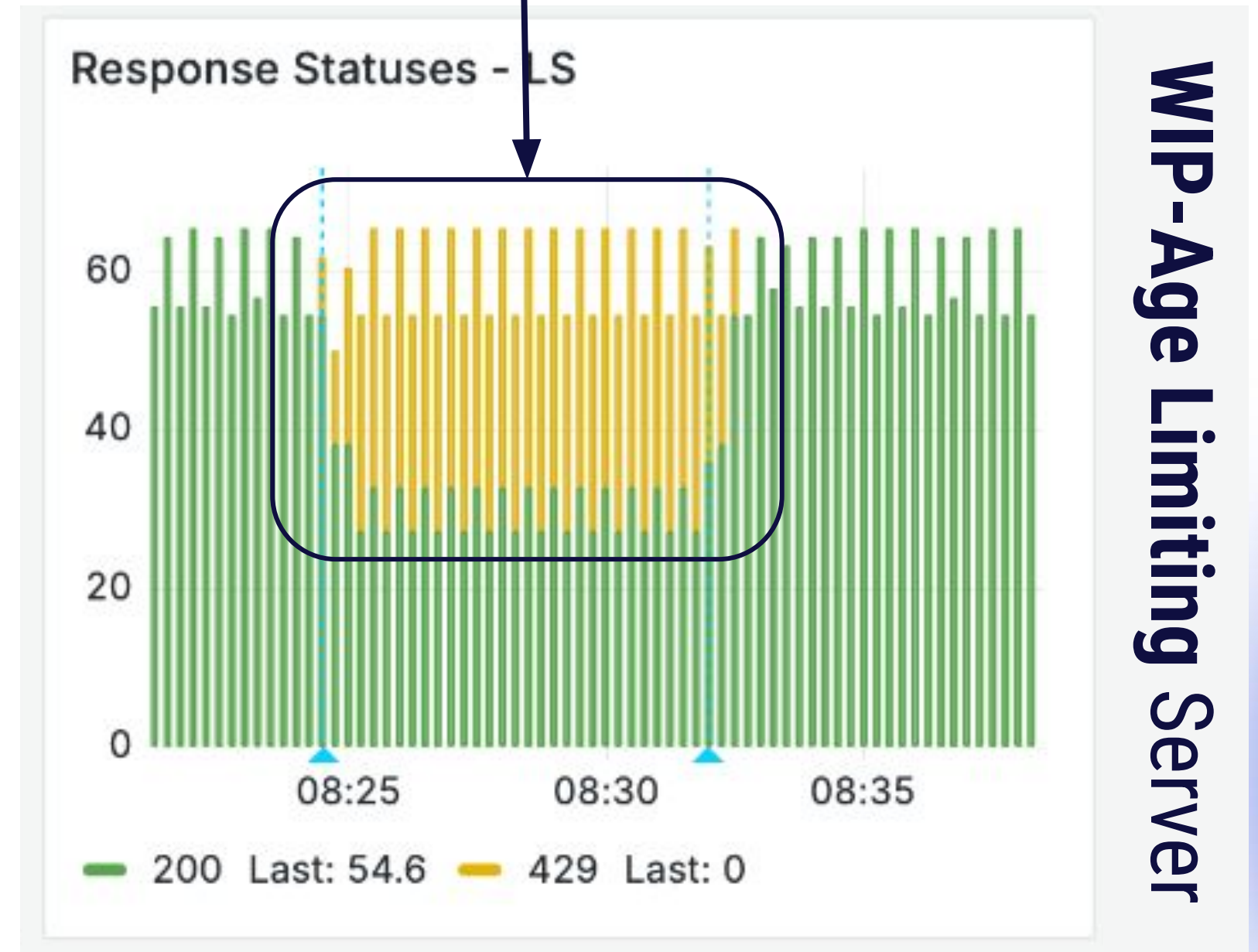
Result 2: Degraded Dependency

Pretends everything is 200 OK

Default Spring Boot Server



Graceful degradation

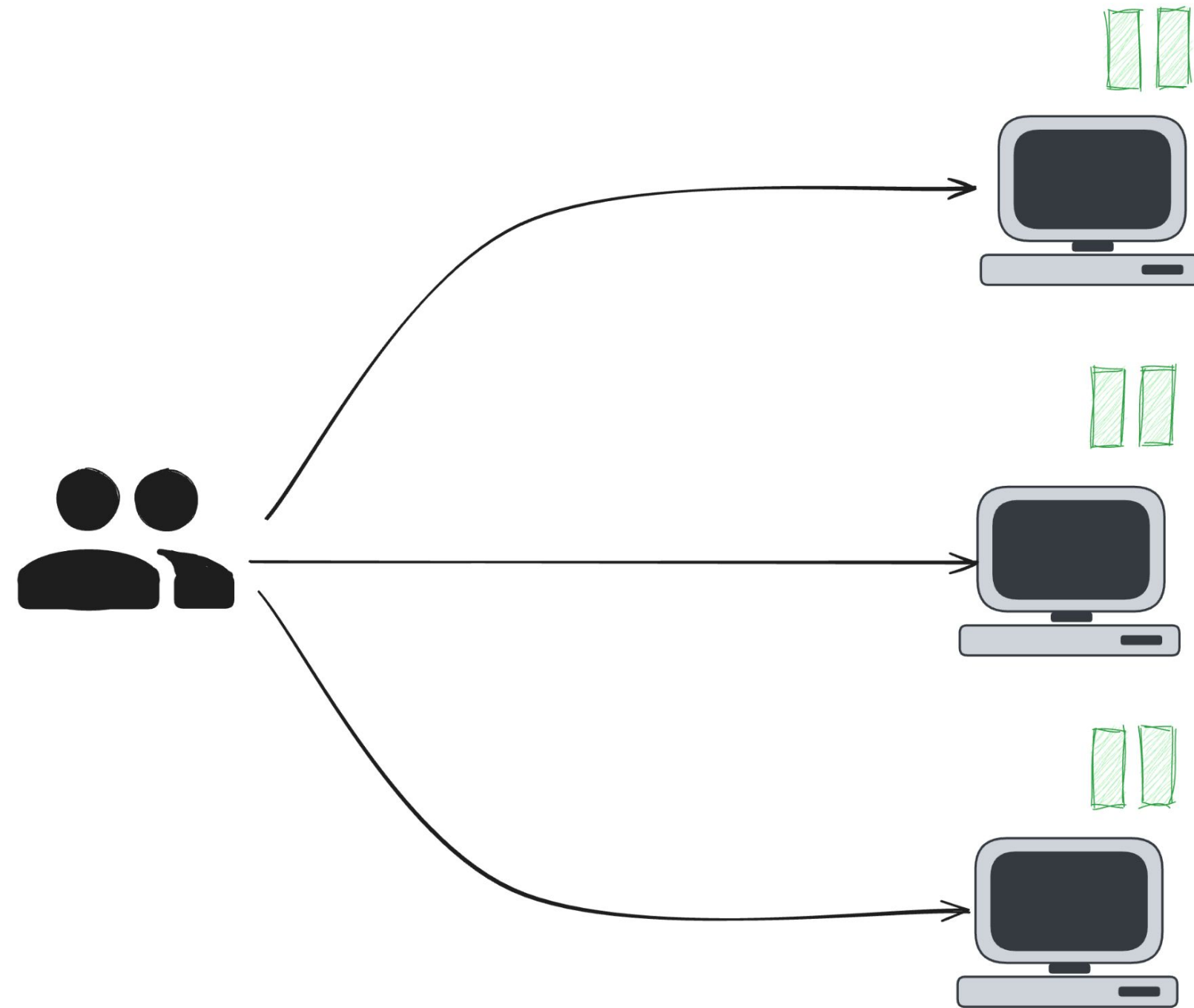


End-to-End Resilience with Flow Metrics

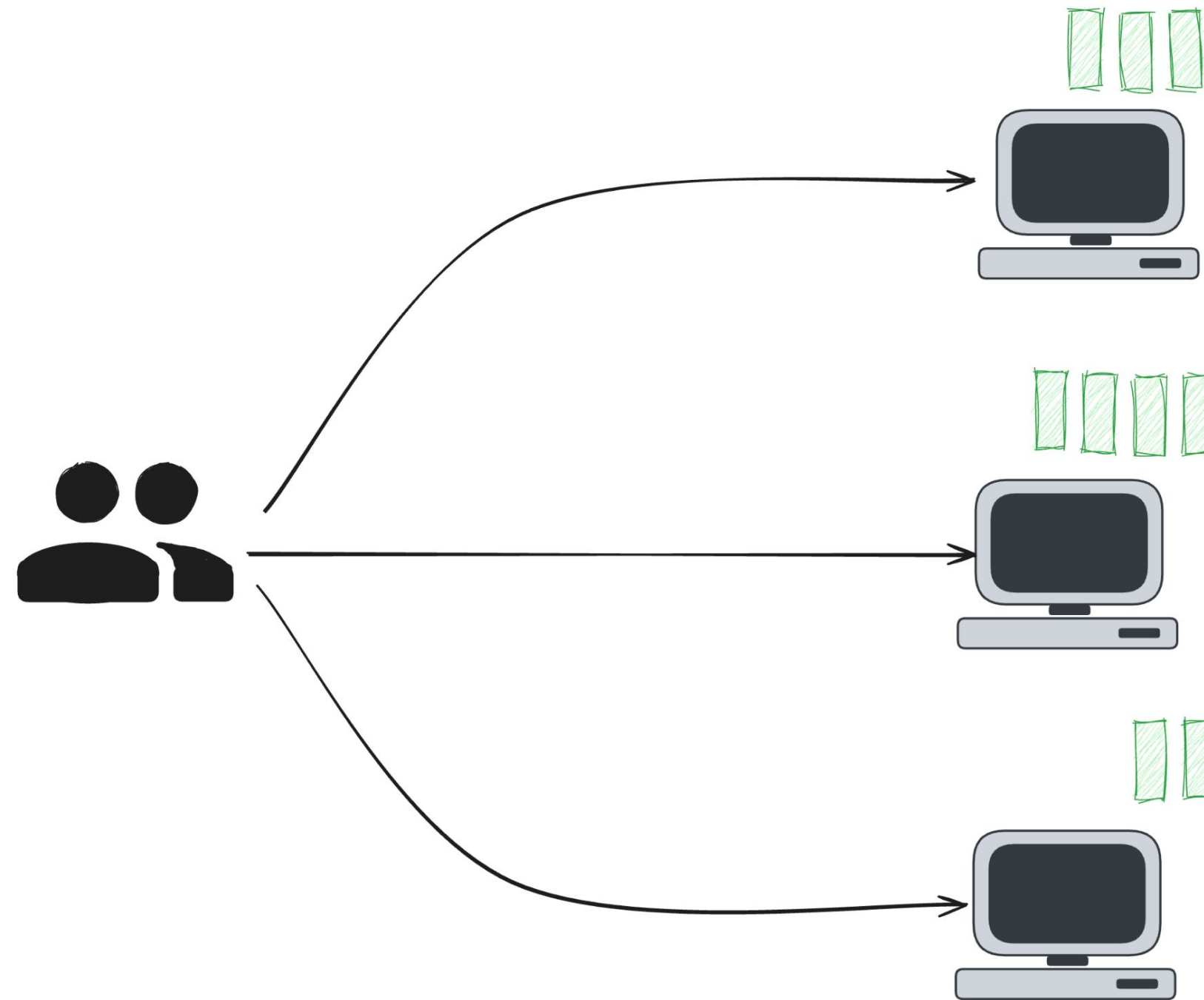
Local Measurements to Global Resilience



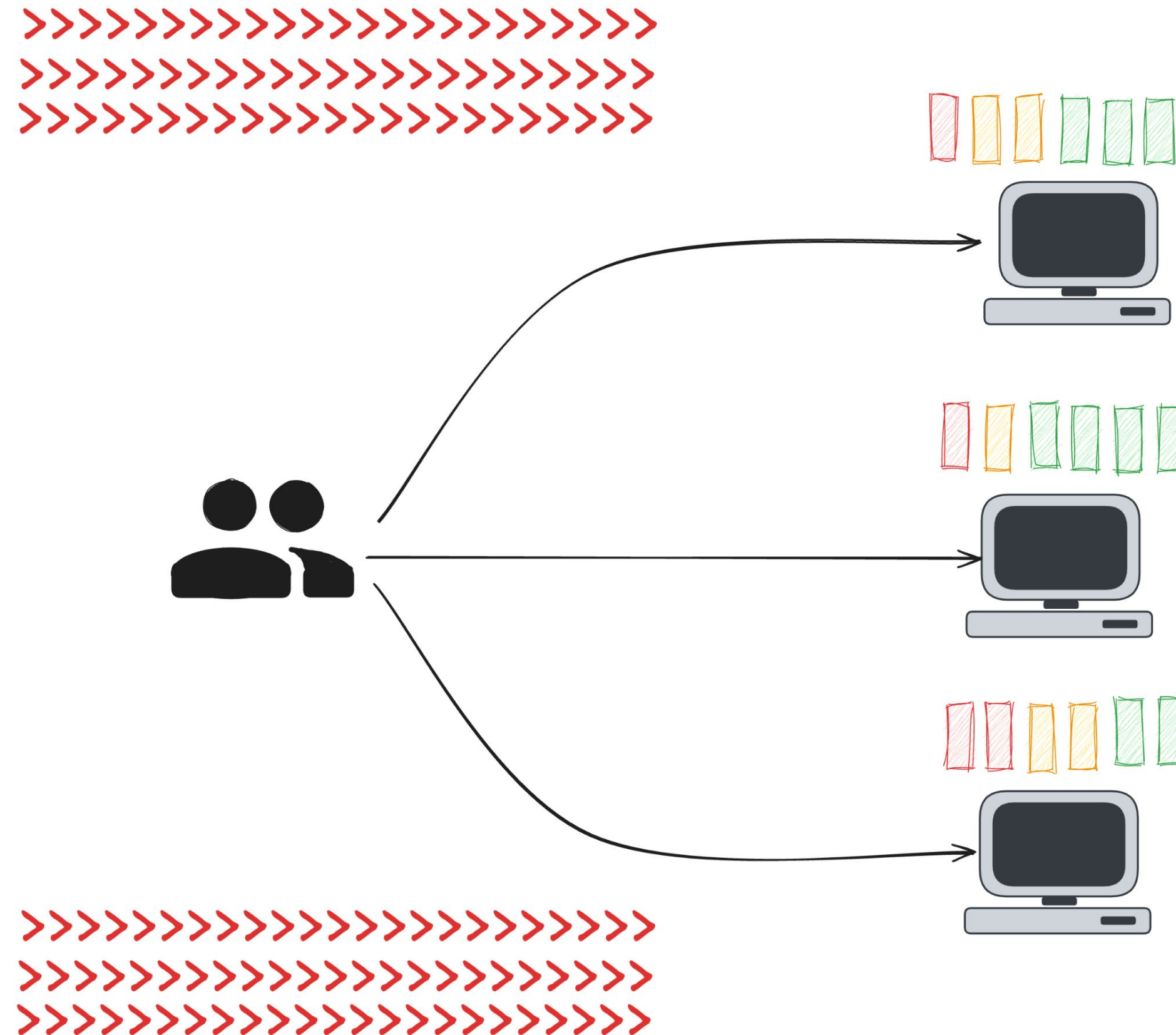
Local Measurements to Global Resilience



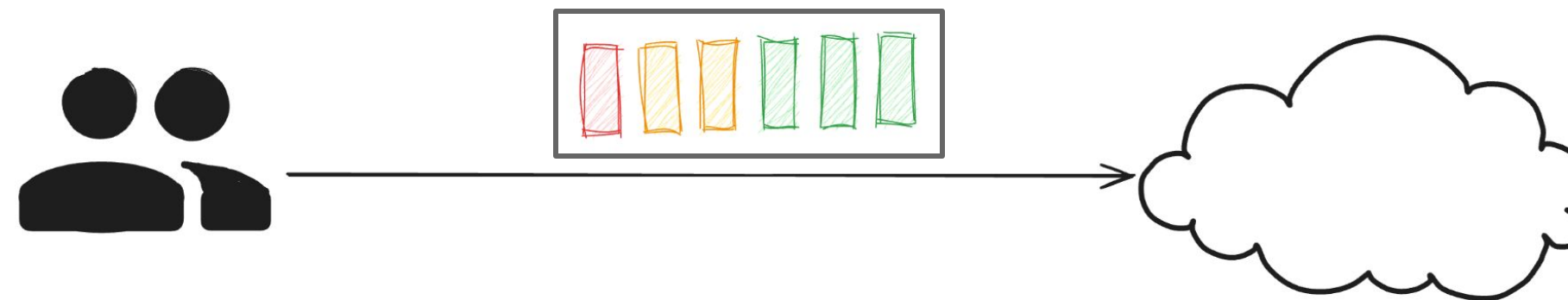
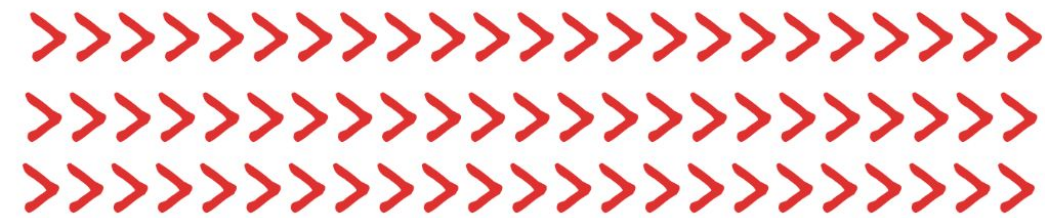
Local Measurements to Global Resilience



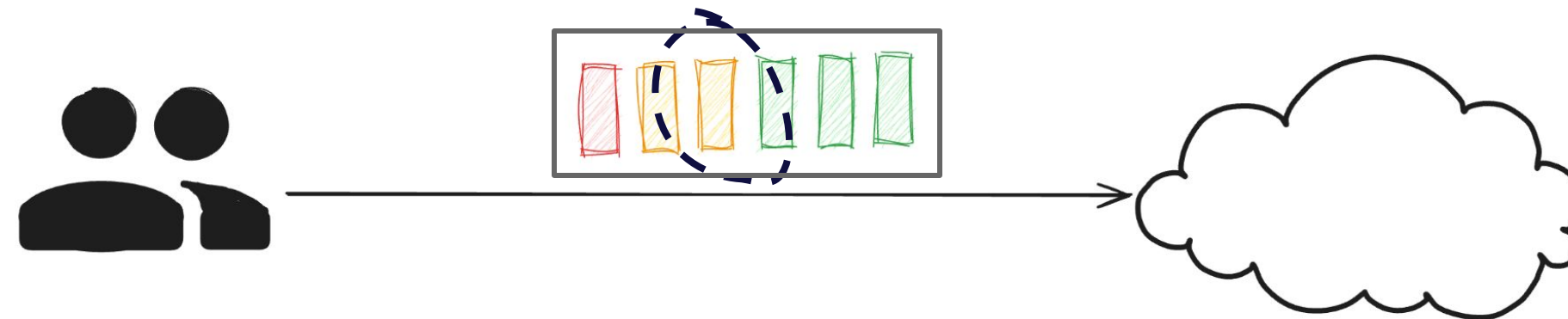
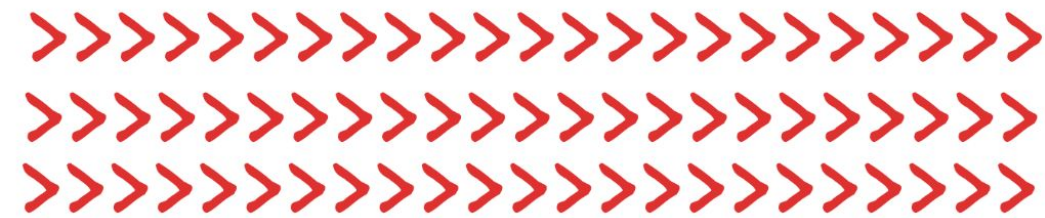
Local Measurements to Global Resilience



Local Measurements to Global Resilience

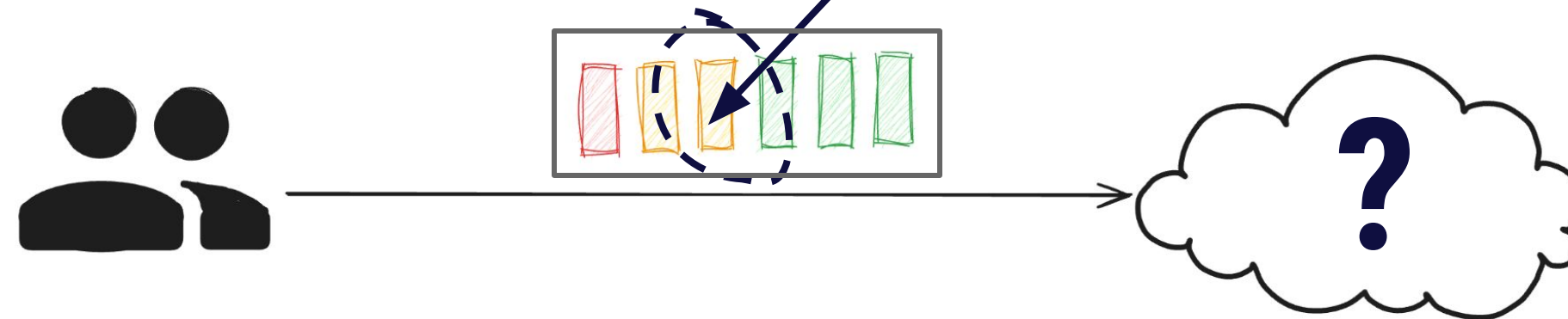


Local Measurements to Global Resilience



Local Measurements to Global Resilience

Measure the **flow of individual requests**, not the server's capacity



Independent of System Characteristics

- Machines in a Cluster
- Memory and Garbage Collection
- Distribution of type of requests
- Number of downstream dependencies
- CPU Utilization
- ...

Load is not what you should balance: Introducing Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

Abstract

We present Prequal (*Probing to Reduce Queuing and Latency*), a load balancer for distributed multi-tenant systems. Prequal aims to minimize real-time request latency in the presence of heterogeneous server capacities and non-uniform, time-varying antagonist load. It actively probes server load to leverage the *power of d choices* paradigm, extending it with asynchronous and reusable probes. Cutting against received wisdom, Prequal does not balance CPU load, but instead selects servers according to estimated latency and active requests-in-flight (RIF). We explore its major design features on a testbed system and evaluate it on YouTube, where it has been deployed for more than two years. Prequal has dramatically decreased tail latency, error rates, and resource use, enabling YouTube and other production systems at Google to run at much higher utilization.

1 Introduction

We report our experience deploying the power of d choices (PodC) load balancing paradigm [1, 2, 18] to run multiple large-scale web services at Google, focusing on YouTube, where it has run successfully for two years. To our knowledge, this is the first public report of PodC being used successfully for load balancing at this scale.

The PodC paradigm involves sampling $d \geq 2$ servers for their load, and sending the next request to the server with minimum load. The two central questions for any implementation are: (1) what signal is used to represent the load, and (2) how is the sampling done? We answer those questions with

primary new innovations. First, we combine RIF and latency in a new way that works especially well, called the *hot-cold lexicographic* (HCL) rule. Second, we introduce a novel asynchronous probing mechanism that reduces probing overheads (CPU + critical-path latency) while retaining the freshness of the load signal offered by synchronous probing.

For purposes of this paper, we refer to Prequal as a load balancing *system*, but technically, it is load a balancing *policy* implemented within Google’s Stubby RPC framework (externally, gRPC [24]). As such, we expect that some of our innovations could be integrated into these other existing load balancers. Our main thrust is the two innovations above (HCL, async probing); other implementation details of our system are incidental to our message.

Prequal has been deployed in a diverse collection of 20+ large-scale services at Google over the past two years. In addition to driving most of the YouTube serving stack, it has found success in a wide variety of other applications, including search ads, logs processing, and serving ML models. In the typical application, query processing times are $O(10 - 100)$ milliseconds, but in one system the queries take $O(10)$ minutes, and in another they range from seconds to hours. Most of these services are part of a complex tangle of services calling each other, and we have seen benefits from deploying Prequal for the most critical services, regardless of whether others in the tangle are also using Prequal.

We care about CPU and latency overheads from probing, but in our deployment experience we have found them to be small and more than pay for themselves. Probe response times within a data center are well below 1 millisecond. For some applications, fractions of a millisecond matter, so we

Load is not what you should balance: Introducing Prequal

Bartek Wydrowski
Google Research

Robert Kleinberg
Google Research

Stephen M. Rumble
Google (YouTube)

Aaron Archer
Google Research

Abstract

We present Prequal (*Probing to Reduce Queuing and La-*

primary new innovations. First, we combine RIF and latency in a new way that works especially well, called the *hot-cold* *Latency-aware* (HCL) rule. Second, we introduce a novel way

with asynchronous and reusable probes. Cutting against received wisdom, Prequal does not balance CPU load, but instead selects servers according to estimated latency and active requests-in-flight (RIF). We explore its major design features

has been deployed for more than two years. Prequal has dramatically decreased tail latency, error rates, and resource use, enabling YouTube and other production systems at Google to run at much higher utilization.

1 Introduction

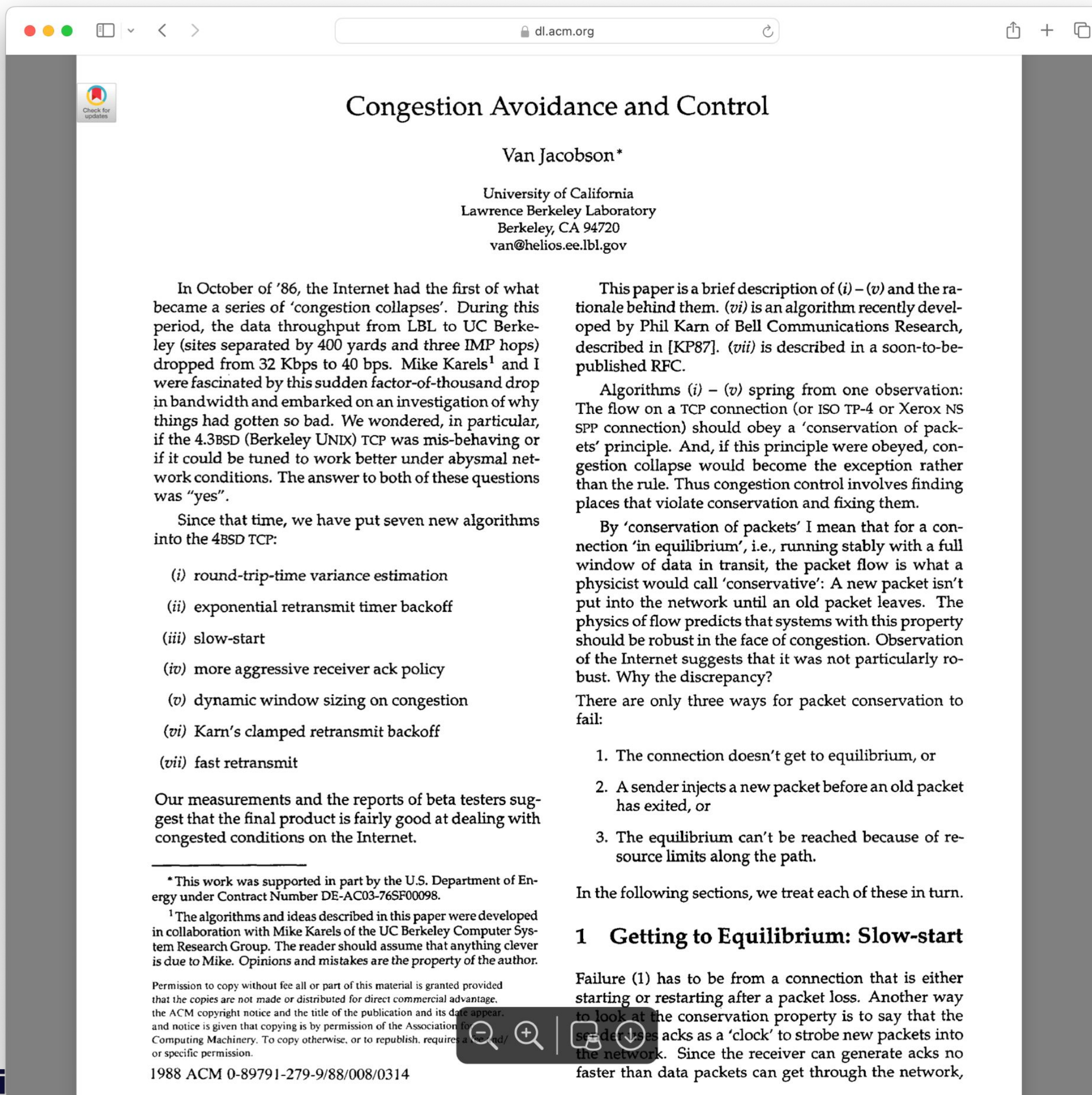
We report our experience deploying the power of d choices (PodC) load balancing paradigm [1, 2, 18] to run multiple large-scale web services at Google, focusing on YouTube, where it has run successfully for two years. To our knowledge, this is the first public report of PodC being used successfully for load balancing at this scale.

The PodC paradigm involves sampling $d \geq 2$ servers for their load, and sending the next request to the server with minimum load. The two central questions for any implementation are: (1) what signal is used to represent the load, and (2) how is the sampling done? We answer those questions with

(HCL, async probing), other implementation details of our system are incidental to our message.

Prequal has been deployed in a diverse collection of 20+ large-scale services at Google over the past two years. In addition to driving most of the YouTube serving stack, it has found success in a wide variety of other applications, including search ads, logs processing, and serving ML models. In the typical application, query processing times are $O(10 - 100)$ milliseconds, but in one system the queries take $O(10)$ minutes, and in another they range from seconds to hours. Most of these services are part of a complex tangle of services calling each other, and we have seen benefits from deploying Prequal for the most critical services, regardless of whether others in the tangle are also using Prequal.

We care about CPU and latency overheads from probing, but in our deployment experience we have found them to be small and more than pay for themselves. Probe response times within a data center are well below 1 millisecond. For some applications, fractions of a millisecond matter, so we



dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and three IMP hops) dropped from 32 Kbps to 40 bps. Mike Karels¹ and I were fascinated by this sudden factor-of-thousand drop in bandwidth and embarked on an investigation of why things had gotten so bad. We wondered, in particular, if the 4.3BSD (Berkeley UNIX) TCP was mis-behaving or if it could be tuned to work better under abysmal network conditions. The answer to both of these questions was "yes".

Since that time, we have put seven new algorithms into the 4BSD TCP:

- (i) round-trip-time variance estimation
- (ii) exponential retransmit timer backoff
- (iii) slow-start
- (iv) more aggressive receiver ack policy
- (v) dynamic window sizing on congestion
- (vi) Karn's clamped retransmit backoff
- (vii) fast retransmit

Our measurements and the reports of beta testers suggest that the final product is fairly good at dealing with congested conditions on the Internet.

*This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

¹The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

This paper is a brief description of (i) – (v) and the rationale behind them. (vi) is an algorithm recently developed by Phil Karn of Bell Communications Research, described in [KP87]. (vii) is described in a soon-to-be-published RFC.

Algorithms (i) – (v) spring from one observation: The flow on a TCP connection (or ISO TP-4 or Xerox NS SPP connection) should obey a 'conservation of packets' principle. And, if this principle were obeyed, congestion collapse would become the exception rather than the rule. Thus congestion control involves finding places that violate conservation and fixing them.

By 'conservation of packets' I mean that for a connection 'in equilibrium', i.e., running stably with a full window of data in transit, the packet flow is what a physicist would call 'conservative': A new packet isn't put into the network until an old packet leaves. The physics of flow predicts that systems with this property should be robust in the face of congestion. Observation of the Internet suggests that it was not particularly robust. Why the discrepancy?

There are only three ways for packet conservation to fail:

1. The connection doesn't get to equilibrium, or
2. A sender injects a new packet before an old packet has exited, or
3. The equilibrium can't be reached because of resource limits along the path.

In the following sections, we treat each of these in turn.

1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the sender uses acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,

dl.acm.org

Congestion Avoidance and Control

Van Jacobson*

University of California
Lawrence Berkeley Laboratory
Berkeley, CA 94720
van@helios.ee.lbl.gov

In October of '86, the Internet had the first of what became a series of 'congestion collapses'. During this period, the data throughput from LBL to UC Berkeley (sites separated by 400 yards and three IMP hops)

This paper is a brief description of (i) – (v) and the rationale behind them. (vi) is an algorithm recently developed by Phil Karn of Bell Communications Research, described in [KP87]. (vii) is described in a soon-to-be-

- On any timeout, set *cwnd* to half the current window size (this is the multiplicative decrease).
- On each ack for new data, increase *cwnd* by $1/cwnd$ (this is the additive increase).¹⁰
- When sending, send the minimum of the receiver's advertised window and *cwnd*.

Our measurements and the reports of beta testers suggest that the final product is fairly good at dealing with congested conditions on the Internet.

* This work was supported in part by the U.S. Department of Energy under Contract Number DE-AC03-76SF00098.

¹ The algorithms and ideas described in this paper were developed in collaboration with Mike Karels of the UC Berkeley Computer System Research Group. The reader should assume that anything clever is due to Mike. Opinions and mistakes are the property of the author.

Permission to copy without fee all or part of this material is granted provided that the copies are not made or distributed for direct commercial advantage, the ACM copyright notice and the title of the publication and its date appear, and notice is given that copying is by permission of the Association for Computing Machinery. To copy otherwise, or to republish, requires a fee and/or specific permission.

1988 ACM 0-89791-279-9/88/008/0314

2. A sender injects a new packet before an old packet has exited, or

3. The equilibrium can't be reached because of resource limits along the path.

In the following sections, we treat each of these in turn.

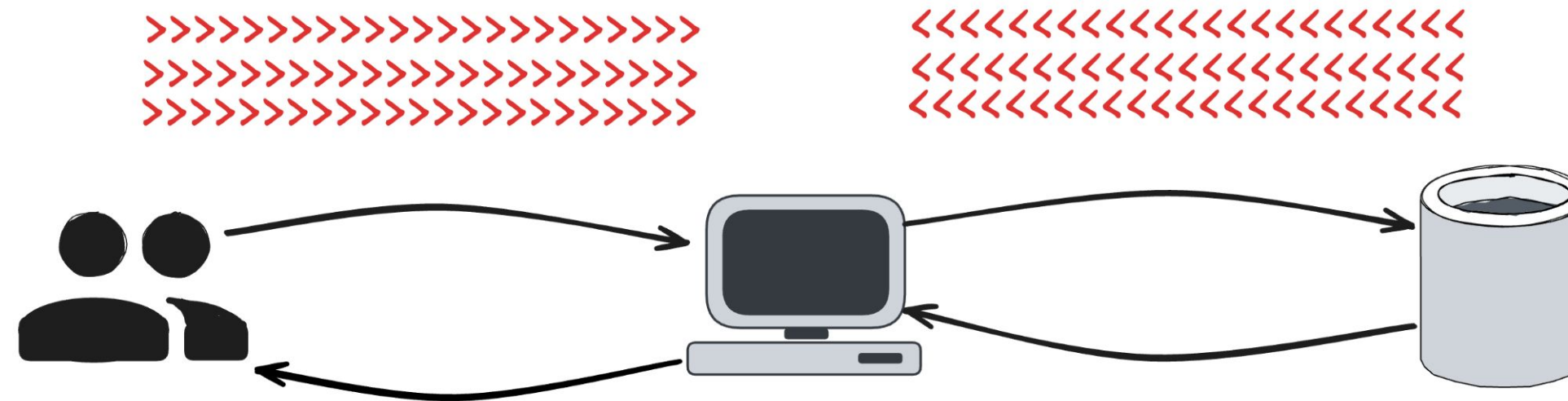
1 Getting to Equilibrium: Slow-start

Failure (1) has to be from a connection that is either starting or restarting after a packet loss. Another way to look at the conservation property is to say that the sender uses acks as a 'clock' to strobe new packets into the network. Since the receiver can generate acks no faster than data packets can get through the network,

Resilience By Design is an Insurance Policy

Your Service Is at Risk

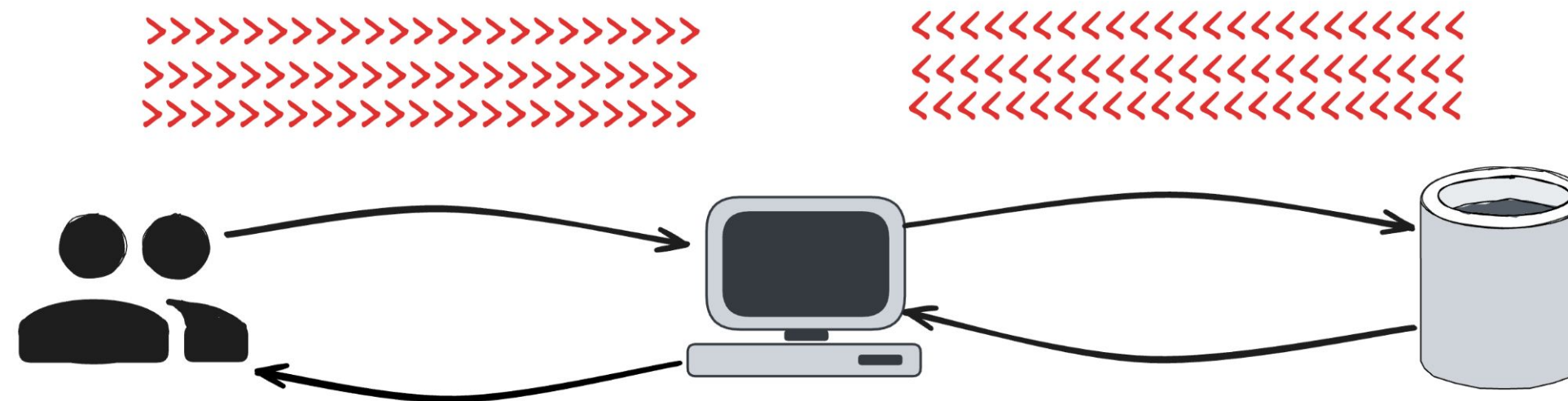
- Sudden burst of traffic
- Slow dependency
- Latency bugs
- ...



Your Service Is at Risk

- Sudden burst of traffic
- Slow dependency
- Latency bugs
- ...

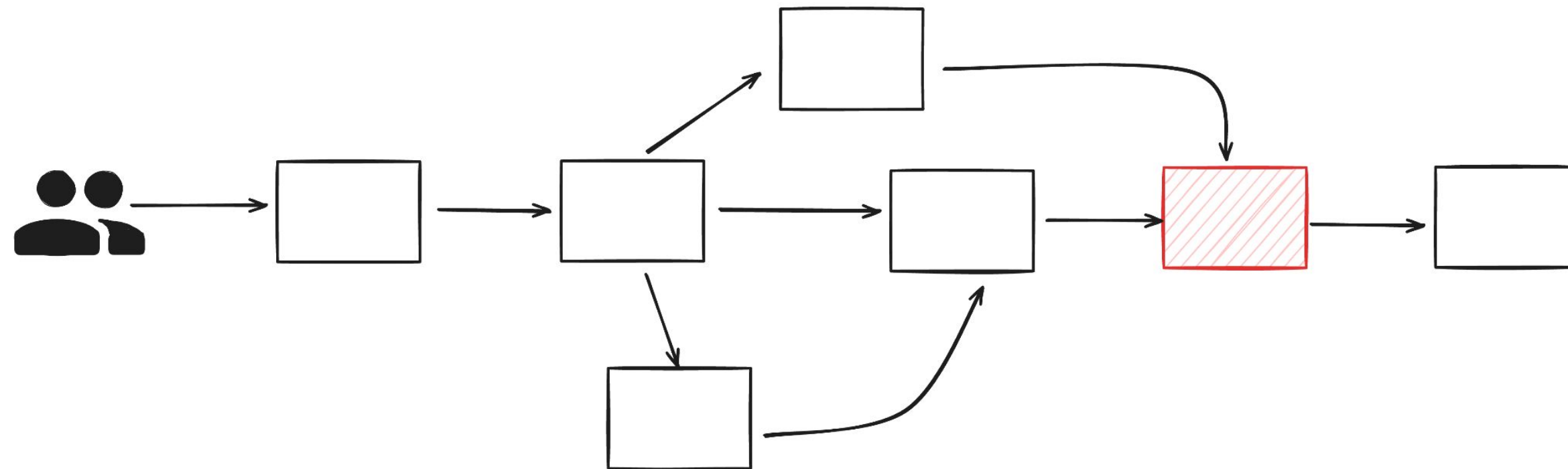
Flow metrics detects and protects your service



Your ~~Service~~ System Is at Risk

- Sudden burst of traffic
- Slow dependency
- Latency bugs
- ...

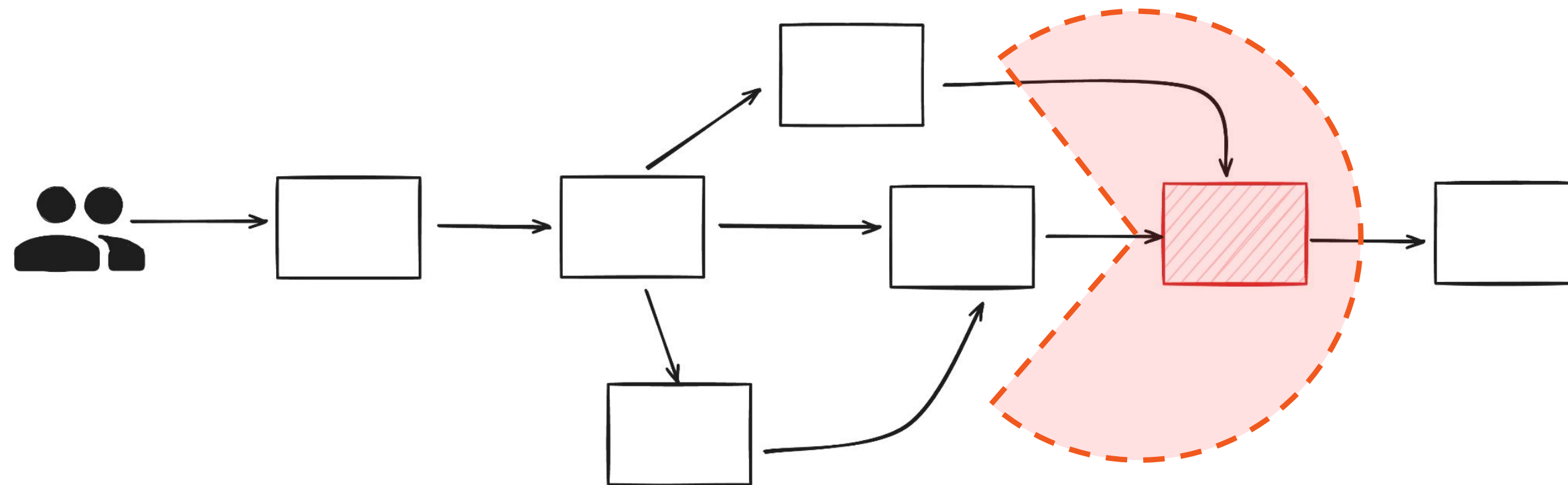
Flow metrics detects and protects your **system** against **cascading failures**



Your ~~Service~~ System Is at Risk

- Sudden burst of traffic
- Slow dependency
- Latency bugs
- ...

Flow metrics detects and protects your **system** against **cascading failures**



An Insurance Policy: Resilience With Flow Metrics

- **Acknowledgement of limits of a system**

An Insurance Policy: Resilience With Flow Metrics

- **Acknowledgement of limits** of a system
- Effective **communication** during outages

An Insurance Policy: Resilience With Flow Metrics

- **Acknowledgement of limits** of a system
- Effective **communication** during outages
- **Independent** of system characteristics

An Insurance Policy: Resilience With Flow Metrics

- **Acknowledgement of limits** of a system
- Effective **communication** during outages
- **Independent** of system characteristics
- **Simple** to implement with little overhead

An Insurance Policy: Resilience With Flow Metrics

- **Acknowledgement of limits** of a system
- Effective **communication** during outages
- **Independent** of system characteristics
- **Simple** to implement with little overhead

**Embed Flow of Business Value
into System Design**

github.com/mourjo/monster-scale-2025

Thank you!

Questions?

monster-scale-2025@mourjo.me

[linkedin.com/in/mourjo](https://www.linkedin.com/in/mourjo)

<https://mourjo.me>

github.com/mourjo

What Flow Metrics Teaches Us About Designing Resilient Systems

github.com/mourjo/monster-scale-2025